

Trabajo Fin de Máster
Máster en Seguridad en Tecnologías de la
Información y las Comunicaciones

Análisis de la capacidad de Velociraptor para la
detección de ataques HIDS de la matriz MITRE
ATT&CK Enterprise

Autor: Julio García-Amorena Reina

Tutor: Francisco Javier Muñoz Calle

Dpto. de Ingeniería Telemática
Escuela Técnica Superior de Ingeniería
Universidad de Sevilla

Sevilla, 2026



Trabajo Fin de Máster

Máster en Seguridad en Tecnologías de la Información y las Comunicaciones

Análisis de la capacidad de Velociraptor para la detección de ataques HIDS de la matriz MITRE ATT&CK Enterprise

Autor:

Julio García-Amorena Reina

Tutor:

Francisco Javier Muñoz Calle

Profesor Colaborador

Dpto. de Ingeniería Telemática

Escuela Técnica Superior de Ingeniería

Universidad de Sevilla

Sevilla, 2026

Trabajo Fin de Máster: Análisis de la capacidad de Velociraptor para la detección de ataques
HIDS de la matriz MITRE ATT&CK Enterprise

Autor: Julio García-Amorena Reina

Tutor: Francisco Javier Muñoz Calle

El tribunal nombrado para juzgar el proyecto arriba indicado, compuesto por los siguientes miembros:

Presidente:

Vocales:

Secretario:

Acuerdan otorgarle la calificación de:

Sevilla, 2026

Secretario del Tribunal

Dedico este Trabajo de Fin de Máster a mi familia. A mi padre, Agustín García-Amorena, por haberme apoyado siempre, por ser una inspiración en mi vida y un ejemplo constante y a mi madre, Sara Reina, por su protección, cariño y presencia incondicional. También quiero dedicar este trabajo a mis primos, Guillermo Reina y Jesús Gómez de la Mata, mis mejores amigos y compañeros de vida; y a mi pareja, Irene Reyes, quien no solo me animó a tomar este camino, sino que me ha acompañado en cada paso.

Finalmente, quiero recordar de forma especial a mi padrino, Juan García-Amorena, y nuevamente a mi padre, Agustín García-Amorena, quienes me inculcaron el amor por el conocimiento técnico y han sido dos de las personas que más he admirado.

Agradecimientos

Quiero expresar mi más sincero agradecimiento a todas las personas que me han apoyado, aconsejado y escuchado a lo largo del desarrollo de este proyecto.

En primer lugar, a mi familia: a mi padre, Agustín, pieza fundamental en mi vida y ejemplo a seguir y a mi madre, Sara, por estar siempre a mi lado y ayudarme a forjar mi carácter. A mi pareja, Irene, quiero agradecerle especialmente que me diera el valor para seguir el camino que realmente deseaba, ayudándome a superar prejuicios, dudas y complejos. Su apoyo incondicional, su confianza en mí y su constante motivación han sido esenciales para alcanzar mis objetivos. A mis amigos y compañeros, Jorge, Miguel, Alimar y Reisaac, gracias por acompañarme durante este año, por vuestro apoyo y por haber hecho más llevadero el curso. A mis primos, Guillermo y Jesús, gracias por estar siempre disponibles cuando he necesitado una corrección, un consejo o un punto de vista sincero. Les estoy profundamente agradecido.

Finalmente, quiero expresar mi más sincero agradecimiento a mi tutor, Francisco Javier Muñoz Calle, por su orientación, disponibilidad y apoyo durante el desarrollo de este Trabajo de Fin de Máster. Su implicación, cercanía y criterio han sido fundamentales para enfocar y completar este proyecto, así como para convertirlo en una propuesta plenamente alineada con mis intereses académicos y profesionales.

A todos, mi más sincero agradecimiento.

Por último, al igual que cerré mi Trabajo de Fin de Grado con los versos de “Caminante, no hay camino”, pertenecientes a *Proverbios y cantares XXIX*, del ilustre poeta Antonio Machado, me gustaría cerrar este Trabajo de Fin de Máster con los siguientes versos del mismo autor:

*Todo pasa y todo queda,
pero lo nuestro es pasar;
pasar haciendo caminos,
caminos sobre la mar.*

Antonio Machado, *Proverbios y cantares XXIX*

Resumen

El incremento de los ciberataques dirigidos a infraestructuras corporativas ha puesto de manifiesto las limitaciones de los enfoques de seguridad basados exclusivamente en la prevención. En los entornos actuales, donde los atacantes emplean técnicas cada vez más sofisticadas y adaptativas, se asume que los mecanismos defensivos perimetrales pueden ser eventualmente superados. En consecuencia, resulta esencial disponer de capacidades avanzadas de detección y respuesta a nivel de endpoint. En este contexto, las tecnologías de detección de intrusiones basadas en host (HIDS), así como soluciones más avanzadas como EDR (Endpoint Detection and Response) y XDR (Extended Detection and Response), desempeñan un papel fundamental al proporcionar visibilidad sobre la actividad del sistema operativo, los procesos en ejecución y otros indicadores de compromiso.

El presente Trabajo Fin de Máster, titulado “Análisis de la capacidad de Velociraptor para la detección de ataques HIDS de la matriz MITRE ATT&CK Enterprise”, evalúa la capacidad de Velociraptor para actuar como aproximación HIDS/DFIR en Windows mediante artifacts públicos y personalizados. La evaluación se centra en técnicas MITRE ATT&CK Enterprise ejecutadas de forma controlada en laboratorio, diferenciando ejecución de la técnica, visibilidad host-based, detección mediante CLIENT_EVENT, alerta, evidencia forense y salida externa.¹

La memoria consolida una arquitectura de artifacts P1/P2/P3/P4 y un router SERVER_EVENT limitado al enrutamiento, persistencia y notificación. En el subconjunto experimental TEC-001 a TEC-009, los artifacts custom CLIENT_EVENT detectaron 9/9 técnicas y emitieron/exportaron 379 filas CLIENT_EVENT, incluidas emisiones repetidas: P1=19, P2=118, P3=109 y P4=133. La campaña de falsos positivos ejecutó 10/10 pruebas benignas con 0 hits; 82 filas no reconciliadas fueron excluidas y no se computaron como falsos positivos definitivos. En seis campañas con artifacts públicos hubo cinco resultados positivos de visibilidad y uno no concluyente; Hayabusa CH aportó detección específica en 3/9 técnicas. Wazuh base detectó 3/9 y Wazuh custom 4/9, con una unión complementaria de 7/9 que no constituye una quinta solución. El benchmark registró 9/9 runs válidos; en VR_TEC_RUNNER se observaron 0,95 % de CPU media, 54 % de pico, 57,42 MB de RAM media y 57,97 MB de pico, como coste agregado del cliente y no de P1–P4 por separado. ETW quedó NO CONCLUYENTE; TrackNetwork no acreditó atribución fiable de proceso; y el HTTP 200 de TEC-009 procede del runner independiente, no de una detección CLIENT_EVENT. Los resultados no se extrapolan a toda MITRE ATT&CK ni a producción.

Palabras clave: ciberseguridad, HIDS, Velociraptor, MITRE ATT&CK, ransomware, DFIR, detección de intrusiones, monitorización de endpoints.

¹ MITRE ATT&CK. “Enterprise Matrix”: <https://attack.mitre.org/matrices/enterprise/>

Abstract

The increasing number of cyberattacks targeting corporate infrastructures has highlighted the limitations of security approaches based solely on prevention. In modern threat environments, where attackers employ increasingly sophisticated and adaptive techniques, it is assumed that perimeter defense mechanisms may eventually be bypassed. Consequently, it is essential to develop advanced detection and response capabilities at the endpoint level. In this context, host-based intrusion detection technologies (HIDS), together with more advanced solutions such as EDR (*Endpoint Detection and Response*) and XDR (*Extended Detection and Response*), play a fundamental role by providing visibility into operating system activity, running processes, and other indicators of compromise.

This Master's Thesis, entitled “Analysis of Velociraptor's Capability to Detect HIDS Attacks from the MITRE ATT&CK Enterprise Matrix”, evaluates Velociraptor as a Windows HIDS/DFIR-oriented platform using public and custom artifacts. The evaluation is based on controlled MITRE ATT&CK Enterprise techniques executed in a laboratory environment and explicitly separates technique execution, host-based visibility, CLIENT_EVENT detection, alerting, forensic evidence and external output.

The thesis consolidates a P1/P2/P3/P4 artifact architecture and a SERVER_EVENT router restricted to routing, persistence and notification. Within the TEC-001 to TEC-009 experimental subset, the custom CLIENT_EVENT artifacts detected 9/9 techniques and emitted/exported 379 CLIENT_EVENT rows, including repeated emissions: P1=19, P2=118, P3=109 and P4=133. The false-positive campaign completed 10/10 benign tests with 0 definitive hits; 82 unreconciled rows were excluded from the definitive false-positive count. Six public-artifact campaigns produced five positive visibility results and one inconclusive result; Hayabusa CH provided technique-specific detection for 3/9 techniques. Wazuh base detected 3/9 and Wazuh custom 4/9, with a complementary 7/9 union that is not treated as a fifth solution. The benchmark recorded 9/9 valid runs; VR_TEC_RUNNER showed 0.95% average CPU, a 54% peak, 57.42 MB average RAM and a 57.97 MB peak, as aggregate client cost rather than per-artifact P1–P4 consumption. ETW remained INCONCLUSIVE; TrackNetwork did not provide reliable process attribution; and the TEC-009 HTTP 200 originated from the independent runner, not from a CLIENT_EVENT detection. The results are not extrapolated to the full MITRE ATT&CK matrix or to production.

Keywords: cybersecurity, HIDS, Velociraptor, MITRE ATT&CK, ransomware, DFIR, intrusion detection, endpoint monitoring.

Índice

Capítulo I: Introducción.....	1
1.1. Motivación y descripción del problema.....	1
1.2. Objetivos del proyecto	2
1.3. Estado del arte.....	3
1.3.1. Detección en endpoint: HIDS, EDR y XDR.....	3
1.3.2. MITRE ATT&CK como marco de referencia.....	3
1.3.3. Velociraptor: naturaleza, capacidades y uso habitual.....	4
1.3.4. Criterios de evaluación de herramientas de detección	5
Capítulo II: Planificación	6
2.1. Estimación de tiempos y plazos	6
2.2. Presupuesto	7
2.3. Tabla de planificación general	7
2.4. Tabla de planificación por capítulos	8
2.4.1. Planificación del capítulo I	8
2.4.2. Planificación del capítulo II	9
2.4.3. Planificación del capítulo III.....	10
2.4.4. Planificación del capítulo IV.....	10
2.4.5. Planificación del capítulo V	11
2.4.6. Planificación del capítulo VI.....	11
2.4.7. Planificación del capítulo VII	12
2.4.8. Planificación del capítulo VIII.....	13
2.4.9. Planificación del capítulo IX.....	14
2.4.10. Planificación del capítulo X.....	14
2.5. Diagrama de Gantt	15
2.5.1. Diagrama de Gantt-Capítulos uno y dos.....	16
2.5.2. Diagrama de Gantt-Capítulo tres	16
2.5.3. Diagrama de Gantt-Capítulo cuatro	16

2.5.4. Diagrama de Gantt-Capítulo cinco	16
2.5.5. Diagrama de Gantt-Capítulo seis	17
2.5.6. Diagrama de Gantt-Capítulo siete.....	18
2.5.7. Diagrama de Gantt-Capítulos ocho y nueve	18
Capítulo III: Fundamentos teóricos	19
3.1. Enfoques de detección y análisis en endpoint.....	19
3.1.1. Detección en HIDS tradicionales.....	19
3.1.2. Detección en DFIR	20
3.1.3. Detección en EDR.....	20
3.1.4. Posición de Velociraptor frente a estos enfoques	21
3.2. Arquitectura interna Velociraptor.....	22
3.2.1. Diferencias entre capas de Velociraptor	22
3.2.2. Flujo cliente-servidor y formatos de datos.....	24
3.3. Artifacts y modelo de ejecución.....	26
3.3.1. Qué es un artifact	26
3.3.2. Tipos de artifacts según el contexto de ejecución	26
3.3.3. Collections, Hunts y Monitoring.....	27
3.4. Velociraptor Query Language (VQL)	28
3.4.1. Estructura de una consulta VQL	28
3.4.2. Query Engine, plugins y lazy evaluation	29
3.4.3. Event queries y monitorización continua.....	30
3.5. Telemetría disponible en el endpoint	31
3.6. Limitaciones de visibilidad en HIDS/DFIR.....	32
3.6.1. Visibilidad parcial del comportamiento del atacante	32
3.6.2. Limitaciones de visibilidad en DFIR	33
Capítulo IV: Entorno de trabajo, metodología y selección de técnicas	34
4.1. Diseño del entorno	35
4.1.1. Configuración base de la máquina virtual.....	35
4.1.2. Componentes técnicos desplegados	36

4.1.3. Identificación del cliente y alcance del laboratorio.....	36
4.2. Instalación, instrumentación y configuración base	37
4.2.1. Servidor Velociraptor	37
4.2.2. Agente Velociraptor.....	37
4.2.3. Instrumentación con Sysmon, PowerShell 4104 y eventos Windows	39
4.3. Metodología experimental	40
4.3.1. Principios metodológicos.....	40
4.3.2. Organización del entorno de trabajo	40
4.3.3. Control de versiones y trazabilidad del desarrollo	41
4.3.4. Flujo general de trabajo.....	42
4.3.5. Matriz principal de control experimental.....	43
4.3.6. Registro de evidencias	44
4.3.7. Diseño de pruebas controladas.....	44
4.3.8. Scripts de laboratorio validados	45
4.4. Selección de técnicas MITRE ATT&CK	45
4.4.1. Criterios de selección.....	46
4.4.2. Técnicas seleccionadas	46
4.4.3. Tácticas ATT&CK representadas por el subconjunto evaluado.....	47
4.4.4. Decisión metodológica sobre TEC-009	47
4.5. Estrategia de artifacts y monitorización.....	49
4.5.1. Artifacts públicos considerados	49
4.5.2. Brechas detectadas y necesidad de artifacts personalizados	50
4.6. Benchmark y alertabilidad	52
4.6.1. Validación operativa mediante alertas.....	52
4.7. Decisiones metodológicas cerradas	54
Capítulo V: Desarrollo experimental y ejecución de pruebas	55
5.1. Escenarios de ataque	55
5.2. Desarrollo experimental por técnica	56
5.2.1. TEC-001 - Ejecución mediante PowerShell	56

5.2.2. TEC-002 - Ejecución mediante Windows Command Shell	57
5.2.3. TEC-003 - Creación de tarea programada	58
5.2.4. TEC-004 - Persistencia mediante claves Run del registro	60
5.2.5. TEC-005 - Ejecución mediante servicio Windows	61
5.2.6. TEC-006 - Descubrimiento de software de seguridad	62
5.2.7. TEC-007 - Cifrado de datos	63
5.2.8. TEC-008 – Destrucción de datos	65
5.2.9. TEC-009 - Staging, archivado ZIP y exfiltración HTTP controlada.....	66
5.2.10. Runner de ejecución global de técnicas	69
5.2.11. Runner de falsos positivos	71
5.3. Recopilación de telemetría.....	73
5.3.1. Telemetría captada en el sistema.....	74
5.3.2. Telemetría captada en Velociraptor	75
5.3.3. Salida externa de alertado (JSONL/Discord).....	76
5.4. Configuración y uso de artifacts	92
5.4.1. Artifacts de fábrica utilizados	92
5.4.2. Artifacts personalizados	94
5.4.3. Limitaciones encontradas durante la configuración.....	94
Capítulo VI: Resultados de detección	96
6.1. Resultados por técnica MITRE	96
6.2. Cobertura de detección	97
6.3. Casos sin detección y detección parcial	99
6.4. Evidencias observadas y falsos positivos.....	100
6.5. Calidad de datos y limitaciones	101
6.6. Resultados definitivos de artifacts públicos.....	102
Capítulo VII: Evaluación de Velociraptor y benchmark de rendimiento	104
7.1. Metodología de benchmark.....	104
7.2. Impacto en CPU	105
7.3. Impacto en memoria	105

7.4. Impacto en disco	106
7.5. Relación detección / coste.....	106
Capítulo VIII: Validación comparativa con Wazuh como HIDS.....	107
8.1. Objetivo de la comparación	107
8.2. Arquitectura del entorno Wazuh.....	108
8.3. Instalación del servidor Wazuh	109
8.4. Instalación y validación del agente Windows	110
8.5. Fuentes de telemetría configuradas.....	112
8.6. Reglas personalizadas TFM en Wazuh	113
8.7. Metodología de campañas separadas	115
8.8. Campaña B: Wazuh base.....	116
8.9. Campaña C: Wazuh custom	119
8.10. Evidencias gráficas en el dashboard de Wazuh.....	124
8.11. Comparación Wazuh – Velociraptor.....	127
8.12. Limitaciones observadas.....	128
8.13. Registro y almacenamiento de evidencias	129
Capítulo IX: Discusión y trabajo futuro	130
9.1. Discusión integrada de resultados.....	130
9.2. Ampliación de técnicas MITRE.....	130
9.3. Desarrollo de nuevos artifacts.....	131
9.4. Ciclo de vida y trazabilidad de los artifacts	132
9.5. Automatización de detecciones.....	132
9.6. Integración SIEM/SOC y despliegue en entornos reales	132
Capítulo X: Conclusiones	133
10.1. Conclusiones principales	133
10.2. Respuesta a objetivos.....	134
10.3. Limitaciones identificadas	135
10.4. Valor del enfoque	135
Capítulo XI: Referencias bibliográficas y anexos	137

11.1. Finalidad de los anexos	139
11.2. Anexo A. Técnicas MITRE seleccionadas y trazabilidad	139
11.3. Anexo B. Configuración del laboratorio	141
11.4. Anexo C. Fuentes de telemetría y criterio de evidencia	141
11.5. Anexo D. Artifacts Velociraptor validados.....	142
11.6. Anexo E. Runners, scripts y ground truth	143
11.7. Anexo F. Evidencias y datasets finales.....	143
11.8. Anexo G. Resultados completos de visibilidad, detección y falsos positivos.....	144
11.9. Anexo H. Benchmark de rendimiento	145
11.10. Anexo I. Validación comparativa con Wazuh.....	146
11.11. Anexo J. TEC-009 - Exfiltración HTTP controlada en laboratorio.....	146
11.12. Anexo K. Glosario técnico	147
11.13. Anexo L. Checklist de defensa metodológica	148
11.14. Anexo M. Declaración responsable sobre el uso de inteligencia artificial.....	149

Índice de tablas

Tabla 1: Tipos de artifacts según su modo de ejecución	4
Tabla 2: Planificación general.....	8
Tabla 3: Planificación del capítulo I	9
Tabla 4: Planificación del capítulo II	9
Tabla 5: Planificación del capítulo III.....	10
Tabla 6: Planificación del capítulo IV.....	11
Tabla 7: Planificación del capítulo V	11
Tabla 8: Planificación del capítulo VI.....	11
Tabla 9: Planificación del capítulo VII	12
Tabla 10: Planificación del capítulo VIII	13
Tabla 11: Planificación del capítulo IX.....	14
Tabla 12: Planificación del capítulo X.....	15
Tabla 13: Funciones, limitaciones y riesgos del enfoque HIDS	20
Tabla 14: Funciones, limitaciones y riesgos del enfoque DFIR.....	20
Tabla 15: Funciones, limitaciones y riesgos del enfoque EDR.....	21
Tabla 16: Posición de Velociraptor frente a HIDS, DFIR y EDR	21
Tabla 17: Capas funcionales de la arquitectura de Velociraptor	23
Tabla 18: Formatos de datos en el flujo cliente-servidor de Velociraptor.....	25
Tabla 19: Campos principales de definición de artifacts.....	26
Tabla 20: Tipos de artifacts según el contexto de ejecución	27
Tabla 21: Diferencias entre collections, hunts, monitoring y SERVER_EVENT.....	27
Tabla 22: Fuentes principales de telemetría y utilidad en detección.....	31
Tabla 23: Diferencia entre visibilidad, detección, alerta y salida externa.....	32
Tabla 24: Criterios metodológicos del capítulo IV	34
Tabla 25: Parámetros iniciales de la máquina virtual.....	35
Tabla 26: Componentes técnicos principales	36
Tabla 27: Identificación del endpoint usado en el laboratorio	36
Tabla 28: Parámetros funcionales del servidor Velociraptor.....	37
Tabla 29: Fuentes de telemetría habilitadas y técnicas relacionadas	39
Tabla 30: Tratamiento metodológico de Sysmon Event ID 26	39
Tabla 31: Principios metodológicos aplicados en la fase experimental	40
Tabla 32: Criterios de control de versiones y trazabilidad del proyecto	41
Tabla 33: Snapshots utilizados para control del laboratorio	42
Tabla 34: Flujo general de trabajo por técnica	43

Tabla 35: Hojas principales de la matriz de control experimental	43
Tabla 36: Tipos de evidencias documentadas por técnica.....	44
Tabla 37: Controles de seguridad aplicados a las pruebas	45
Tabla 38: Scripts de laboratorio y papel metodológico.....	45
Tabla 39: Criterios de validación de los scripts de laboratorio	45
Tabla 40: Criterios usados para seleccionar técnicas MITRE ATT&CK	46
Tabla 41: Técnicas MITRE ATT&CK seleccionadas para la fase experimental.....	47
Tabla 42: Tácticas ATT&CK representadas por el subconjunto evaluado empleando denominador 14	47
Tabla 43: Criterio de interpretación de TEC-009.....	48
Tabla 44: Artifacts públicos considerados en la evaluación metodológica	50
Tabla 45: Brechas y limitaciones de los artifacts públicos en la evaluación.....	51
Tabla 46: Escenarios ejecutados en el benchmark de rendimiento	52
Tabla 47: Escenarios de ataque evaluados	56
Tabla 48: Casos benignos incluidos en el runner de falsos positivos.....	73
Tabla 49: Telemetría esperada por técnica experimental	75
Tabla 50: Resultados definitivos del runner de falsos positivos	78
Tabla 51: Artifacts de fábrica y públicos considerados durante la fase experimental.....	93
Tabla 52: Artifacts custom por perfil y función operativa	94
Tabla 53: Resultados finales de Velociraptor por técnica MITRE evaluada	97
Tabla 54: Distribución de filas CLIENT_EVENT emitidas/exportadas por perfil custom.....	98
Tabla 55: Distribución de filas CLIENT_EVENT por técnica en la campaña real	99
Tabla 56: Resultados definitivos de las campañas con artifacts públicos	102
Tabla 57: Criterios empleados para evaluar Wazuh como solución de comparación	108
Tabla 58: Parámetros principales del entorno Wazuh desplegado	108
Tabla 59: Fuentes de telemetría utilizadas por Wazuh en la campaña	113
Tabla 60: Reglas personalizadas Wazuh utilizadas para aproximar los perfiles P1-P4 de Velociraptor	114
Tabla 61: Campañas definidas para la validación comparativa	116
Tabla 62: Resultados de visibilidad y alerta de la campaña Wazuh base.....	118
Tabla 63: Resultados agregados de la campaña Wazuh custom.....	122
Tabla 64: Distribución de reglas TFM activadas durante la campaña custom.....	122
Tabla 65: Comparación final entre Wazuh base, Wazuh custom y Velociraptor custom	127
Tabla 66: Mapeo de severidad usado entre perfiles Velociraptor y niveles Wazuh	128
Tabla 67: Técnicas MITRE ATT&CK propuestas para trabajo futuro.....	131
Tabla 68: Correspondencia entre preguntas de investigación, resultados y limitaciones	134

Índice de figuras

Figura 1: Monitorización con Toggl Track	6
Figura 2: Diagrama de Gantt general.....	16
Figura 3: Planificación Gantt de los capítulos I y II	16
Figura 4: Planificación Gantt del capítulo III	16
Figura 5: Planificación Gantt del capítulo IV	16
Figura 6: Planificación Gantt del capítulo V.....	17
Figura 7: Planificación Gantt del capítulo VI	17
Figura 8: Planificación Gantt del capítulo VII.....	18
Figura 9: Planificación Gantt de los capítulos VIII y IX	18
Figura 10: Arquitectura interna básica VR.....	22
Figura 11: Flujo de datos de Velociraptor	25
Figura 12: Instalación del agente Velociraptor.....	38
Figura 13: Datos recibidos desde los agentes Velociraptor.....	38
Figura 14: Estructura de directorios de la máquina física.....	41
Figura 15: Directorio de la máquina virtual	41
Figura 16: Arquitectura del flujo de alertas	53
Figura 17: Evidencia de ejecución de TEC-001	56
Figura 18: Evidencia de ejecución de TEC-002	57
Figura 19: Evidencia de la tarea programada TEC-003.....	59
Figura 20: Evidencia de persistencia mediante clave Run en TEC-004	60
Figura 21: Evidencia de ejecución del servicio TEC-005.....	61
Figura 22: Evidencia de descubrimiento de software de seguridad en TEC-006	62
Figura 23: Evidencia de cifrado controlado en TEC-007	64
Figura 24: Script de borrado de fichero	65
Figura 25: Evidencia del receptor HTTP controlado	67
Figura 26: Evidencia de ejecución de TEC-009	68
Figura 27: Evidencias obtenidas en Velociraptor.....	75
Figura 28: Resumen del runner global de técnicas	96
Figura 29: Distribución de las 379 filas CLIENT_EVENT emitidas/exportadas, incluidas emisiones repetidas por perfil custom.....	97
Figura 30: Arquitectura de artifacts custom y router	98
Figura 31: Captura textual de la evidencia de upload HTTP controlado en TEC-009.....	100
Figura 32: Campaña definitiva de falsos positivos	100
Figura 33: Controles de calidad aplicados a los resultados de detección y FP	101

Figura 34: Resultado de las seis campañas con artifacts públicos	103
Figura 35: Coste operativo observado del cliente Velociraptor por escenario	105
Figura 36: Controles de calidad del benchmark final	106
Figura 37: Validación del agente LT29 como agente activo desde agent_control	109
Figura 38: Panel de acceso al dashboard de Wazuh.....	110
Figura 39: Validación de conectividad e instalación del agente Wazuh desde PowerShell	111
Figura 40: Comprobación del servicio WazuhSvc en ejecución en el endpoint Windows	111
Figura 41: Reactivación de reglas personalizadas TFM y servicio wazuh-manager activo.....	115
Figura 42: Resumen de la campaña Wazuh base, con ausencia de reglas TFM 110xxx.....	119
Figura 43: Cobertura específica comparada sobre nueve técnicas experimentales	123
Figura 44: Resumen de campaña Wazuh custom con reglas TFM activadas.....	123
Figura 45: Resumen de severidades en el dashboard tras la campaña	124
Figura 46: Dashboard de Wazuh filtrado por agent.name LT29	124
Figura 47: Visibilidad de PowerShell 4104 en Wazuh.....	125
Figura 48: Visibilidad de Sysmon Event ID 1, asociado a creación de procesos.....	125
Figura 49: Visibilidad de Sysmon Event ID 11, asociado a creación de ficheros.....	126
Figura 50: Ejemplo de detección custom mediante la regla 110202.....	126
Figura 51: Ausencia de resultados para rule.id 110201 en la campaña custom	126
Figura 52: Vista de alertas de severidad alta/crítica generadas en Wazuh	127

Índice de códigos

Código 1: Ejemplo de consulta VQL	28
Código 2: Ejemplo de obtención de información mediante VQL.....	28
Código 3: Ejemplo de consulta de claves del registro mediante VQL.....	29
Código 4: Ejemplo de evaluación diferida en VQL.....	29
Código 5: Control del laboratorio	41
Código 6: Procedimiento de ejecución de TEC-001	56
Código 7: Procedimiento de ejecución de TEC-002.....	57
Código 8: Procedimiento de ejecución de TEC-003.....	59
Código 9: Procedimiento de ejecución de TEC-004.....	60
Código 10: Procedimiento de ejecución de TEC-005.....	61
Código 11: Procedimiento de ejecución de TEC-006 mediante PowerShell	62
Código 12: Procedimiento de ejecución de TEC-006 mediante WMIC	63
Código 13: Procedimiento de ejecución de TEC-007.....	64
Código 14: Procedimiento de ejecución de TEC-008.....	65
Código 15: Ejecución del receptor HTTP controlado	67
Código 16: Procedimiento de ejecución de TEC-009.....	68
Código 17: Ejecución del runner global de técnicas TEC-001 a TEC-009.....	69
Código 18: Ejecución del runner de falsos positivos	71
Código 19: Instalación base de dependencias y despliegue all-in-one de Wazuh.....	109
Código 20: Validación de conectividad desde Windows hacia Wazuh	110
Código 21: Instalación del agente Wazuh en Windows	111
Código 22: Fuentes eventchannel configuradas en el agente Windows.....	112
Código 23: Reinicio y comprobación del agente Wazuh en Windows	113
Código 24: Deshabilitación de la primera versión de reglas tras problemas de estabilidad	113
Código 25: Validación sintáctica y reinicio tras cargar reglas custom TFM.....	115
Código 26: Desactivación de reglas custom para la campaña Wazuh base	117
Código 27: Marcado de inicio y líneas iniciales de logs en campaña base.....	117
Código 28: Ejecución de runners TEC y falsos positivos desde Windows.....	117
Código 29: Extracción de delta de archives.json y alerts.json en campaña base	118
Código 30: Reactivación de reglas custom TFM para la campaña Wazuh custom.....	120
Código 31: Inicio de campaña Wazuh custom	120
Código 32: Extracción de delta en campaña Wazuh custom.....	121

Acrónimos y abreviaturas

API	Application Programming Interface. Interfaz que permite que distintos programas o sistemas se comuniquen entre sí mediante métodos, peticiones o funciones definidas.
ATT&CK	Adversarial Tactics, Techniques and Common Knowledge. Framework de MITRE para clasificar tácticas, técnicas y procedimientos de atacantes.
CPU	Central Processing Unit.
CSV	Comma-Separated Values. Formato tabular basado en texto donde los campos se separan normalmente por comas o punto y coma. Se usa para exportar datos a hojas de cálculo o herramientas de análisis.
DFIR	Digital Forensics and Incident Response.
EDR	Endpoint Detection and Response.
EID	Event Identifier.
FN	Falso Negativo.
FP	Falso Positivo.
GUI	Graphical User Interface. Interfaz gráfica de usuario.
HIDS	Host-based Intrusion Detection System. Sistema de detección de intrusiones basado en host.
HTTPS	HyperText Transfer Protocol Secure. Versión segura de HTTP que utiliza TLS para cifrar la comunicación entre cliente y servidor.
I/O	Input/Output.
IOA	Indicator of Attack.
IOC	Indicator of Compromise.
JSON	JavaScript Object Notation. Formato de intercambio de datos basado en texto, legible por humanos y ampliamente usado en APIs, configuraciones y exportaciones.
JSONL	JSON Lines. Formato en el que cada línea contiene un objeto JSON independiente. Es útil para almacenar y procesar grandes volúmenes de eventos o registros.
MSE	Mean Squared Error. Error cuadrático medio.
MSI	Microsoft Installer. Formato de paquete de instalación utilizado en Windows para desplegar software, servicios o agentes de forma estructurada.
OS / SO	Operating System / Sistema Operativo. Software base del sistema.
PID	Process Identifier. Identificador único de un proceso en ejecución.

API	Application Programming Interface. Interfaz que permite que distintos programas o sistemas se comuniquen entre sí mediante métodos, peticiones o funciones definidas.
PowerShell 4104	Event ID 4104 de PowerShell, asociado al Script Block Logging. Registra bloques de código PowerShell ejecutados, siendo útil para detectar scripts sospechosos, comandos ofuscados o actividad fileless.
Protobuf	Protocol Buffers. Formato binario de serialización de datos desarrollado por Google, usado para estructurar mensajes de forma compacta y eficiente entre sistemas.
RAM	Random Access Memory.
SIEM	Security Information and Event Management.
Sigma	Lenguaje abierto y genérico para describir reglas de detección de amenazas en logs. Permite expresar patrones de detección reutilizables y adaptables a distintas plataformas SIEM o herramientas de análisis.
SOC	Security Operations Center.
Sysmon	System Monitor. Herramienta de Microsoft Sysinternals para generar telemetría avanzada de Windows.
Sysmon Event ID	Identificador numérico de evento generado por Sysmon. Cada Event ID representa un tipo concreto de actividad del sistema, como creación de procesos, conexiones de red, creación de ficheros o modificaciones del registro.
TEC-00X	Identificador interno usado en el TFM para cada técnica evaluada.
TLS	Transport Layer Security. Protocolo criptográfico utilizado para proteger comunicaciones en red mediante cifrado, autenticación e integridad de los datos transmitidos.
Txxxx	Identificador de técnica MITRE ATT&CK.
VN	Verdadero Negativo.
VP	Verdadero Positivo.
VQL	Velociraptor Query Language.
VR	Velociraptor.
XDR	Extended Detection and Response.
YAML	YAML Ain't Markup Language. Formato de serialización legible empleado para definir artifacts y configuraciones.

Capítulo I: Introducción

Este capítulo presenta el contexto general en el que se desarrolla el trabajo y establece las bases necesarias para comprender el problema. En primer lugar, se expone la motivación del estudio y la descripción del problema, analizando la relevancia de las capacidades de detección en ciberseguridad y el papel de los sistemas HIDS en entornos actuales. A continuación, se definen los objetivos del proyecto, especificando el alcance del análisis y las metas que se persiguen con la evaluación de Velociraptor. Finalmente, se presenta el estado del arte, en el que se revisan las principales tecnologías, enfoques y herramientas existentes en el ámbito de la detección de intrusiones basadas en host y su relación con MITRE ATT&CK.

1.1. Motivación y descripción del problema

Tradicionalmente, las organizaciones han abordado la seguridad desde una perspectiva principalmente preventiva, asumiendo que la inversión en soluciones de protección o personal especializado permitiría evitar estos ciberataques. Sin embargo, la evolución del panorama ha demostrado que incluso las organizaciones con más inversión en ciberseguridad pueden ser comprometidas. Informes recientes muestran que las intrusiones en infraestructuras corporativas continúan aumentando y que los atacantes reducen cada vez más el tiempo necesario para moverse dentro de estas. Según el Data Breach Investigations Report 2025 de Verizon², el 68% de las brechas analizadas involucraron el factor humano o el abuso de credenciales, lo que pone de manifiesto que los mecanismos de prevención tradicionales no siempre son suficientes para evitar el acceso a los sistemas.

Ante esta situación, el modelo defensivo ha evolucionado hacia enfoques en los que se asume que el compromiso puede llegar a producirse, por lo que la prioridad pasa a ser la detección temprana y la respuesta. En este contexto, las tecnologías de monitorización del endpoint, como los HIDS y las soluciones EDR, permiten observar eventos críticos del sistema, como la creación de procesos, la ejecución de scripts o la modificación de configuraciones críticas.

En este marco, resulta de interés analizar el papel de Velociraptor como herramienta de detección basada en host. Aunque su uso está ampliamente consolidado en entornos de DFIR, su arquitectura basada en artifacts permite también implementar mecanismos de monitorización continua. Por ello, este trabajo se centra en evaluar su capacidad para detectar técnicas MITRE ATT&CK a nivel de host, mediante artifacts nativos y personalizados, y en analizar el impacto de dicha monitorización sobre el rendimiento del sistema.

² 2025 Data Breach Investigations Report | Verizon.
<https://www.verizon.com/business/resources/reports/dbir/>

1.2. Objetivos del proyecto

El objetivo principal de este Trabajo Fin de Máster es evaluar la capacidad de Velociraptor para detectar un subconjunto acotado de técnicas MITRE ATT&CK a nivel de host, analizando su posible uso como sistema HIDS en un entorno de laboratorio controlado. Para ello, el estudio se centra en comprobar qué capacidad de detección ofrece la herramienta mediante el uso de artifacts nativos y, únicamente cuando resulte necesario, mediante la personalización de artifacts orientados a técnicas concretas.

A partir de este objetivo general, el trabajo plantea las siguientes cuestiones:

- RQ1: ¿Qué cobertura de detección ofrece Velociraptor, mediante artifacts CLIENT_EVENT personalizados, sobre el subconjunto TEC-001 a TEC-009 evaluado en Windows?
- RQ2: ¿Qué capacidades de visibilidad y detección específica aportan los artifacts públicos evaluados, y qué limitaciones presentan en el subconjunto experimental?
- RQ3: ¿Qué impacto en CPU, memoria y duración introduce el cliente Velociraptor en los escenarios BASELINE_NO_VR, VR_IDLE y VR_TEC_RUNNER?
- RQ4: ¿Cómo responde la arquitectura personalizada P1–P4 a las brechas de detección observadas en los artifacts públicos sin confundir detección, contexto y evidencia forense?
- RQ5: ¿Qué comportamiento presentan los artifacts personalizados ante las pruebas benignas FP-001 a FP-010 y qué limitaciones afectan al cómputo definitivo de falsos positivos?
- RQ6: ¿Cómo deben separarse metodológicamente visibilidad, detección CLIENT_EVENT, alerta, evidencia forense y salida externa SERVER_EVENT/JSONL/Discord?

RQ7: ¿Cómo se compara el modelo HIDS/DFIR de Velociraptor con Wazuh en cobertura específica, alertabilidad y trazabilidad dentro del mismo subconjunto evaluado?

La comparación prevista inicialmente con FortiEDR se sustituyó por Wazuh como adaptación metodológica motivada por la disponibilidad de una plataforma HIDS/SIEM reproducible en el laboratorio. Este cambio no altera el objetivo comparativo y no implica equivalencia directa entre las unidades de ambos sistemas.

La metodología y los resultados mantienen una correspondencia explícita: RQ1, RQ2, RQ4 y RQ5 se responden en el capítulo VI; RQ3, en el capítulo VII; RQ6, mediante la separación de capas descrita en los capítulos V y VI; y RQ7, en el capítulo VIII y en la tabla final de correspondencia.

1.3. Estado del arte

En este apartado se revisan los principales conceptos, tecnologías y herramientas relacionados con la detección de intrusiones en endpoints, con el fin de contextualizar el problema abordado en este trabajo. Para ello, se presentan los enfoques HIDS, EDR y XDR, el papel de MITRE ATT&CK como marco de referencia para el análisis de técnicas de ataque, las capacidades de Velociraptor en entornos de monitorización y DFIR, así como los principales criterios empleados para evaluar este tipo de soluciones.

1.3.1. Detección en endpoint: HIDS, EDR y XDR

Un Host Intrusion Detection System (HIDS) es un sistema de seguridad diseñado para monitorizar la actividad de un host individual con el objetivo de detectar comportamientos potencialmente maliciosos. Para ello analiza eventos generados por el sistema operativo, como la creación de procesos, modificaciones en archivos del sistema, cambios en configuraciones críticas o registros de actividad. Su función principal es identificar indicadores de compromiso o comportamientos anómalos dentro del propio endpoint, sin necesitar acceso a internet.

Las soluciones Endpoint Detection and Response (EDR) amplían el enfoque de los HIDS tradicionales incorporando capacidades adicionales de recopilación continua de telemetría, análisis de comportamiento y respuesta ante incidentes. Mientras que un HIDS se centra en la detección de eventos sospechosos en un host, las plataformas EDR permiten correlacionar eventos, investigar incidentes desde una consola centralizada y aplicar acciones de respuesta sobre los endpoints comprometidos.

Por su parte, las plataformas Extended Detection and Response (XDR) extienden este modelo integrando información procedente de múltiples fuentes de seguridad, como endpoints, red, correo electrónico o servicios en la nube. Su objetivo es correlacionar eventos entre distintos sistemas para mejorar la visibilidad global del entorno y facilitar la detección de ataques que se desarrollan a través de múltiples vectores.

1.3.2. MITRE ATT&CK como marco de referencia

MITRE ATT&CK (Adversarial Tactics, Techniques and Common Knowledge) es un framework que recopila y organiza tácticas y técnicas utilizadas por adversarios. La matriz ATT&CK Enterprise se centra en entornos corporativos y describe comportamientos asociados a distintas fases de un ataque, como ejecución, persistencia, discovery, exfiltración o impacto (MITRE, s. f.-h).

En esta memoria se utiliza ATT&CK como taxonomía de referencia para seleccionar y describir técnicas, no como prueba automática de detección. La asociación de un evento a una técnica requiere correlacionar comportamiento ejecutado, telemetría host-based, lógica del artifact y evidencia conservada (MITRE, s. f.-h).

1.3.3. Velociraptor: naturaleza, capacidades y uso habitual

Velociraptor es una plataforma de código abierto orientada a Digital Forensics and Incident Response (DFIR) que permite la recopilación remota y el análisis de información procedente de múltiples endpoints mediante una arquitectura cliente-servidor. Su diseño permite ejecutar consultas sobre sistemas monitorizados para obtener evidencias, analizar actividad sospechosa o investigar incidentes de seguridad (Velociraptor, s. f.-a; Velociraptor, s. f.-d).

El funcionamiento de la herramienta se basa en artifacts, que empaquetan consultas VQL y configuración en ficheros YAML. VQL permite interactuar con fuentes del endpoint como procesos, archivos, registro, eventos del sistema operativo, Sysmon o PowerShell. Los artifacts pueden emplearse para consultas puntuales, análisis histórico o monitorización continua cuando se diseñan como event artifacts (Velociraptor, s. f.-a; Velociraptor, s. f.-d).

Aunque Velociraptor se ha utilizado tradicionalmente en tareas de análisis forense y respuesta ante incidentes, su arquitectura también permite implementar monitorización continua mediante colas de eventos y server monitoring. Esa capacidad abre la posibilidad de aproximarlos a una herramienta de detección basada en host, siempre que exista una lógica de detección explícita encima de la telemetría (Velociraptor, s. f.-b; Velociraptor, s. f.-c).

En función de su comportamiento y de la fuente de información utilizada, los artifacts pueden clasificarse en tres tipos principales:

Tipo	Qué recoge	Intervalo
Snapshot artifact	Estado actual del sistema	Instante de ejecución
Client Artifacts sobre logs	Eventos históricos	Lo que contenga el log
Client_Event artifacts	Eventos en tiempo real	Desde su activación

Tabla 1: Tipos de artifacts según su modo de ejecución

En esta memoria, los artifacts de tipo CLIENT_EVENT son los más relevantes para aproximar monitorización HIDS, porque ejecutan la lógica de observación en el endpoint y permiten clasificar eventos host-based de forma continua. Esta clasificación no convierte automáticamente cualquier evento visible en alerta; exige reglas, contexto y evidencia trazable.

A pesar de estas capacidades, el uso de Velociraptor como mecanismo HIDS orientado a la detección sistemática de técnicas MITRE ATT&CK no debe asumirse por defecto. Este trabajo analiza en qué medida puede emplearse con ese propósito mediante artifacts nativos y personalizados en un entorno de laboratorio controlado (MITRE, s. f.-h; Velociraptor, s. f.-a).

1.3.4. Criterios de evaluación de herramientas de detección

Para la evaluación de las herramientas de detección se van a considerar los siguientes criterios:

1. Cobertura de detección: capacidad de la herramienta para identificar las técnicas MITRE ATT&CK ejecutadas durante las pruebas. Se medirá como tasa de detección por técnica, indicando si la actividad generada produce o no evidencia observable mediante los artifacts utilizados.
2. Visibilidad del sistema / falsos negativos: análisis de los casos en los que una técnica se ejecuta correctamente pero no genera información detectable a través de los artifacts evaluados. Este criterio permite identificar limitaciones de visibilidad o dependencias de fuentes de datos concretas.
3. Impacto en rendimiento: evaluación del coste que introduce la monitorización mediante Velociraptor sobre el endpoint. Esta parte se expandirá en el capítulo de benchmark, pero a rasgos generales se analizarán métricas como CPU (%), uso de memoria RAM y actividad de disco (I/O).
4. Complejidad operativa: análisis del esfuerzo necesario para desplegar y utilizar las capacidades de detección. Se evaluará principalmente:
 - Si es necesario o no modificar artifacts existentes
 - Desarrollo de artifacts personalizados
 - Ajustes necesarios en la configuración del agente o del servidor.

Estos criterios permiten justificar las métricas utilizadas para evaluar el comportamiento de Velociraptor como posible sistema de detección basado en host.

Capítulo II: Planificación

En este capítulo se definirán los recursos, plazos y actividades que conforman el proyecto con el objetivo de proporcionar una guía que facilite el seguimiento de un cronograma realizado mediante un diagrama de Gantt que abarque todo el ciclo de vida de este. Este cronograma ha sido creado con la aplicación de GanttProject³, esto por lo sencilla que es de usar y por toda la documentación disponible en internet sobre ella. Una vez definida una planificación general, se definirá una planificación por cada capítulo de este trabajo y un método, en concreto Toggl Track⁴ de seguimiento de este.

2.1. Estimación de tiempos y plazos

Para el cálculo y seguimiento de horas reales, como se ha comentado en la introducción, se ha usado la herramienta de Toggl Track, que al realizar una medición precisa del tiempo invertido en el proyecto ha ayudado a mejorar la precisión de la planificación.

Dado que se trata de un Trabajo Fin de Máster, la dedicación total estimada se ha fijado en 150 horas, distribuidas a lo largo del desarrollo del proyecto con una carga aproximada de entre 12 y 15 horas semanales desde su inicio, el 5 de marzo de 2026. Dentro de esta estimación global se incluyen tanto las tareas de desarrollo técnico como las labores de redacción, revisión y cierre de la memoria, sin contemplar en este cómputo la preparación ni la defensa oral del trabajo.

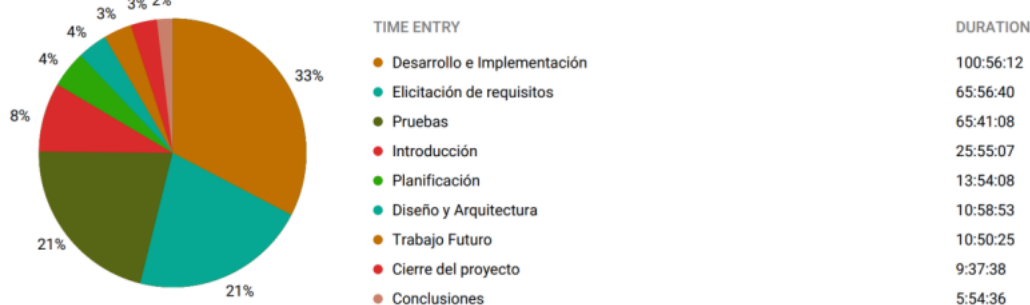


Figura 1: Monitorización con Toggl Track

- Fecha de finalización estimada: 26/06/2026
- Fecha de finalización real: 26/06/2026

³ GanttProject: Es una aplicación de código abierto para la gestión de proyectos que como su nombre indica está basada en los diagramas de Gantt, que permite planificar tareas, asignar recursos y exportar cronogramas en múltiples formatos.

⁴ Toggl Track: Es una aplicación de seguimiento de tiempo orientada a la productividad personal y de equipos, que permite registrar y categorizar el tiempo dedicado a distintas tareas o proyectos, con generación de informes detallados.

2.2. Presupuesto

El presupuesto se ha realizado segmentándolo en costes de personal y de material:

Costes de Personal

Según el Modelo Constructivo de Costos (COCOMO) y declarando que para el desarrollo de este proyecto solo he participado yo, suponiendo una medida de tiempo según la normativa de entre 150 y 200 horas, a un sueldo de unos 10 €/hora, podríamos obtener un coste de personal de entre 1500€ y 2000€ + IVA. En la práctica el coste de mi trabajo es de 0€/hora, por lo que $0€ \times 150 \text{ horas} = 0€$, esto significa que no se han incurrido en costes de personal.

Costes de material

Los elementos mínimos y necesarios para el desarrollo del proyecto han sido estos.

- Portátil MSI + Windows 10: 700€.

El resto de los programas y herramientas han sido de uso libre, por lo que únicamente contaremos con los gastos de material del equipo personal y la licencia del SO, o sea, 700€.

2.3. Tabla de planificación general

Capítulos	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
Capítulo I: Introducción	1 semana	12 h	12 h	05/03/2026	12/03/2026
Capítulo II: Planificación	1 semana	16 h	12 h	13/03/2026	20/03/2026
Capítulo III: Fundamentos teóricos	1 semana	14 h	13 h	21/03/2026	28/03/2026
Capítulo IV: Entorno de trabajo, metodología y selección de técnicas	2 semanas	24 h	24 h	29/03/2026	11/04/2026
Capítulo V: Desarrollo experimental y ejecución de pruebas	3 semanas	28 h	42 h	12/04/2026	03/05/2026
Capítulo VI: Resultados de detección	3 semanas	16 h	34 h	04/05/2026	24/05/2026
Capítulo VII: Evaluación de Velociraptor y	3 semanas	20 h	36 h	25/05/2026	14/06/2026

Capítulos	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
benchmark de rendimiento					
Capítulo VIII: Validación comparativa con Wazuh como HIDS	4 semanas	14 h	46 h	15/06/2026	12/07/2026
Capítulo IX: Trabajo futuro	2 semanas	8 h	12 h	13/07/2026	26/07/2026
Capítulo X: Conclusiones, revisión final y cierre de memoria	7 semanas	8 h	45 h	27/07/2026	14/09/2026
Total	aprox. 6 meses	160 h	246 h	05/03/2026	14/09/2026

Tabla 2: Planificación general

2.4. Tabla de planificación por capítulos

Este apartado presenta la planificación estructurada del trabajo desglosada por capítulos, con el fin de detallar con mayor profundidad el contenido y los objetivos de cada sección de la memoria.

2.4.1. Planificación del capítulo I

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
1.1 Motivación y descripción del problema	3 días	4 h	4 h	05/03/2026	07/03/2026
1.2 Objetivos del proyecto	2 días	3 h	3 h	08/03/2026	09/03/2026
1.3 Estado del arte	3 días	5 h	5 h	10/03/2026	12/03/2026
Total Capítulo I	1 semana	12 h	12 h	05/03/2026	12/03/2026

Tabla 3: Planificación del capítulo I

2.4.2. Planificación del capítulo II

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
2.1 Estimación de tiempos y plazos	2 días	4 h	3 h	13/03/2026	14/03/2026
2.2 Presupuesto	1 día	2 h	1 h	15/03/2026	15/03/2026
2.3 Tabla de planificación general	2 días	4 h	3 h	16/03/2026	17/03/2026
2.4 Tabla de planificación por capítulos	1 día	3 h	2 h	18/03/2026	18/03/2026
2.5 Diagrama de Gantt	2 días	3 h	3 h	19/03/2026	20/03/2026
Total Capítulo II	1 semana	16 h	12 h	13/03/2026	20/03/2026

Tabla 4: Planificación del capítulo II

2.4.3. Planificación del capítulo III

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
3.1 Enfoques de detección y análisis en endpoint	2 días	3 h	3 h	21/03/2026	22/03/2026
3.2 Arquitectura interna Velociraptor	2 días	3 h	3 h	23/03/2026	24/03/2026
3.3 Artifacts y modelo de ejecución	2 días	3 h	3 h	25/03/2026	26/03/2026
3.4 Velociraptor Query Language (VQL)	1 día	3 h	2 h	27/03/2026	27/03/2026
3.5 Telemetría y limitaciones de visibilidad en HIDS/DFIR	1 día	2 h	2 h	28/03/2026	28/03/2026
Total Capítulo III	1 semana	14 h	13 h	21/03/2026	28/03/2026

Tabla 5: Planificación del capítulo III

2.4.4. Planificación del capítulo IV

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
4.1 Diseño del entorno	3 días	6 h	6 h	29/03/2026	31/03/2026
4.2 Instalación, instrumentación y configuración base	3 días	6 h	6 h	01/04/2026	03/04/2026
4.3 Metodología experimental	3 días	5 h	5 h	04/04/2026	06/04/2026
4.4 Selección de técnicas MITRE ATT&CK	2 días	4 h	4 h	07/04/2026	08/04/2026
4.5 Estrategia de artifacts, benchmark y alertabilidad	2 días	3 h	3 h	09/04/2026	11/04/2026
Total Capítulo IV	2 semanas	24 h	24 h	29/03/2026	11/04/2026

Tabla 6: Planificación del capítulo IV

2.4.5. Planificación del capítulo V

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
5.1 Escenarios de ataque	4 días	6 h	8 h	12/04/2026	15/04/2026
5.2 Desarrollo experimental por técnica	7 días	10 h	18 h	16/04/2026	22/04/2026
5.3 Recopilación de telemetría	5 días	6 h	8 h	23/04/2026	27/04/2026
5.4 Configuración y uso de artifacts	6 días	6 h	8 h	28/04/2026	03/05/2026
Total Capítulo V	3 semanas	28 h	42 h	12/04/2026	03/05/2026

Tabla 7: Planificación del capítulo V

2.4.6. Planificación del capítulo VI

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
6.1 Resultados por técnica MITRE	6 días	5 h	12 h	04/05/2026	09/05/2026
6.2 Cobertura de detección	5 días	4 h	8 h	10/05/2026	14/05/2026
6.3 Casos sin detección o detección parcial	4 días	3 h	6 h	15/05/2026	18/05/2026
6.4 Evidencias observadas	6 días	4 h	8 h	19/05/2026	24/05/2026
Total Capítulo VI	3 semanas	16 h	34 h	04/05/2026	24/05/2026

Tabla 8: Planificación del capítulo VI

2.4.7. Planificación del capítulo VII

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
7.1 Metodología de benchmark	4 días	4 h	6 h	25/05/2026	28/05/2026
7.2 Impacto en CPU	4 días	4 h	7 h	29/05/2026	01/06/2026
7.3 Impacto en memoria	4 días	4 h	6 h	02/06/2026	05/06/2026
7.4 Impacto en disco	4 días	4 h	6 h	06/06/2026	09/06/2026
7.5 Relación detección / coste	3 días	2 h	6 h	10/06/2026	12/06/2026
7.6 Interpretación global del benchmark	2 días	2 h	5 h	13/06/2026	14/06/2026
Total Capítulo VII	3 semanas	20 h	36 h	25/05/2026	14/06/2026

Tabla 9: Planificación del capítulo VII

2.4.8. Planificación del capítulo VIII

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
8.1 Objetivo de la comparación	2 días	2 h	3 h	15/06/2026	16/06/2026
8.2 Arquitectura del entorno Wazuh	4 días	2 h	7 h	17/06/2026	20/06/2026
8.3 Instalación del servidor Wazuh	4 días	2 h	7 h	21/06/2026	24/06/2026
8.4 Instalación y validación del agente Windows	3 días	2 h	5 h	25/06/2026	27/06/2026
8.5 Fuentes de telemetría configuradas	3 días	2 h	5 h	28/06/2026	30/06/2026
8.6 Reglas personalizadas TFM en Wazuh	4 días	2 h	8 h	01/07/2026	04/07/2026
8.7 Metodología de campañas separadas	2 días	1 h	3 h	05/07/2026	06/07/2026
8.8 Campañas Wazuh base y Wazuh custom	4 días	1 h	5 h	07/07/2026	10/07/2026
8.9 Comparación Wazuh – Velociraptor y limitaciones	2 días	2 h	3 h	11/07/2026	12/07/2026
Total Capítulo VIII	4 semanas	14 h	46 h	15/06/2026	12/07/2026

Tabla 10: Planificación del capítulo VIII

2.4.9. Planificación del capítulo IX

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
9.1 Ampliación de técnicas MITRE	4 días	2 h	3 h	13/07/2026	16/07/2026
9.2 Desarrollo de nuevos artifacts	3 días	2 h	3 h	17/07/2026	19/07/2026
9.3 Automatización de detecciones	3 días	2 h	3 h	20/07/2026	22/07/2026
9.4 Integración SIEM/SOC y despliegue en entornos reales	4 días	2 h	3 h	23/07/2026	26/07/2026
Total Capítulo IX	2 semanas	8 h	12 h	13/07/2026	26/07/2026

Tabla 11: Planificación del capítulo IX

2.4.10. Planificación del capítulo X

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
10.1 Conclusiones principales	1 semana	3 h	10 h	27/07/2026	02/08/2026
10.2 Limitaciones identificadas	1 semana	2 h	8 h	03/08/2026	09/08/2026
10.3 Valor del enfoque propuesto	1 semana	3 h	8 h	10/08/2026	16/08/2026
10.4 Revisión de coherencia entre capítulos	2 semanas	0 h	9 h	17/08/2026	30/08/2026
10.5 Maquetación, referencias, anexos y cierre de memoria	2 semanas	0 h	10 h	31/08/2026	14/09/2026
Total Capítulo X	7 semanas	8 h	45 h	27/07/2026	14/09/2026

2.5. Diagrama de Gantt

A continuación, se presenta la planificación de cada capítulo mediante diagramas de Gantt. Como se indicó en la introducción de este capítulo, para la elaboración de todos los diagramas se ha utilizado la aplicación GanttProject, debido a su sencillez de uso y a la experiencia previa adquirida con esta herramienta en el desarrollo de mi TFG y en la asignatura Planificación y Gestión de Proyectos Informáticos (PGPI), cursada en el Grado en Ingeniería Informática - Ingeniería de Computadores.

El diagrama se ha generado a partir de las horas estimadas, ya que su finalidad es servir como guía de seguimiento durante el desarrollo del proyecto. Además, se ha optado por segmentar los diagramas en varias partes con el fin de mejorar la calidad y legibilidad de las imágenes.

La siguiente imagen representa la planificación general del proyecto, que posteriormente se irá detallando por capítulos.

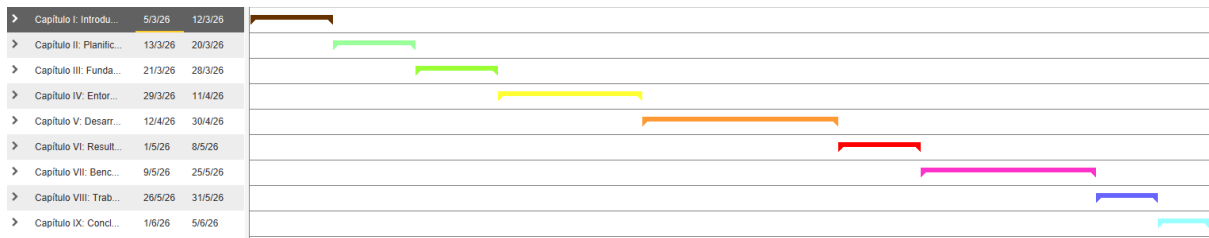


Figura 2: Diagrama de Gantt general

2.5.1. Diagrama de Gantt-Capítulos uno y dos

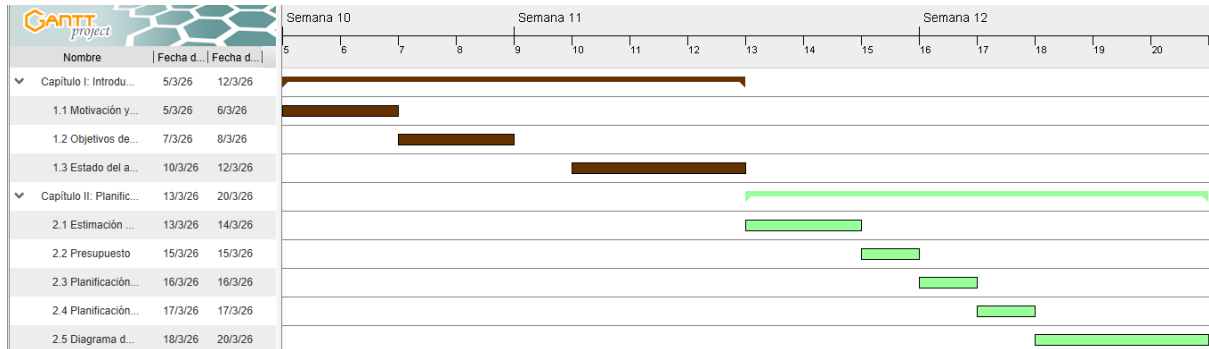


Figura 3: Planificación Gantt de los capítulos I y II

2.5.2. Diagrama de Gantt-Capítulo tres

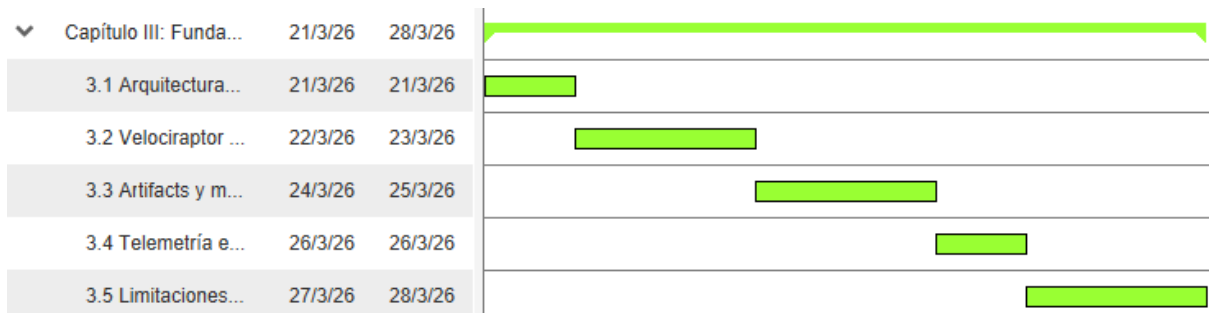


Figura 4: Planificación Gantt del capítulo III

2.5.3. Diagrama de Gantt-Capítulo cuatro

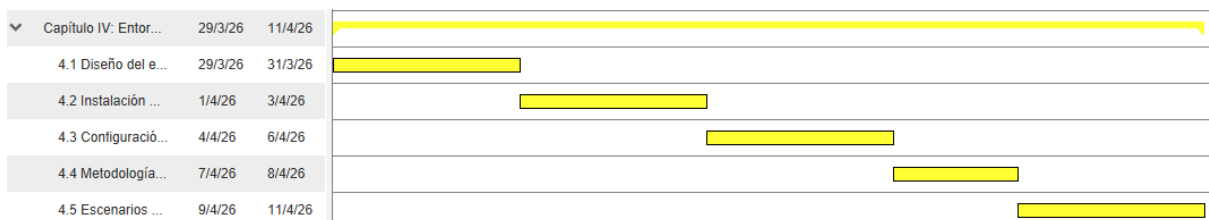


Figura 5: Planificación Gantt del capítulo IV

2.5.4. Diagrama de Gantt-Capítulo cinco

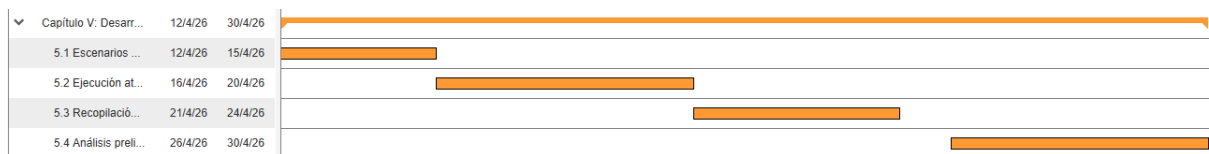


Figura 6: Planificación Gantt del capítulo V

2.5.5. Diagrama de Gantt-Capítulo seis

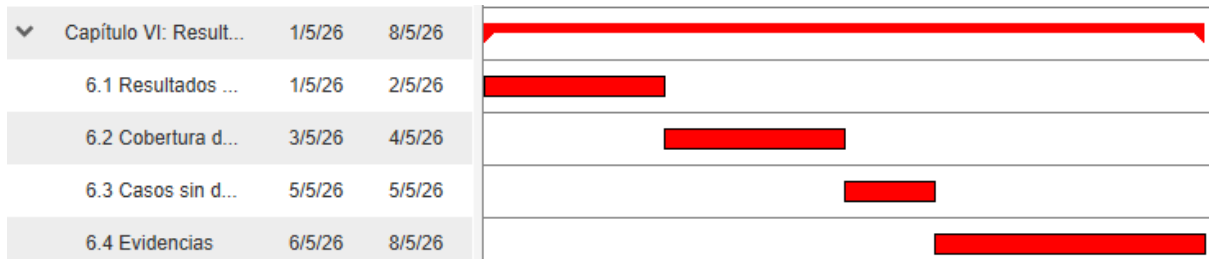


Figura 7: Planificación Gantt del capítulo VI

2.5.6. Diagrama de Gantt-Capítulo siete

El capítulo VII se cerró finalmente como benchmark de rendimiento de Velociraptor. La comparación prevista con una solución EDR comercial se descartó por falta de licencia y se sustituyó por un capítulo específico de validación comparativa con Wazuh como plataforma HIDS/SIEM de contraste.



Figura 8: Planificación Gantt del capítulo VII

2.5.7. Diagrama de Gantt-Capítulos ocho y nueve

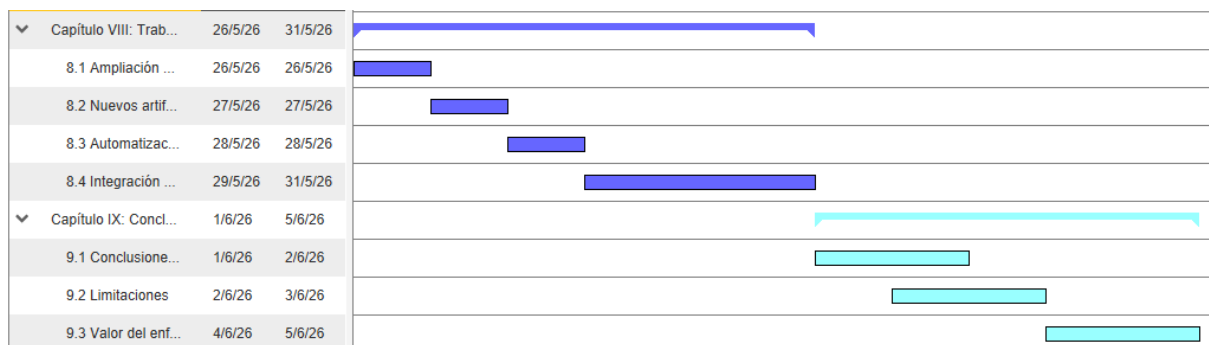


Figura 9: Planificación Gantt de los capítulos VIII y IX

Capítulo III: Fundamentos teóricos

En este capítulo se establecen los fundamentos técnicos necesarios para evaluar la viabilidad de Velociraptor como plataforma HIDS/EDR en endpoints Windows. En primer lugar, se revisan los enfoques de detección y análisis en endpoint -HIDS tradicionales, DFIR y EDR- para situar correctamente el papel de Velociraptor. A continuación, se describe su arquitectura cliente-servidor, el modelo de ejecución basado en artifacts, el lenguaje Velociraptor Query Language (VQL), las fuentes de telemetría host-based disponibles y las limitaciones inherentes a la visibilidad en sistemas basados en host.

3.1. Enfoques de detección y análisis en endpoint

La protección y análisis de endpoints puede abordarse desde varios enfoques complementarios. Aunque todos trabajan sobre sistemas finales, no persiguen exactamente el mismo objetivo ni operan con el mismo grado de automatización, profundidad o capacidad de respuesta. Para este TFM se diferencian cuatro aproximaciones: HIDS tradicionales, DFIR, EDR y Velociraptor.

3.1.1. Detección en HIDS tradicionales

Un HIDS ⁵o sistema de detección de intrusiones basado en host monitoriza las características de un único equipo y los eventos que se producen dentro de ese host para identificar actividad sospechosa o potencialmente maliciosa. Su ámbito natural es local: procesos, eventos del sistema, integridad de ficheros, registros, autenticaciones, servicios y cambios de configuración.

En términos generales, los HIDS tradicionales suelen basarse en reglas, firmas, patrones de eventos, comprobación de integridad y análisis de logs. Su principal ventaja es que observan el resultado de la actividad directamente en el host, incluso cuando esta no es visible desde una sonda de red. Sin embargo, su eficacia, en parte igual como veremos luego la del DFIR, depende de la calidad de las reglas, de la configuración de auditoría del sistema operativo y de la capacidad para reducir falsos positivos.

Fortalezas	visibilidad local, reglas explícitas, bajo coste relativo, utilidad para auditoría y cumplimiento
Limitaciones	dependencia de logs habilitados, reglas rígidas, dificultad para contextualizar comportamientos legítimos y riesgo de ruido operativo

⁵ NIST Computer Security Resource Center. “Host-Based Intrusion Detection and Prevention System”: https://csrc.nist.gov/glossary/term/host_based_intrusion_detection_and_prevention_system

Fortalezas	visibilidad local, reglas explícitas, bajo coste relativo, utilidad para auditoría y cumplimiento
Relación con este TFM	proporcionan el marco conceptual de detección host-based frente al que se evalúa Velociraptor.

Tabla 13: Funciones, limitaciones y riesgos del enfoque HIDS

3.1.2. Detección en DFIR

El enfoque DFIR ⁶se centra en la adquisición, preservación, análisis e interpretación de evidencias digitales durante o después de un incidente. A diferencia de un HIDS, cuyo objetivo principal es detectar actividad durante su ejecución, el análisis DFIR suele reconstruir hechos a partir de artefactos persistentes: logs, ficheros, registro, procesos, memoria, timelines y otros rastros del sistema

Velociraptor encaja de forma natural en este ámbito porque permite ejecutar consultas distribuidas sobre endpoints, recoger evidencias concretas y realizar análisis remoto de forma selectiva. No obstante, la capacidad DFIR no implica automáticamente capacidad HIDS. Para aproximarse a un funcionamiento HIDS es necesario que existan artifacts de monitorización continua, filtros de detección, normalización de resultados y mecanismos de alertado.

Fortalezas	profundidad analítica, adquisición selectiva, reconstrucción de incidentes y trazabilidad forense.
Limitaciones	dependencia de evidencias que permanezcan disponibles; pérdida posible por rotación de logs, limpieza o ausencia de auditoría previa.
Relación con este TFM	Velociraptor se evalúa como herramienta DFIR con capacidad potencial de operar como HIDS mediante artifacts de tipo CLIENT_EVENT.

Tabla 14: Funciones, limitaciones y riesgos del enfoque DFIR

3.1.3. Detección en EDR

Las soluciones EDR⁷, Endpoint Detection and Response, monitorizan endpoints de forma continua para detectar indicios de amenaza y facilitar acciones de investigación, contención o respuesta. Suelen incorporar telemetría persistente, reglas mantenidas por el proveedor, analítica de comportamiento, scoring, consola SOC y acciones de respuesta como aislamiento o finalización.

⁶ NIST SP 800-86. “Guide to Integrating Forensic Techniques into Incident Response”: <https://csrc.nist.gov/pubs/sp/800/86/final>

⁷ Microsoft Security. “What is EDR? Endpoint Detection and Response”: <https://www.microsoft.com/en-us/security/business/security-101/what-is-edr-endpoint-detection-response>

A diferencia de un EDR comercial, Velociraptor no incorpora por defecto una capa cerrada de analítica avanzada, priorización propietaria o respuesta automatizada equivalente. Su valor reside en la transparencia y flexibilidad: el analista puede definir qué recoger, cómo filtrarlo y cómo clasificarlo.

Fortalezas	monitorización continua del endpoint, correlación de eventos, analítica de comportamiento, priorización de alertas y capacidades de respuesta.
Limitaciones	dependencia del proveedor, menor transparencia interna de las reglas, coste económico superior, posible efecto “caja negra”. Necesidad de ajuste de FPs.
Relación con este TFM	proporcionan el marco conceptual de referencia para comparar qué capacidades se esperan de una plataforma moderna de detección en endpoint.

Tabla 15: Funciones, limitaciones y riesgos del enfoque EDR

3.1.4. Posición de Velociraptor frente a estos enfoques

Velociraptor ocupa una posición intermedia entre una plataforma DFIR distribuida y una herramienta HIDS flexible. Puede recoger evidencias bajo demanda como una herramienta forense, ejecutar hunts sobre múltiples endpoints y mantener monitorización continua mediante artifacts de tipo CLIENT_EVENT. Sin embargo, observar eventos no equivale por sí solo a detectar amenazas ni a generar una alerta operativa (Velociraptor, s. f.-a; Velociraptor, s. f.-b).

Enfoque	Función principal	Fortaleza	Limitación principal	Papel en el TFM
HIDS	Detectar eventos/reglas.	Visibilidad local.	Ruido y reglas rígidas.	Marco HIDS.
DFIR	Investigar incidentes.	Profundidad forense.	Evidencia residual.	Base forense.
EDR	Monitorizar, detectar y responder.	Consola SOC y respuesta.	Caja negra/coste.	Referencia conceptual.
Velociraptor	Adquirir, buscar y monitorizar.	Transparencia + YAML/VQL.	Requiere diseño.	Herramienta evaluada.

Tabla 16: Posición de Velociraptor frente a HIDS, DFIR y EDR

Por tanto, la hipótesis del trabajo no es que Velociraptor sea equivalente a un EDR comercial, sino que, mediante artifacts adecuados, puede transformar telemetría host-based en detecciones reproducibles, contextualizadas y accionables frente a técnicas MITRE ATT&CK seleccionadas.

3.2. Arquitectura interna Velociraptor

Velociraptor utiliza una arquitectura cliente-servidor. El analista interactúa con la interfaz gráfica o API del servidor, selecciona artifacts, define parámetros y lanza colecciones, hunts o monitorización continua. El servidor coordina la ejecución, pero la lógica de consulta se ejecuta principalmente en el endpoint, donde el cliente accede a las fuentes locales de telemetría (Velociraptor, s. f.-a; Velociraptor, s. f.-d).

Se ha resumido el flujo funcional desde la intervención del analista hasta la devolución de resultados desde los endpoints. Para ello se ha realizado un esquema usando la aplicación web draw.io.

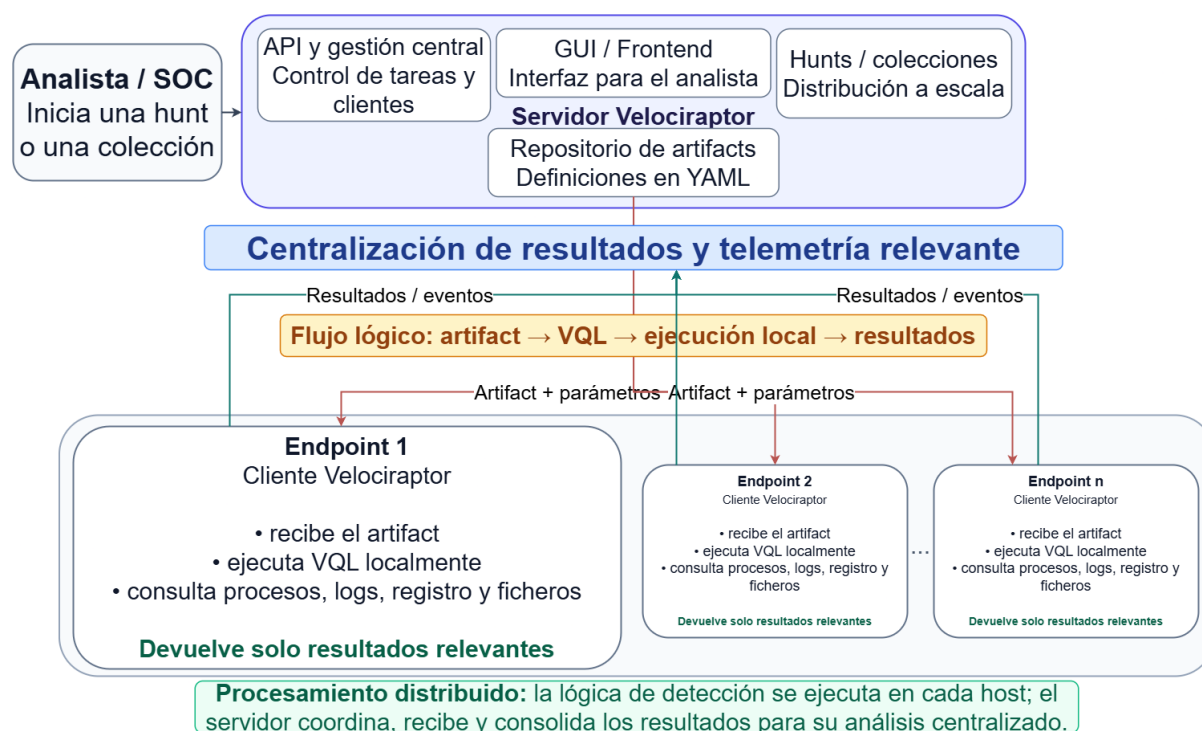


Figura 10: Arquitectura interna básica VR

3.2.1. Diferencias entre capas de Velociraptor

Para comprender el funcionamiento de Velociraptor como herramienta HIDS/DFIR, es necesario distinguir sus principales capas funcionales. Velociraptor no debe entenderse únicamente como un agente instalado en un equipo, sino como una arquitectura compuesta por cliente, servidor, artifacts y distintos modos de ejecución.

- **Client:** agente instalado en el endpoint. Proporciona visibilidad directa sobre el host y accede a procesos, ficheros, registro, eventos de Windows, Sysmon, PowerShell y conexiones de red.

- Server: consola y backend central. Gestiona clientes, agenda colecciones, coordina hunts, mantiene la configuración de monitoring, recibe resultados y almacena evidencias.
- Artifacts: unidades lógicas de análisis definidas en YAML y VQL. Determinan qué se recoge, cómo se filtra y qué comportamiento se considera relevante.
- Modos de ejecución: collections, hunts y monitoring. Definen si la ejecución es puntual, masiva o continua.

Capa/componente	Qué es	Uso principal
Client	Agente instalado en el endpoint Windows	Recoge telemetría local: eventos, logs, procesos, ficheros, Sysmon, PowerShell 4104, conexiones de red y cambios del sistema
Server	Consola y backend central de Velociraptor	Gestiona clientes, recibe resultados, almacena evidencias, lanza colecciones, coordina hunts y procesa eventos
Artifacts	Lógica de análisis definida en YAML + VQL	Unidad principal de recogida, filtrado, detección y monitorización
Collections	Ejecución puntual de artifacts sobre uno o varios endpoints concretos	Investigación DFIR, recogida de evidencias específicas y análisis de incidentes
Hunts	Ejecución masiva de artifacts sobre múltiples endpoints	Threat hunting, búsqueda de indicadores y análisis a escala en varios equipos
Monitoring / CLIENT_EVENT	Ejecución continua de artifacts en el endpoint	Monitorización HIDS, detección continua y generación de eventos en tiempo real o casi real
SERVER_EVENT	Artifact ejecutado en el servidor a partir de eventos recibidos	Normalización, correlación ligera, alertado y envío a canales externos como Discord, webhook o SIEM

Tabla 17: Capas funcionales de la arquitectura de Velociraptor

Esta separación es relevante para el TFM porque permite diferenciar cuatro conceptos: visibilidad, detección, alerta en tiempo real y salida externa. Que Velociraptor observe un evento no implica necesariamente que lo clasifique como una técnica MITRE, ni que genere una alerta operativa,

ni que esta sea enviada a un canal externo. Para este proyecto necesitaremos que VR sepa diferenciar entre monitorización y alerta y entre evento maligno y benigno lo máximo posible.

3.2.2. Flujo cliente-servidor y formatos de datos

Velociraptor no envía los resultados como JSON plano entre cliente y servidor. Internamente, la comunicación cliente-servidor se realiza mediante mensajes Protobuf cifrados, transportados sobre HTTPS POST. Después, el servidor representa esos resultados como tablas, filas, exports JSON/CSV/JSONL o ficheros descargables.

Desde la perspectiva del analista, conviene distinguir entre formato lógico, formato de transporte y formato de explotación analítica. El analista trabaja con artifacts YAML y consultas VQL; el transporte interno se realiza mediante mensajes estructurados; y el servidor representa los resultados como tablas, logs, uploads o exportaciones en JSON, JSONL, CSV o ZIP.

Fase	Formato principal	Función
Definición del artifact	YAML	Describe el artifact, parámetros, sources y metadatos.
Lógica de consulta	VQL	Define qué fuente se consulta, qué campos se devuelven y qué filtros se aplican.
Transporte interno	Protobuf / MessageList	Agrupar mensajes cliente-servidor con consultas, resultados o control de ejecución.
Canal de comunicación	HTTPS POST / TLS	Transporta los MessageList entre cliente y servidor.
Representación en servidor	Result sets / tablas	Permite visualizar filas, columnas, logs y eventos desde la GUI.
Exportación	JSON, JSONL, CSV, ZIP	Facilita análisis externo, anexos de evidencias o tratamiento posterior.
Salida externa	JSON sobre HTTP POST	Permite enviar alertas a Discord, webhook o SIEM desde SERVER_EVENT.

Tabla 18: Formatos de datos en el flujo cliente-servidor de Velociraptor

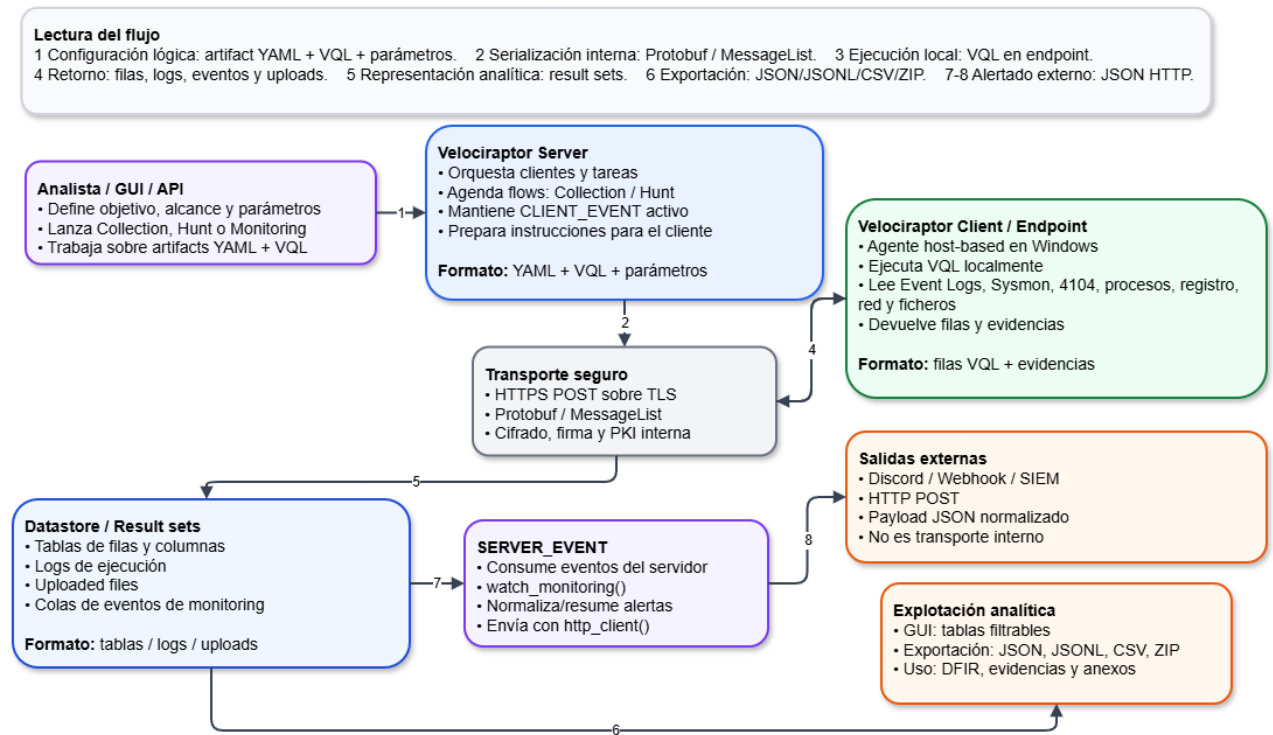


Figura 11: Flujo de datos de Velociraptor

La carga principal se ejecuta en el cliente: el endpoint ejecuta las consultas VQL y el servidor recoge los resultados. Esta separación es coherente con el modelo de artifacts de Velociraptor, en el que la consulta se distribuye al cliente para devolver resultados concretos al servidor (Velociraptor, s. f.-a; Velociraptor, s. f.-d).

3.3. Artifacts y modelo de ejecución

Como se ha comentado con anterioridad, Velociraptor sigue un modelo de ejecución distribuida. En lugar de recopilar primero todos los datos brutos del endpoint y analizarlos después en el servidor, el sistema envía la lógica de análisis al propio host. Su filosofía es clara: formular consultas dirigidas sobre los endpoints y devolver solo información de alto valor, evitando adquisiciones masivas que no escalan bien.

3.3.1. Qué es un artifact

Un artifact es la unidad lógica principal de Velociraptor. Permite empaquetar una o varias consultas VQL junto con su configuración en un fichero YAML estructurado. Gracias a este modelo, el analista no necesita ejecutar manualmente consultas VQL aisladas, sino que puede seleccionar artifacts reutilizables, parametrizables y gestionados desde el servidor (Velociraptor, s. f.-a).

La documentación oficial indica además que cada source genera una tabla de resultados y que, si hay varias sources, deben tener nombres distintos para evitar ambigüedad en la salida (Velociraptor, s. f.-a).

Los elementos principales de un artifact son:

Campo	Función
Name	Identificador único del artifact
description	Descripción funcional y buscable desde la interfaz
Type	Contexto de ejecución: CLIENT_EVENT, SERVER, SERVER_EVENT...
parameters	Variables configurables por el analista
sources	Bloques de ejecución VQL que generan resultados
precondition	Condición opcional para decidir si debe ejecutarse
Query	Consulta VQL que realiza la recogida, filtrado o detección

Tabla 19: Campos principales de definición de artifacts

En el contexto del TFM, los artifacts públicos se consideran una línea base de visibilidad y detección parcial. Los artifacts personalizados permiten adaptar la lógica al entorno, normalizar campos, reducir falsos positivos y clasificar señales según técnicas MITRE seleccionadas.

3.3.2. Tipos de artifacts según el contexto de ejecución

Velociraptor diferencia los artifacts según el lugar y el modo en que se ejecutan. Esta clasificación es importante porque no todos tienen valor HIDS directo.

Tipo	Dónde actúa	Uso principal
CLIENT	Endpoint	Recogida puntual de evidencias
CLIENT_EVENT	Endpoint	Monitorización continua del host
SERVER	Servidor	Tareas administrativas o consultas del servidor
SERVER_EVENT	Servidor	Procesamiento de eventos, alertado y automatización
INTERNAL	Servidor	Colas internas del sistema

Tabla 20: Tipos de artifacts según el contexto de ejecución

La documentación oficial⁸ indica que CLIENT_EVENT establece colas de eventos de cliente en el servidor y contiene VQL enviada al cliente para generar datos. Por su parte, SERVER_EVENT establece colas de eventos en el servidor.

3.3.3. Collections, Hunts y Monitoring

Los artifacts pueden ejecutarse mediante distintos modos:

Modo	Alcance	Temporalidad	Uso adecuado
Collection	Uno o varios endpoints seleccionados.	Puntual; finaliza al devolver resultados.	Investigación concreta, adquisición de evidencias, validación de hipótesis.
Hunt	Múltiples endpoints.	Puntual o programado; orientado a escala.	Threat hunting, búsqueda de hashes, procesos, ficheros o indicadores.
CLIENT_EVENT	Endpoints con la configuración de monitoring aplicada.	Continuo; permanece activo.	Detección HIDS en tiempo casi real y generación de eventos.
SERVER_EVENT	Servidor.	Continuo sobre colas de eventos.	Normalización, correlación ligera, alertado y salida externa.

Tabla 21: Diferencias entre collections, hunts, monitoring y SERVER_EVENT

Esta diferencia es importante para el diseño experimental. Los hunts y collections permiten verificar visibilidad y recuperar evidencias, pero no representan por sí mismos una detección continua. Para evaluar Velociraptor como HIDS, el foco debe situarse en CLIENT_EVENT y en la capacidad de los artifacts para generar eventos filtrados, relevantes y accionables.

⁸ Documentación sobre los campos de artifacts: https://docs.velociraptor.app/docs/artifacts/basic_fields/

3.4. Velociraptor Query Language (VQL)

Como se ha podido deducir por las explicaciones previas, el Velociraptor Query Language (VQL⁹) es el lenguaje utilizado por Velociraptor para consultar y procesar información procedente de los endpoints. Su sintaxis se inspira en SQL, pero su funcionamiento se diferencia de los sistemas de bases de datos tradicionales. En lugar de consultar tablas persistentes, VQL opera sobre plugins, que generan dinámicamente información del sistema en forma de filas.

Este lenguaje constituye el núcleo funcional de Velociraptor, ya que permite definir la lógica de adquisición de evidencias, detección de comportamientos sospechosos o recopilación de telemetría del endpoint. Las consultas VQL se integran normalmente dentro de artifacts, desde donde se distribuyen a los clientes para su ejecución local (Velociraptor, s. f.-d).

3.4.1. Estructura de una consulta VQL

Como se acaba de comentar en el punto previo, las consultas VQL siguen una estructura similar a las consultas SQL, basada principalmente en las cláusulas SELECT y FROM. La cláusula FROM especifica el plugin que actuará como fuente de datos, mientras que SELECT define qué campos o expresiones se devolverán como resultado.

Una estructura simplificada sería la siguiente:

```
$ SELECT column1, column2  
$ FROM plugin(parametros)  
$ WHERE condicion
```

Código 1: Ejemplo de consulta VQL

En este modelo, el plugin indicado en la cláusula FROM genera filas de datos que posteriormente pueden filtrarse, transformarse o procesarse mediante condiciones y expresiones dentro de la consulta.

Un ejemplo más realista, para obtener la información de un endpoint sería algo así:

```
$ SELECT Name, Pid  
$ FROM pslist()
```

Código 2: Ejemplo de obtención de información mediante VQL

Esta consulta utiliza el plugin pslist() para obtener los procesos activos del sistema y devuelve el nombre del proceso junto con su identificador.

⁹ Velociraptor Documentation. “VQL Fundamentals”: <https://docs.velociraptor.app/docs/vql/fundamentals/>

Otro ejemplo para consulta de claves de persistencia en el registro:

```
$ SELECT Key, Value
$ FROM registry(key=
"HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows\\CurrentVersion\\Run")
```

Código 3: Ejemplo de consulta de claves del registro mediante VQL

3.4.2. Query Engine, plugins y lazy evaluation

El Query Engine es el motor que procesa las consultas VQL. Su funcionamiento básico consiste en llamar al plugin indicado en la cláusula FROM, recibir filas generadas por dicho plugin, aplicar filtros y emitir únicamente las filas que cumplen los criterios definidos. La documentación oficial describe que los plugins VQL no son tablas estáticas en disco, sino componentes que producen filas dinámicamente.

- Internamente, el funcionamiento básico es:
- El plugin indicado en FROM genera una o varias filas.
- Cada fila se evalúa frente a filtros y condiciones.
- Las columnas del SELECT se calculan cuando son necesarias.
- El resultado final se devuelve como filas estructuradas.

Un aspecto importante del Query Engine es la lazy evaluation. Velociraptor intenta retrasar la evaluación de expresiones hasta que sea realmente necesaria. Esto evita trabajo innecesario, por ejemplo, calcular columnas complejas en filas que después van a ser descartadas por un filtro. Según la documentación, este comportamiento es clave para escribir consultas VQL eficientes.

Un ejemplo del funcionamiento del lazy evaluation:

```
$ SELECT Name, hash(path=Exe)
$ FROM pslist()
$ WHERE Name = "powershell.exe"
```

Código 4: Ejemplo de evaluación diferida en VQL

En el ejemplo anterior, pslist() genera filas de procesos. El filtro WHERE limita el resultado a powershell.exe y solo para esas filas se calcula el hash del ejecutable. Sin lazy evaluation, el motor tendría que calcular el hash de todos los procesos antes de descartar los que no coinciden, lo que incrementaría operaciones de disco y consumo de CPU.

Esta característica es relevante para el TFM porque permite diseñar artifacts más eficientes: primero se filtra por señales simples y posteriormente se calculan campos más costosos, como hashes, búsquedas en disco, parseo de eventos o correlaciones.

3.4.3. Event queries y monitorización continua

Las consultas VQL convencionales devuelven un conjunto de resultados y finalizan. Las event queries¹⁰, en cambio, se apoyan en event plugins que no terminan por sí mismos: permanecen esperando sucesos y emiten filas cuando el evento ocurre. La documentación oficial indica que estas consultas devuelven resultados parciales hasta que son canceladas.

En la arquitectura de client monitoring¹¹, el cliente mantiene un conjunto de event queries en paralelo dentro de su tabla de monitorización. Estas consultas pueden ejecutarse de forma autónoma en el endpoint, incluso aunque el servidor solo sea necesario para sincronizar cambios de configuración. Cuando producen filas, el cliente las envía al servidor, que las escribe en el datastore.

Este modelo es el más relevante para una aproximación HIDS, ya que permite capturar actividad efímera que podría perderse con una collection puntual. No obstante, los eventos no se remiten necesariamente de forma instantánea: Velociraptor puede agrupar filas en memoria y enviarlas por lotes para evitar comunicaciones demasiado frecuentes. Por ello, en el TFM se hablará de tiempo casi real, no de tiempo real estricto.

¹⁰ Velociraptor Documentation. “Event Queries”: <https://docs.velociraptor.app/docs/vql/events/>

¹¹ Velociraptor Documentation. “Client Monitoring”: Disponible en: <https://docs.velociraptor.app/docs/clients/monitoring/>

3.5. Telemetría disponible en el endpoint

La capacidad de detección de Velociraptor depende de la telemetría disponible en el endpoint, es decir, de los datos del sistema que pueden ser observados y consultados mediante VQL. Esta información procede de distintas fuentes del propio host y permite identificar actividad sospechosa, cambios en el sistema o evidencias asociadas a técnicas de ataque.

Las fuentes principales de telemetría y su utilidad son:

Fuente de telemetría	Qué puede observar Velociraptor	Qué puede ayudar a detectar
Procesos	procesos activos, PID, usuario, ruta, línea de comandos, relación padre-hijo	ejecución sospechosa, LOLBins, procesos anómalos, post-explotación
Windows Event Logs	autenticaciones, creación de procesos, errores, eventos de seguridad, PowerShell, Sysmon si existe	accesos anómalos, ejecución de comandos, persistencia, movimiento lateral
Sistema de ficheros	archivos, rutas, fechas, tamaños, hashes	malware en disco, ficheros sospechosos, cambios en ubicaciones sensibles
Registro de Windows	claves, valores y rutas de configuración	persistencia, modificaciones del sistema, claves Run/RunOnce
PowerShell	comandos ejecutados, scripts, Script Block Logging, actividad registrada	ataques fileless, uso malicioso de scripts, automatización ofensiva

Tabla 22: Fuentes principales de telemetría y utilidad en detección

3.6. Limitaciones de visibilidad en HIDS/DFIR

Los sistemas basados en host dependen de las fuentes de información disponibles en el endpoint. Si una acción no genera eventos, si el log correspondiente no está habilitado o si el artifact no consulta la fuente adecuada, Velociraptor no podrá detectarla. Esta limitación no es específica de Velociraptor; afecta a cualquier enfoque host-based.

Por ello, en este TFM se diferencia explícitamente entre cuatro niveles:

Nivel	Significado
Visibilidad	El evento existe y puede observarse
Detección	Una lógica lo clasifica como relevante
Alerta RT	Se genera desde monitorización continua
Salida externa	Se envía a JSONL, Discord, webhook o SIEM
Evidencia	Queda rastro verificable: log, captura, hash, CSV, JSON
Ruido / FP	Actividad legítima clasificada como sospechosa

Tabla 23: Diferencia entre visibilidad, detección, alerta y salida externa

Esta separación evita sobredimensionar los resultados. Observar un evento no equivale a detectar una técnica MITRE. Generar una detección no implica necesariamente que exista una alerta operativa. Y enviar una notificación externa no demuestra por sí solo que la lógica de detección sea correcta.

3.6.1. Visibilidad parcial del comportamiento del atacante

La visibilidad de un HIDS es parcial por naturaleza. Aunque el agente pueda acceder a múltiples fuentes del endpoint, no siempre es posible reconstruir completamente el comportamiento del atacante. Una misma técnica MITRE ATT&CK puede ejecutarse de formas distintas y no todas generan las mismas evidencias.

Esto puede provocar falsos negativos en varios casos:

- La técnica no genera eventos observables.
- El evento existe, pero no está habilitado el log correspondiente.
- El artifact no monitoriza la fuente adecuada.
- La regla de detección no contempla esa variante concreta.
- El comportamiento se confunde con actividad legítima del sistema.

Por este motivo, la detección no se valora únicamente por la existencia de eventos, sino por la capacidad de transformar esa telemetría en señales útiles, contextualizadas y con bajo nivel de falsos positivos.

3.6.2. Limitaciones de visibilidad en DFIR

En un contexto DFIR, las limitaciones de visibilidad difieren de las existentes en la detección en tiempo real. Mientras que un HIDS pretende observar eventos durante su ejecución, el análisis DFIR suele realizarse de forma posterior, a partir de las evidencias que permanecen disponibles en el sistema.

Por tanto, la investigación depende de los artefactos que hayan sobrevivido tras la actividad analizada. Algunos eventos pueden haberse perdido por rotación de logs, limpieza del sistema, sobrescritura, falta de auditoría o ausencia de una configuración previa adecuada. Además, determinadas técnicas pueden dejar evidencias débiles, incompletas o ambiguas, lo que dificulta su atribución a una acción concreta.

En este sentido, Velociraptor puede aportar valor tanto en la adquisición de evidencias como en su análisis posterior, pero su eficacia depende de la calidad y persistencia de la telemetría disponible en el endpoint. Si una acción no fue registrada o sus rastros fueron eliminados, la capacidad de reconstrucción será limitada.

Esta es la diferencia que resulta más relevante porque no se evalúa únicamente la capacidad de Velociraptor para consultar información del sistema (que al ser un DFIR se da por correcta), sino su utilidad para convertir evidencias host-based en detecciones interpretables y alertas accionables. Posteriormente, esta capacidad se contrastará con los resultados experimentales y con el impacto operativo observado en el endpoint.

A partir de esta base teórica, el trabajo no evalúa únicamente si Velociraptor puede consultar información del endpoint, sino si dicha información puede transformarse en detecciones reproducibles, alertas operativas, evidencias trazables y señales con un nivel aceptable de ruido.

Capítulo IV: Entorno de trabajo, metodología y selección de técnicas

En este capítulo se define el entorno experimental utilizado para evaluar Velociraptor como plataforma HIDS/DFIR, la metodología de trabajo aplicada y el criterio de selección de técnicas MITRE ATT&CK. El objetivo no es presentar todavía los resultados, sino fijar una base metodológica reproducible que permita interpretar posteriormente la visibilidad obtenida, la capacidad de detección, la generación de alertas en tiempo casi real y el impacto operativo de la monitorización sobre el endpoint.

La metodología se apoya en tres principios: ejecución controlada de técnicas, trazabilidad de evidencias y separación estricta entre visibilidad, detección, alerta operativa y salida externa. Esta separación evita considerar como detección aquello que únicamente corresponde a telemetría bruta, transporte de eventos o notificación externa.

Nota: Este capítulo establece el diseño experimental. Los resultados concretos por técnica, la tasa de detección, los falsos positivos, las evidencias observadas y el impacto cuantitativo de rendimiento se analizarán en los capítulos posteriores.

Tabla 24: Criterios metodológicos del capítulo IV

4.1. Diseño del entorno

Como se comentó en el anterior capítulo, el laboratorio se diseñó como un entorno controlado sobre Windows 10, orientado a reproducir comportamientos host-based asociados a técnicas MITRE ATT&CK sin emplear malware real ni afectar a datos legítimos. La arquitectura se basa en el modelo cliente-servidor de Velociraptor: el servidor coordina la ejecución de artifacts y el endpoint ejecuta localmente las consultas VQL y devuelve resultados estructurados.

4.1.1. Configuración base de la máquina virtual

La fase experimental se desarrolló sobre una máquina virtual proporcionada por mi tutor y utilizada como entorno de laboratorio. La configuración se mantuvo estable durante las pruebas para reducir variabilidad y facilitar la repetición de escenarios.

Parámetro	Valor
Hipervisor	Oracle VirtualBox
Sistema operativo	Windows 10 64 bits
Memoria RAM	6000 MB
Procesadores virtuales	4 vCPU
Memoria de vídeo	128 MB
Adaptador de red	NAT
Chipset	PIIX3
UEFI	Desactivado
Secure Boot	Desactivado
TPM	No configurado

Tabla 25: Parámetros iniciales de la máquina virtual

La elección de un laboratorio virtualizado permite repetir las pruebas, restaurar snapshots y aislar las simulaciones del sistema físico. No obstante, este diseño también introduce una limitación: los resultados de rendimiento deben interpretarse como mediciones relativas del laboratorio y no como una estimación directa de impacto en una infraestructura corporativa completa.

4.1.2. Componentes técnicos desplegados

Componentes	Versión / referencia	Uso metodológico
Velociraptor	0.75.6	Servidor, GUI y agente cliente instalados en el entorno Windows de laboratorio
Sysmon	15.15 / schema 4.90	Generación de telemetría avanzada de procesos, red, ficheros, registro y borrado
PowerShell Script Block Logging	Event ID 4104	Visibilidad sobre bloques de código PowerShell ejecutados
Windows Event Logs	System / Security / PowerShell / Sysmon	Fuente base para Event ID 7045, PowerShell 4104 y eventos Sysmon
Discord webhook	HTTP 204 esperado	Validación de salida externa mediante SERVER_EVENT/router
Matriz Excel de control	Análisis_Técnicas_TFM_Velociraptor_DEF.xlsx	Trazabilidad de técnicas, artifacts, evidencias, alertabilidad y benchmark

Tabla 26: Componentes técnicos principales

4.1.3. Identificación del cliente y alcance del laboratorio

Elemento	Valor
ClientId observado	C.364907172793f8c7
Host observado	LT29.ait.us.es
Usuario observado	LT29\seguridad
Endpoint evaluado	Windows 10 Pro 22H2
Ámbito	Pruebas locales host-based sobre una única VM controlada

Tabla 27: Identificación del endpoint usado en el laboratorio

El alcance experimental queda limitado a un único endpoint Windows. Esta decisión permite controlar el entorno y documentar con precisión cada prueba, pero impide generalizar directamente los resultados a redes heterogéneas con múltiples sistemas operativos, usuarios y cargas de trabajo.

4.2. Instalación, instrumentación y configuración base

La preparación del entorno no se limitó al despliegue del agente de Velociraptor. Para que la evaluación tuviera valor, fue necesario instrumentar el endpoint con fuentes de telemetría suficientes y establecer mecanismos de validación de conectividad, evidencias y alertas.

4.2.1. Servidor Velociraptor

El servidor Velociraptor se desplegó en modo cliente-servidor, habilitando la GUI de administración y el frontend de comunicación con agentes. El archivo `server.config.yaml` concentró los parámetros de red, GUI, datastore y certificados. Durante la configuración inicial se creó un usuario administrador para acceder a la consola y gestionar clientes, artifacts, collections, hunts y monitoring.

Elemento	Valor	Función
Frontend	0.0.0.0:8443	Punto de escucha para agentes
GUI	127.0.0.1:8889	Interfaz local de administración
Configuración	<code>server.config.yaml</code>	Archivo principal del servidor
Datastore	Local del laboratorio	Almacenamiento de resultados y eventos
Autenticación GUI	Usuario administrador local	Acceso controlado a la consola de gestión

Tabla 28: Parámetros funcionales del servidor Velociraptor

4.2.2. Agente Velociraptor

El agente se generó desde la propia GUI mediante `Server.Utils.CreateMSI`. El instalador MSI incorporó la configuración del servidor y los parámetros de conexión necesarios para registrar el endpoint en la consola. Tras la instalación, se validó la conectividad mediante la aparición del cliente en la sección de endpoints gestionados y la ejecución de artifacts de inventario.

Tool VelociraptorWindowsMSI

Figura 12: Instalación del agente Velociraptor

State	FlowId	Artifacts	Created	Last Active	Creator	Mb	Rows
✖	F.D6MA323F7R970	Windows.Forensics.Pst	2026-03-07T22:07:04.182Z	2026-03-07T22:12:54.480Z	admin	0 b	0
✔	F.D6MA2RIQ9PJQU	Windows.Detection.Yara.Device Windows.Detection.Yara.Process	2026-03-07T22:06:38.465Z	2026-03-07T22:07:49.998Z	admin	0 b	0
✔	F.D6MA038S7U2K	Windows.System.Plist	2026-03-07T22:00:45.539Z	2026-03-07T22:00:55.053Z	admin	0 b	160
✔	F.D6M9VKG6BFCB	Windows.System.Services	2026-03-07T21:59:46.805Z	2026-03-07T21:59:48.882Z	admin	0 b	303
✔	F.D6M7P00BIST1G	Windows.System.PowerShell	2026-03-07T19:30:43.378Z	2026-03-07T19:30:44.955Z	admin	0 b	1
✔	F.D6M7P1NVEJK5Q	Windows.System.PowerShell	2026-03-07T19:30:18.104Z	2026-03-07T19:30:24.867Z	admin	0 b	1
✔	F.D6M7EL9HMM9CI	Generic.Client.Info	2026-03-07T19:07:01.119Z	2026-03-07T19:07:03.303Z	InterrogationService	0 b	28

Figura 13: Datos recibidos desde los agentes Velociraptor

Como comprobación básica, se ejecutaron artifacts de inventario como Windows.System.Pslist y Windows.System.Services. La devolución de procesos y servicios confirmó que el cliente podía ejecutar consultas VQL y enviar resultados al servidor. Los artifacts que no devolvieron resultados en esta fase, como búsquedas YARA o ficheros PST, se interpretaron como coherentes con un estado basal sin evidencias específicas presentes.

4.2.3. Instrumentación con Sysmon, PowerShell 4104 y eventos

Windows

Para aumentar la visibilidad host-based, el endpoint se instrumentó con Sysmon y con PowerShell Script Block Logging. Esta configuración resulta esencial para las técnicas asociadas a ejecución, persistencia, discovery, impacto, sabotaje y staging.

Fuente	Evento	Qué aporta	Técnicas relacionadas
Sysmon ID 1	Process Creation	Creación de procesos, línea de comandos, imagen y usuario	TEC-001, TEC-002, TEC-003, TEC-004, TEC-006
Sysmon ID 3	Network Connection	Conexión de red asociada a proceso	TEC-009 contextual
Sysmon ID 11	FileCreate	Creación de ficheros, extensiones y rutas	TEC-007, TEC-009
Sysmon ID 12/13/14	Registry	Creación, modificación y borrado de claves/valores	TEC-004
Sysmon ID 22	DNS	Resoluciones DNS si están habilitadas	Apoyo contextual

Tabla 29: Fuentes de telemetría habilitadas y técnicas relacionadas

Nota: Sysmon ID 26 se utiliza como evidencia forense de borrado, no como alerta directa a Discord. Su envío sin consolidación podría producir una alerta por fichero eliminado y generar ruido operativo excesivo.

Tabla 30: Tratamiento metodológico de Sysmon Event ID 26

4.3. Metodología experimental

La metodología seguida fue iterativa e incremental. Aunque la memoria se presenta de forma secuencial, el desarrollo técnico combinó revisión documental, preparación del laboratorio, ejecución de pruebas, ajuste de artifacts, captura de evidencias y registro en la matriz experimental.

Además de las ejecuciones individuales, se empleó un runner global para repetir la campaña completa TEC-001 a TEC-009 bajo un patrón homogéneo. Este runner no forma parte de la detección, sino del procedimiento experimental.

De forma complementaria, se diseñó un runner de falsos positivos para generar actividad benigna controlada y evaluar si la lógica de detección producía ruido operativo o sobreclasificación.

4.3.1. Principios metodológicos

Principio	Aplicación en el TFM
Reproducibilidad	Cada técnica se ejecuta mediante procedimiento definido, entorno controlado y evidencias archivadas
Seguridad	No se ejecuta malware real ni se trabaja sobre datos legítimos; las acciones se limitan a directorios de prueba
Trazabilidad	Cada prueba queda vinculada a técnica MITRE, artifact, evidencia, resultado y benchmark
Separación conceptual	Se distingue visibilidad, detección, alerta en tiempo casi real y salida externa
Cobertura	La cobertura MITRE se calcula sobre 14 tácticas y se separa cobertura principal, ampliada y contextual

Tabla 31: Principios metodológicos aplicados en la fase experimental

4.3.2. Organización del entorno de trabajo

Para mantener la trazabilidad se separó la organización de la máquina física y la máquina virtual. En la máquina física se conservaron memoria, evidencias, planificación y documentación auxiliar. En la máquina virtual se mantuvieron los scripts de prueba, directorios de laboratorio, configuración de Velociraptor, Sysmon y ficheros de benchmark:

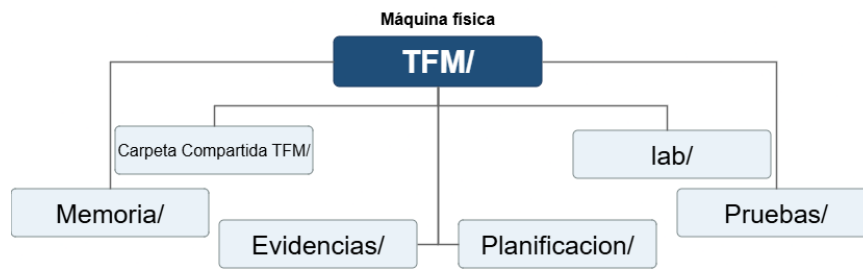


Figura 14: Estructura de directorios de la máquina física

Para la máquina virtual se usó la siguiente estructura de directorio:

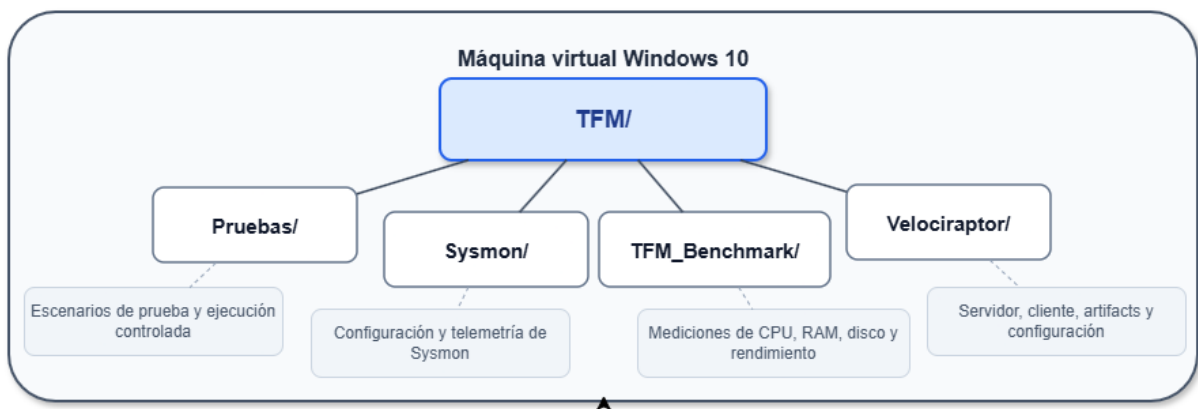


Figura 15: Directorio de la máquina virtual

4.3.3. Control de versiones y trazabilidad del desarrollo

El desarrollo aplicó control de versiones a scripts, artifacts, runners, configuraciones y documentos de apoyo. Cada elemento se vinculó a su versión, estado de validación, hash SHA-256 y evidencia asociada para conservar la trazabilidad entre código, ejecución y resultado.

La promoción de un componente a estado validado dependió de la revisión del autor y, cuando correspondía, de su ejecución en el laboratorio. La denominación del archivo se utilizó como identificador auxiliar y nunca como prueba suficiente de validez experimental.

Tabla 32: Criterios de control de versiones y trazabilidad del proyecto

Código 5: Control del laboratorio

La máquina virtual se guardó en snapshots antes y después de hitos relevantes. Esta práctica permitió restaurar el entorno tras pruebas con impacto, repetir escenarios y reducir el riesgo de contaminar evidencias entre técnicas.

La última de las snapshot fue la 05, ya que a partir de ahí no se crearon más pruebas con riesgo que pudieran salirse de control o tardaran en recuperarse.

Snapshot	Finalidad
00_Windows_Inicial	Estado base del sistema operativo
01_TFM_Velociraptor_Base_Lab	Velociraptor instalado y entorno inicial disponible
02_VR_Con_Artifacts_Funcionando	Artifacts base validados
03_TFM_Montado_Configurado	Estructura de trabajo y laboratorio preparados
04_Antes_de_persistencia	Punto previo a pruebas con cambios de configuración
05_Despues_de_ransom	Estado posterior a simulación controlada de cifrado
06_Iniciando_pruebas	Estado de inicio de Runners
VR_7_Probando_REDES	Probando el ataque de red con TEC-009
Instantánea 8 19062026	Estado final donde se ejecutan todas las pruebas

Tabla 33: Snapshots utilizados para control del laboratorio

4.3.4. Flujo general de trabajo

El trabajo experimental se desarrolló siguiendo un ciclo común para cada técnica analizada. Aunque cada caso requirió ajustes concretos, el flujo general fue el siguiente:

Paso	Actividad
1	Preparar o restaurar snapshot de laboratorio
2	Seleccionar técnica MITRE ATT&CK y objetivo de prueba
3	Diseñar ejecución controlada y segura
4	Ejecutar script o comando en el endpoint
5	Observar telemetría en Velociraptor, Sysmon y logs Windows.
6	Evaluar artifacts públicos disponibles
7	Ajustar consultas VQL o artifacts personalizados si procede
8	Registrar evidencias, capturas y referencias de logs
9	Clasificar resultado: visibilidad, detección, alerta RT y salida externa.
10	Medir impacto de rendimiento asociado

Paso	Actividad
11	Documentar limitaciones, ruido y falsos positivos
12	Pasar a la siguiente técnica manteniendo trazabilidad

Tabla 34: Flujo general de trabajo por técnica

4.3.5. Matriz principal de control experimental

El archivo Analisis_Tecnicas_TFM_Velociraptor_DEF.xlsx se empleó como matriz de control experimental. Su función fue centralizar la relación entre técnicas, artifacts, pruebas, evidencias, alertabilidad, brechas de cobertura, benchmark y decisiones metodológicas.

Hoja	Función metodológica
02_Seleccion_Tecnicas	Técnicas TEC-001 a TEC-009, táctica, evidencia esperada y artifacts candidatos
03_Evaluacion_VR	Registro consolidado de evaluación y resultados por prueba
04_Control_Publicos	Control operativo de pruebas con artifacts públicos
05_Matriz_Resultados	Matriz de resultados públicos por técnica
07_Catalogo_Artifacts	Catálogo de artifacts y rol metodológico
08_Benignas_FP	Operaciones benignas para contraste y falsos positivos
09_Resultados_Custom	Plan de medición por proceso y escenarios B0-B8
10_Gaps_Custom_P1P4	Brechas de artifacts públicos y necesidad de arquitectura custom
11_Alertabilidad	Modelo de visibilidad, detección, alerta RT y salida externa
12_Plan_Alertas	Plan de validación paso a paso de alertas
14_Arquitectura_Custom	Diseño P4-P1 con sources internas por técnica
15_Publicos_Definitivo	Matriz final de artifacts públicos y veredicto metodológico
16_Scripts_Validados	Scripts revisados, evidencia esperada y correcciones
17_Incoherencias_Cerradas	Decisiones definitivas para evitar contradicciones

Tabla 35: Hojas principales de la matriz de control experimental

Esta matriz evita depender de capturas aisladas. Cada técnica queda vinculada a su procedimiento, fuente de telemetría, artifact, evidencia, clasificación del resultado y medición de rendimiento.

4.3.6. Registro de evidencias

Las evidencias se organizaron en carpetas por técnica siguiendo una nomenclatura basada en fecha, identificador interno y técnica MITRE ATT&CK: AAAAMMDD_TEC-XXX_TXXXX-XXX. Cada carpeta puede contener capturas, consultas VQL, scripts, logs, exportaciones CSV/JSONL y resultados de benchmark.

Tipo de evidencia	Uso
Capturas de Velociraptor	Resultados visibles en GUI, Client Events, Hunts o Collections
Consultas VQL	Consulta ejecutada o fragmento relevante del artifact
Scripts / comandos	Procedimiento controlado ejecutado en la VM
Logs Windows / Sysmon	Event IDs asociados a la técnica
Exportaciones	CSV, JSONL o ZIP cuando sea necesario para anexo o análisis
Benchmark	CSV de Performance Monitor/logman y salida procesada

Tabla 36: Tipos de evidencias documentadas por técnica

4.3.7. Diseño de pruebas controladas

Las pruebas se diseñaron para reproducir comportamientos observables sin introducir daño real. En lugar de malware, se utilizaron comandos y scripts benignos capaces de generar telemetría equivalente a patrones host-based relevantes.

Criterio de seguridad	Aplicación
Datos dummy	Los ficheros modificados o cifrados pertenecen a directorios de laboratorio
Reversibilidad	Las pruebas incluyen limpieza, descifrado o restauración cuando procede
Aislamiento	La ejecución se limita a la VM y a rutas controladas
No explotación real	No se realiza explotación de vulnerabilidades ni movimiento lateral real
Exfiltración HTTP controlada en laboratorio	TEC-009 genera staging, archivado ZIP y envío HTTP controlado hacia un host receptor de laboratorio. No implica datos sensibles reales, terceros comprometidos ni uso de rutas, nombres de scripts o etiquetas TEC como indicadores de compromiso.

Criterio de seguridad	Aplicación
Separación de lógica	Los nombres de scripts, rutas TFM o marcadores TEC no se usan como criterio de detección final

Tabla 37: Controles de seguridad aplicados a las pruebas

4.3.8. Scripts de laboratorio validados

Los scripts usados para el laboratorio son:

Script	Rol
crear_directorio_dummy.ps1	Preparación dataset
enumeracion_v2.ps1	Enumeración previa TEC-007
Scriptransom_v2.ps1	TEC-007 cifrado controlado
descifrar.ps1	Restauración TEC-007
sabotaje.ps1	TEC-008 borrado controlado
exfiltracion_v3.ps1	TEC-009 staging / ZIP / upload controlado
TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1	Runner global TEC-001 a TEC-009
TFM_Run_FP_Tests_v1.ps1	Runner falsos positivos

Tabla 38: Scripts de laboratorio y papel metodológico

Nota: Los scripts permiten reproducibilidad, pero no deben convertirse en indicadores de detección. La lógica final debe basarse en comportamiento host-based: procesos, command-line, 4104, Sysmon, registro, servicios, ficheros o red.

Tabla 39: Criterios de validación de los scripts de laboratorio

4.4. Selección de técnicas MITRE ATT&CK

La selección de técnicas no busca cubrir toda la matriz MITRE ATT&CK Enterprise, sino construir un conjunto acotado, seguro y representativo para evaluar detección host-based en Windows. La selección final incorpora nueve técnicas internas TEC-001 a TEC-009, mapeadas contra ATT&CK Enterprise y documentadas en la matriz de evidencias del TFM (MITRE, s. f.-h; García-Amorena Reina, 2026e).

4.4.1. Criterios de selección

Criterio	Descripción
Observabilidad en host	La técnica debe generar señales consultables mediante procesos, logs, Sysmon, PowerShell, registro, ficheros o red local
Ejecución segura	Debe poder reproducirse sin malware real y sin afectar datos legítimos
Relación con ransomware/insiders	Se priorizan ejecución, persistencia, discovery, impacto, sabotaje, staging, archivado ZIP y exfiltración HTTP controlada en laboratorio
Reproducibilidad	Debe poder ejecutarse varias veces en la VM con procedimiento controlado
Valor para artifacts	Debe permitir comparar artifacts públicos con lógica personalizada
Impacto medible	Debe permitir medir coste operativo con y sin Velociraptor activo

Tabla 40: Criterios usados para seleccionar técnicas MITRE ATT&CK

4.4.2. Técnicas seleccionadas

La selección inicial se centró en siete técnicas, agrupadas en fases representativas de una cadena de ataque: ejecución, persistencia, discovery, servicios del sistema e impacto.

ID	MITRE	Técnica	Táctica
TEC-001	T1059.001	PowerShell	Execution
TEC-002	T1059.003	Windows Command Shell	Execution
TEC-003	T1053.005	Scheduled Task	Execution; Persistence; Privilege Escalation
TEC-004	T1547.001	Registry Run Keys / Startup Folder	Persistence; Privilege Escalation
TEC-005	T1569.002	Service Execution	Execution
TEC-006	T1518.001	Security Software Discovery	Discovery
TEC-007	T1486	Data Encrypted for Impact	Impact
TEC-008	T1485	Data Destruction	Impact
TEC-009	T1074.001 / T1560.001	Local Data Staging / Archive Collected Data / Exfiltration	Collection; Exfiltration HTTP controlada en laboratorio

ID	MITRE	Técnica	Táctica
	/ T1048.003	Over Unencrypted Non-C2 Protocol	

Tabla 41: Técnicas MITRE ATT&CK seleccionadas para la fase experimental

La selección final amplía el enfoque inicial centrado en ransomware incorporando sabotaje, staging, archivado ZIP y exfiltración HTTP controlada. Esta ampliación mejora la representatividad frente a escenarios de insiders y ataques orientados a impacto o transferencia de datos dummy, sin afirmar compromiso de datos sensibles ni de terceros (García-Amorena Reina, 2026e).

4.4.3. Tácticas ATT&CK representadas por el subconjunto evaluado

El denominador de 14 tácticas se mantiene como criterio metodológico. La cobertura se presenta en tres niveles para no inflar el alcance real del estudio: cobertura principal, cobertura ampliada y cobertura contextual con red.

Criterio	Tácticas consideradas	Cálculo	Cobertura
Cobertura principal estricta	Execution, Persistence, Discovery, Collection, Impact	5/14	35,71 %
Cobertura ampliada/secundaria	Las anteriores + Privilege Escalation contextual en TEC-003/TEC-004	6/14	42,86 %
Cobertura contextual con red	Las anteriores + Exfiltration mediante envío HTTP controlado documentado en TEC-009	7/14	50,00 %

Tabla 42: Tácticas ATT&CK representadas por el subconjunto evaluado empleando denominador 14

4.4.4. Decisión metodológica sobre TEC-009

TEC-009 se interpreta como staging, archivado ZIP y exfiltración HTTP controlada en laboratorio, asociada a T1074.001, T1560.001 y T1048.003. El runner independiente prepara datos dummy, crea el ZIP y registra ReceiverReachable=True, HTTPStatus=200, UploadSucceeded=True, SHA-256 local y bytes transferidos en ReceiverResponse. Ese ground truth acredita la ejecución y la respuesta del receptor, pero no constituye una alerta CLIENT_EVENT ni una detección de Generic.Events.TrackNetworkConnections. La detección Velociraptor custom se sustenta en las señales host de staging, archivado y ejecución; la atribución de la conexión a un proceso quedó limitada (MITRE, s. f.-a; MITRE, s. f.-g; MITRE, s. f.-i; García-Amorena Reina, 2026e).

Elemento	Decisión
Técnicas principales	T1074.001 Local Data Staging; T1560.001 Archive Collected Data; T1048.003 Exfiltration Over Unencrypted Non-C2 Protocol
Componente contextual	Exfiltración HTTP controlada hacia receptor de laboratorio documentada por summary
Acción ejecutada	Staging, Compress-Archive, ZIP, envío HTTP POST/upload, SHA-256 local y bytes transferidos
Lo que no se afirma	No se afirma exfiltración de datos sensibles reales ni compromiso de terceros; si falta receiver_log.jsonl o ZIP recibido en el paquete revisado, se mantiene como limitación documental secundaria
Evidencia esperada	PowerShell 4104, Sysmon ID 1/3/11, ReceiverReachable=True, HTTPStatus=200, UploadSucceeded=True, SHA-256 y ReceiverResponse

Tabla 43: Criterio de interpretación de TEC-009

4.5. Estrategia de artifacts y monitorización

La evaluación no se limita a ejecutar técnicas y comprobar si aparece algún evento. La estrategia de artifacts distingue artifacts públicos y artifacts personalizados. El objetivo es comprobar cuándo Velociraptor aporta visibilidad, cuándo clasifica una actividad y cuanto esfuerzo/tiempo hay que echar para generar una monitorización fuerte.

4.5.1. Artifacts públicos considerados

Antes de diseñar artifacts personalizados, se realizó una revisión de artifacts públicos y nativos de Velociraptor con posible utilidad para la detección de las técnicas MITRE ATT&CK seleccionadas. Esta revisión tuvo como objetivo establecer una línea base de comparación, es decir, comprobar hasta qué punto las capacidades ya disponibles en Velociraptor permiten observar, detectar o contextualizar los comportamientos ejecutados en el laboratorio sin necesidad de desarrollar lógica específica para este TFM.

Para clasificar la validez de estos artifacts públicos se considerará lo siguiente:

- Visibilidad: el artifact permite observar eventos o evidencias relacionadas con la técnica ejecutada.
- Detección: el artifact permite identificar un comportamiento relevante y asociarlo razonablemente a una técnica MITRE ATT&CK concreta.
- Alertabilidad: el artifact puede integrarse en un flujo de monitorización continua o alertado operativo.

Artifact público	Tipo	Fuente de telemetría	Técnicas relacionadas
Windows.Events. ProcessCreation	CLIENT_EVENT	Creación de procesos y línea de comandos	TEC-001, TEC-002, TEC-003, TEC-004, TEC-005, TEC-006
Windows.EventLogs.PowerShell / eventos PowerShell 4104	Consulta sobre logs / monitorización según configuración	PowerShell Script Block Logging	TEC-001, TEC-006, TEC-007, TEC-008, TEC-009
Windows.Events. ServiceCreation	CLIENT_EVENT	Eventos de creación de servicios Windows	TEC-005
Windows.Sysmon .* / artifacts	Consulta sobre	Sysmon Event ID 1, 3, 11, 12, 13, 14, 26	TEC-001 a TEC-009, según evento

Artifact público	Tipo	Fuente de telemetría	Técnicas relacionadas
basados en Sysmon	eventos Sysmon		
Generic.Events.TrackNetworkConnections	CLIENT_EVENT	Conexiones de red; sin atribución fiable de proceso en la campaña definitiva	TEC-009
Windows.System.Pslist	CLIENT / consulta puntual	Procesos activos	TEC-001, TEC-002, TEC-006
Windows.System.Services	CLIENT / consulta puntual	Servicios instalados y estado de servicios	TEC-005
Windows.Registry.* / artifacts de registro	CLIENT / consulta puntual	Claves y valores del registro de Windows	TEC-004
Windows.Hayabusa.Monitoring	Monitorización / correlación sobre eventos Windows	Logs de Windows procesados con lógica de detección	TEC-001 a TEC-009, según cobertura de reglas
Windows.ETW.Monitoring	CLIENT_EVENT	Eventos ETW	TEC-001 a TEC-006, según fuente habilitada

Tabla 44: Artifacts públicos considerados en la evaluación metodológica

Los artifacts públicos se emplean como línea base. Cuando aportan únicamente telemetría, no se contabilizan como detección completa. Cuando generan eventos en tiempo casi real, se valora si dicha señal contiene suficiente semántica para asociarla a una técnica MITRE concreta.

4.5.2. Brechas detectadas y necesidad de artifacts personalizados

La revisión de artifacts públicos permitió identificar que Velociraptor dispone de capacidades relevantes para obtener telemetría host-based, especialmente a partir de procesos, eventos de Windows, PowerShell, Sysmon, servicios, registro y conexiones de red. No obstante, desde el punto de vista de una evaluación HIDS, la existencia de telemetría no garantiza por sí sola una detección operativa.

En varias técnicas, los artifacts públicos pueden proporcionar eventos útiles, pero estos eventos requieren interpretación adicional para diferenciar actividad legítima de comportamiento potencialmente malicioso. Por ejemplo, la ejecución de powershell.exe, cmd.exe, reg.exe, schtasks.exe o sc.exe puede ser legítima en un entorno administrativo, por lo que su mera aparición no debe tratarse automáticamente como detección. Para que el evento tenga valor como alerta, debe contextualizarse mediante parámetros, línea de comandos, fuente del evento, técnica MITRE asociada, severidad y relación con el escenario ejecutado.

A partir de esta limitación, se identificaron las siguientes brechas metodológicas:

Brecha	Consecuencia
PowerShell legítimo y sospechoso comparten proceso	Riesgo de FP
CMD, reg, sc y schtasks pueden ser administrativos	Requieren contexto
Borrado masivo genera muchos eventos	Necesita agregación
ZIP local no implica exfiltración	Requiere red/upload
Sysmon aporta telemetría, no detección	Necesita lógica VQL
Discord valida salida, no calidad de detección	No debe contarse como detección

Tabla 45: Brechas y limitaciones de los artifacts públicos en la evaluación

Los artifacts públicos se emplean como línea base. Cuando aportan únicamente telemetría, no se contabilizan como detección completa. Cuando generan eventos en tiempo casi real, se valora si dicha señal contiene suficiente semántica para asociarla a una técnica MITRE concreta.

4.6. Benchmark y alertabilidad

Además de la capacidad de detección, se evaluó el impacto operativo de las pruebas sobre el endpoint. Para ello se utilizó Performance Monitor mediante logman¹², generando tres ficheros CSV con métricas del sistema en estado base sin VR y otros tres en estado base con VR.

Luego, para cada técnica se genera una medición normalmente cercana a un minuto, y siempre inferior a cinco minutos, suficiente para capturar el impacto inmediato de cada técnica sin introducir ruido excesivo por actividad ajena al experimento en el que se ejecuta el ataque. Esta medición se realiza con y sin VR activo igualmente para ver cuanto penaliza VR en la máquina.

Escenario	Objetivo
BASELINE_NO_VR	Medir el rendimiento basal de la máquina virtual sin el cliente Velociraptor activo.
VR_IDLE	Medir el impacto del cliente Velociraptor activo sin ejecutar la carga de técnicas.
VR_TEC_RUNNER	Medir el impacto del cliente Velociraptor durante la ejecución controlada del runner TEC.

Tabla 46: Escenarios ejecutados en el benchmark de rendimiento

Posteriormente, los CSV son procesados mediante scripts desarrollados para el TFM. Estos scripts extraen métricas globales como CPU, memoria e I/O, y métricas específicas del proceso cliente Velociraptor, evitando mezclar el coste del agente con el servidor/GUI del laboratorio.

Las métricas principales consideradas fueron:

- CPU total del sistema.
- RAM disponible.
- Lecturas de disco por segundo.
- Escrituras de disco por segundo
- CPU asociada al proceso de Velociraptor.
- Memoria privada del proceso de Velociraptor.
- I/O de lectura y escritura del proceso de Velociraptor.

4.6.1. Validación operativa mediante alertas

Como se ha comentado varias veces el objetivo es comprobar si una detección generada en el endpoint podía transformarse en una notificación externa accionable.

La arquitectura planteada fue la siguiente:

¹² Logman: herramienta nativa de Windows para automatizar la captura de contadores de rendimiento y exportarlos a registros analizables, como CSV.

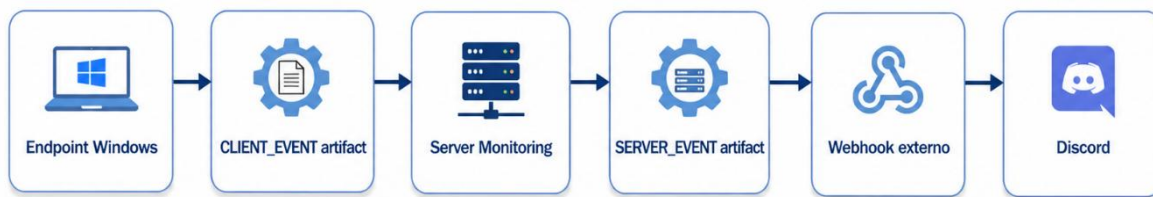


Figura 16: Arquitectura del flujo de alertas

En este modelo, el artifact de tipo `CLIENT_EVENT` observa eventos en el endpoint, el servidor recibe los resultados mediante `Server Monitoring` y un artifact de tipo `SERVER_EVENT` consume dichos eventos para enviarlos a un webhook externo.

Para la validación metodológica, una alerta se consideró correctamente generada cuando se cumplían las siguientes condiciones:

1. El evento era observado por Velociraptor.
2. La lógica de detección lo asociaba a una técnica concreta.
3. El `SERVER_EVENT` procesaba el evento.
4. El webhook devolvía respuesta satisfactoria.
5. La notificación llegaba correctamente a Discord.

De este modo, la prueba no se limitó a comprobar la existencia de telemetría, sino que verificó el flujo desde la observación del evento en el endpoint hasta su salida externa. Las capturas de Discord o JSONL se interpretan como notificación o persistencia posterior, no como fuente primaria de detección.

4.7. Decisiones metodológicas cerradas

1. Separar visibilidad, detección, alerta y salida externa: Evento observado $\neq$ detección MITRE $\neq$ alerta operativa $\neq$ Discord.
2. Usar Sysmon ID 26 para TEC-008: Se habilitó FileDeleteDetected porque registra borrados sin archivar los ficheros eliminados.
3. No usar Sysmon ID 23 como base principal: ID 23 puede archivar ficheros eliminados y generar volumen excesivo.
4. No enviar cada borrado a Discord: Para TEC-008, el borrado masivo puede generar una alerta por fichero; eso sería ruido operativo.
5. Distinguir TEC-007 y TEC-008.
 - a. Si hay cifrado, .aes, AES o crypto $\rightarrow$ TEC-007.
 - b. Si hay borrado directo con Remove-Item $\rightarrow$ TEC-008.
6. Distinguir TEC-009: staging, ZIP y envío HTTP controlado documentado permiten formular exfiltración HTTP controlada en laboratorio. No se afirma exfiltración de datos sensibles reales, ni compromiso de terceros, ni se usan rutas, scripts o etiquetas TEC como IOCs.

Capítulo V: Desarrollo experimental y ejecución de pruebas

En este capítulo se desarrolla la fase experimental del TFM mediante la ejecución controlada de las técnicas MITRE ATT&CK seleccionadas en el laboratorio Windows. Para cada escenario se describe la técnica evaluada, la herramienta o script utilizado, el procedimiento de ejecución, el comportamiento esperado en el endpoint y la telemetría generada.

Además, se documenta cómo Velociraptor recoge y procesa dicha telemetría mediante artifacts de fábrica y personalizados, incluyendo la validación de eventos, la generación de evidencias, el registro en la matriz experimental y la comprobación del flujo de alertado. Los resultados comparativos y su análisis agregado se abordarán posteriormente en el capítulo siguiente de resultados.

5.1. Escenarios de ataque

Para la ejecución de los ataques no se ha usado ejecución de malware real, sino en pruebas controladas y reproducibles diseñadas para generar telemetría host-based observable. El objetivo es provocar comportamientos de MITRE ATT&CK, pero sin introducir riesgo sobre la máquina virtual.

ID interno	Técnica MITRE ATT&CK	Táctica	Escenario de prueba
TEC-001	T1059.001 - PowerShell	Execution	Ejecución de PowerShell con parámetros visibles y flags relevantes.
TEC-002	T1059.003 - Windows Command Shell	Execution	Ejecución de comandos mediante cmd.exe.
TEC-003	T1053.005 - Scheduled Task	Persistence	Creación, ejecución y eliminación de una tarea programada.
TEC-004	T1547.001 - Registry Run Keys / Startup Folder	Persistence	Modificación controlada de claves Run del registro de Windows.
TEC-005	T1569.002 - Service Execution	Execution / System Services	Creación y ejecución de un servicio Windows controlado.
TEC-006	T1518.001 - Security Software Discovery	Discovery	Enumeración de productos de seguridad instalados mediante PowerShell/WMIC.
TEC-007	T1486 - Data Encrypted for Impact	Impact	Simulación controlada de cifrado de archivos dummy.
TEC-008	T1485 - Data Destruction	Impact	Simulación controlada de borrado masivo de ficheros dummy.

ID interno	Técnica MITRE ATT&CK	Táctica	Escenario de prueba
TEC-009	T1074.001 / T1560.001 - Local Data Staging / Archive Collected Data	Collection / Exfiltration	Preparación, compresión y posible envío controlado de datos dummy hacia un receptor de laboratorio.

Tabla 47: Escenarios de ataque evaluados

Los escenarios cubren distintas fases relevantes de una cadena de ataque: ejecución, persistencia, descubrimiento, abuso de servicios e impacto. Esta selección permite evaluar Velociraptor desde una perspectiva host-based, comprobando si puede transformar eventos del endpoint en detecciones útiles asociadas a técnicas MITRE ATT&CK.

5.2. Desarrollo experimental por técnica

Todas las pruebas se realizaron utilizando comandos o scripts controlados y reversibles, además cada técnica se ejecutó de forma aislada, siguiendo el criterio metodológico de una técnica = una prueba controlada. Cuando la prueba modificaba el sistema, se incluyó una fase de limpieza para devolver el entorno a un estado estable.

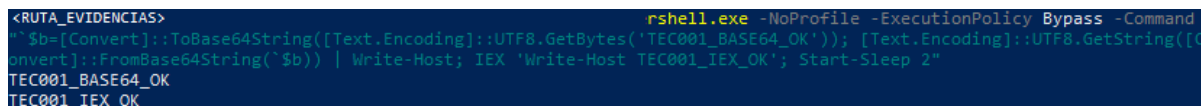
5.2.1. TEC-001 - Ejecución mediante PowerShell

La primera prueba se correspondió con T1059.001 - PowerShell. Se utilizó PowerShell porque es un intérprete legítimo de Windows ampliamente empleado en administración, pero también habitual en cadenas de ataque por su capacidad para ejecutar scripts, automatizar acciones y operar con parámetros avanzados.

- **Herramienta utilizada:** powershell.exe
- **Procedimiento ejecutado:**

```
$ powershell.exe -NoProfile -ExecutionPolicy Bypass -Command
"$b=[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('TEC001_BASE64_OK')); [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('$b')) | Write-Host; IEX 'Write-Host TEC001_IEX_OK'; Start-Sleep 2"
```

Código 6: Procedimiento de ejecución de TEC-001



```
<RUTA_EVIDENCIAS> powershell.exe -NoProfile -ExecutionPolicy Bypass -Command
"$b=[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('TEC001_BASE64_OK')); [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('$b')) | Write-Host; IEX 'Write-Host TEC001_IEX_OK'; Start-Sleep 2"
TEC001_BASE64_OK
TEC001_IEX_OK
```

Figura 17: Evidencia de ejecución de TEC-001

- **Comportamiento esperado en el host:**
 - creación de un proceso powershell.exe;

- presencia de -NoProfile;
- presencia de -ExecutionPolicy Bypass;
- marcador de laboratorio TEC001_OK;
- finalización automática tras unos segundos;
- visibilidad en VR;
- notificación/salida externa opcional mediante Discord.

Esta prueba sirvió como escenario inicial para validar ejecución de intérpretes de comandos y actividad PowerShell no destructiva.

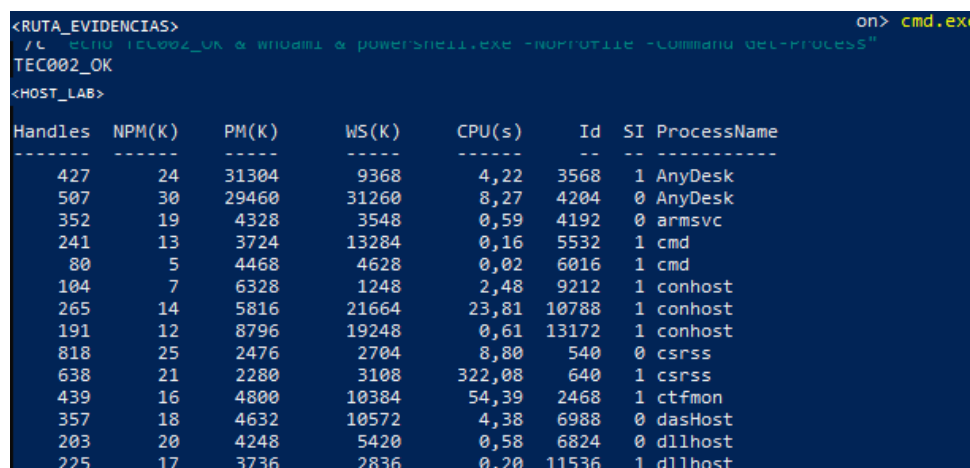
5.2.2. TEC-002 - Ejecución mediante Windows Command Shell

La segunda prueba se correspondió con T1059.003 - Windows Command Shell. Se utilizó cmd.exe para ejecutar un comando simple que generase un archivo temporal de evidencia. Esta técnica permite representar ejecución de comandos del sistema mediante una utilidad nativa de Windows. En el material de trabajo aparece una validación con cmd.exe /c y creación de fichero temporal con marcador TEC002_OK.

- **Herramienta utilizada:** cmd.exe
- **Procedimiento ejecutado:**

```
$ cmd.exe /c "echo TEC002_OK & whoami & powershell.exe -NoProfile -Command Get-Process"
```

Código 7: Procedimiento de ejecución de TEC-002



The screenshot shows a Windows Command Prompt window with a dark blue background. The prompt is on a line starting with '>'. The command entered is 'cmd.exe /c echo TEC002_OK & whoami & powershell.exe -NoProfile -Command Get-Process'. The output shows 'TEC002_OK' on the next line, followed by 'HOST-LAB' on the line after. Then, a list of running processes is displayed in a table format.

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
427	24	31304	9368	4,22	3568	1	AnyDesk
507	30	29460	31260	8,27	4204	0	AnyDesk
352	19	4328	3548	0,59	4192	0	armsvc
241	13	3724	13284	0,16	5532	1	cmd
80	5	4468	4628	0,02	6016	1	cmd
104	7	6328	1248	2,48	9212	1	conhost
265	14	5816	21664	23,81	10788	1	conhost
191	12	8796	19248	0,61	13172	1	conhost
818	25	2476	2704	8,80	540	0	csrss
638	21	2280	3108	322,08	640	1	csrss
439	16	4800	10384	54,39	2468	1	ctfmon
357	18	4632	10572	4,38	6988	0	dasHost
203	20	4248	5420	0,58	6824	0	dllhost
225	17	3736	2836	0,20	11536	1	dllhost

Figura 18: Evidencia de ejecución de TEC-002

- **Comportamiento esperado en el host:**
 - creación de un proceso cmd.exe;
 - uso del parámetro /c;
 - ejecución de un comando simple;
 - creación de un fichero temporal;

- presencia del marcador TEC002_OK;
- visibilidad en VR;
- notificación/salida externa opcional mediante Discord.

Esta prueba permitió simular ejecución básica de comandos sin afectar a componentes críticos del sistema.

5.2.3. TEC-003 - Creación de tarea programada

La tercera prueba se correspondió con T1053.005 - Scheduled Task. Se utilizó schtasks.exe para crear una tarea programada, ejecutarla y eliminarla posteriormente. El procedimiento documentado en las notas incluye creación, ejecución y limpieza de la tarea TFM-TEC-003.

- **Herramienta utilizada:** schtasks.exe
- **Procedimiento ejecutado:**

```

$ taskTime=(Get-Date).AddMinutes(5).ToString("HH:mm")

$ schtasks.exe /create /tn "TFM-TEC-003" /tr "cmd.exe /c echo TEC003_OK > C:\Users\Public\tec003_ok.txt" /sc once /st $taskTime /f

$ schtasks.exe /query /tn "TFM-TEC-003" /v /fo list

$ schtasks.exe /run /tn "TFM-TEC-003"

$ Start-Sleep 3

$ Get-Content "C:\Users\Public\tec003_ok.txt"

$ schtasks.exe /delete /tn "TFM-TEC-003" /f

```

Código 8: Procedimiento de ejecución de TEC-003

```

C:\RUTA_EVIDENCIAS>
//
>> schtasks.exe /create /tn "TFM-TEC-003" /tr "cmd.exe /c echo TEC003_OK > C:\Users\Public\tec003_ok.txt" /sc once /s
t $taskTime /f>>
>> schtasks.exe /query /tn "TFM-TEC-003" /v /fo list
>>
>> schtasks.exe /run /tn "TFM-TEC-003"
>>
>> Start-Sleep 3
>>
>> Get-Content "C:\Users\Public\tec003_ok.txt"
>>
>> schtasks.exe /delete /tn "TFM-TEC-003" /f
Correcto: se creó correctamente la tarea programada "TFM-TEC-003".

Carpeta: \
Nombre de host: LT29
Nombre de tarea: \TFM-TEC-003
Hora próxima ejecución: 24/06/2026 9:59:00
Estado: Listo
Modo de inicio de sesión: Solo interactivo
Último tiempo de ejecución: 24/06/2026 9:54:08
Último resultado: 0
Autor: <HOST_LAB>
Tarea que se ejecutará: cmd.exe /c echo TEC003_OK > C:\Users\Public\tec003_ok.txt
Iniciar en: N/A
Comentario: N/A
Estado de tarea programada: Habilitado
Tiempo de inactividad: Deshabilitado
Administración de energía: Detener en modo Batería, No iniciar en Batería
Ejecutar como usuario: <HOST_LAB>
Eliminar tarea si no se vuelve a programar: Deshabilitado
Eliminar tarea si ejecuta durante X horas y X minutos: 72:00:00
Programación: Datos de programación no disponibles en este formato.
Tipo de programación: Solo una vez
Hora de inicio: 9:59:00
Fecha de inicio: 24/06/2026
Fecha final: N/A
Días: N/A
Meses: N/A
Repetir: cada: Deshabilitado
Repetir: hasta: hora: Deshabilitado
Repetir: hasta: duración: Deshabilitado
Repetir: detener si aún se ejecuta: Deshabilitado
CORRECTO: se ha intentado ejecutar la tarea programada "TFM-TEC-003".
TEC003_OK
Correcto: se eliminó correctamente la tarea programada "TFM-TEC-003".

```

Figura 19: Evidencia de la tarea programada TEC-003

- **Comportamiento esperado en el host:**
 - creación de una tarea programada temporal;
 - ejecución de la tarea;
 - lanzamiento de cmd.exe como acción asociada;

- creación del fichero C:\Users\Public\tec003_ok.txt;
- eliminación posterior de la tarea;
- visibilidad en VR;
- notificación/salida externa opcional mediante Discord

Esta prueba reproduce de forma controlada un mecanismo típico de ejecución diferida o persistencia mediante tareas programadas.

5.2.4. TEC-004 - Persistencia mediante claves Run del registro

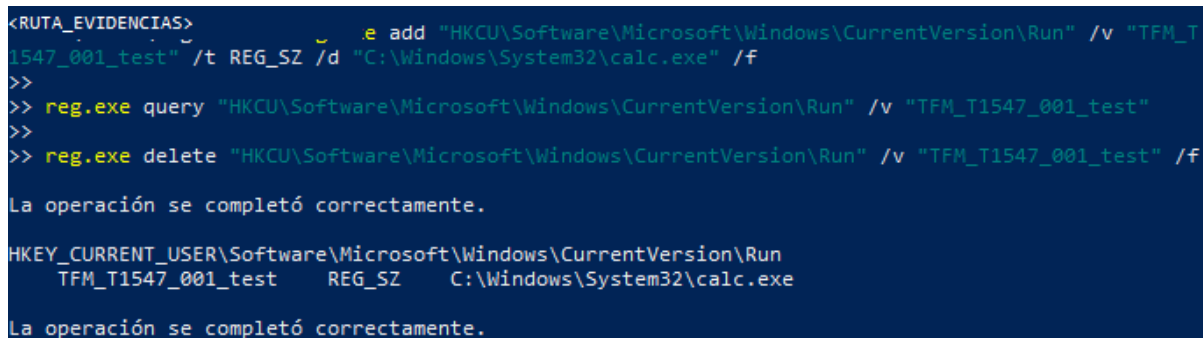
La cuarta prueba se correspondió con T1547.001 - Registry Run Keys / Startup Folder. Se utilizó reg.exe para crear una entrada temporal en la clave Run del usuario actual y eliminarla posteriormente. En las notas se documenta el uso de la clave HKCU\Software\Microsoft\Windows\CurrentVersion\Run con el valor TFM_T1547_001_test.

- **Herramienta utilizada:** reg.exe
- **Procedimiento ejecutado:**

```
$ reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v
"TFM_T1547_001_test" /t REG_SZ /d "C:\Windows\System32\calc.exe" /f

$ reg delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v
"TFM_T1547_001_test" /f
```

Código 9: Procedimiento de ejecución de TEC-004



```
<RUTA_EVIDENCIAS>
C:\Windows\System32\cmd.exe add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "TFM_T1547_001_test" /t REG_SZ /d "C:\Windows\System32\calc.exe" /f
>>
>> reg.exe query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "TFM_T1547_001_test"
>>
>> reg.exe delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "TFM_T1547_001_test" /f
La operación se completó correctamente.

HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
    TFM_T1547_001_test    REG_SZ    C:\Windows\System32\calc.exe
La operación se completó correctamente.
```

Figura 20: Evidencia de persistencia mediante clave Run en TEC-004

- **Comportamiento esperado en el host:**
 - ejecución de reg.exe;
 - creación de un valor temporal en HKCU\...\Run;
 - referencia al binario benigno calc.exe;
 - eliminación del valor creado;
 - ausencia de modificación persistente tras la limpieza;
 - visibilidad en VR;

- notificación/salida externa opcional mediante Discord

Esta prueba permitió representar una técnica clásica de persistencia sin instalar software adicional ni modificar claves críticas del sistema.

5.2.5. TEC-005 - Ejecución mediante servicio Windows

La quinta prueba se correspondió con T1569.002 - Service Execution. Se utilizó `sc.exe` desde una consola con privilegios de administrador para crear, arrancar y eliminar un servicio temporal. El procedimiento documentado emplea el nombre TFM TEC005 Test Service y ejecuta un comando benigno que crea un fichero de evidencia.

- **Herramienta utilizada:** `sc.exe`
- **Procedimiento ejecutado:**

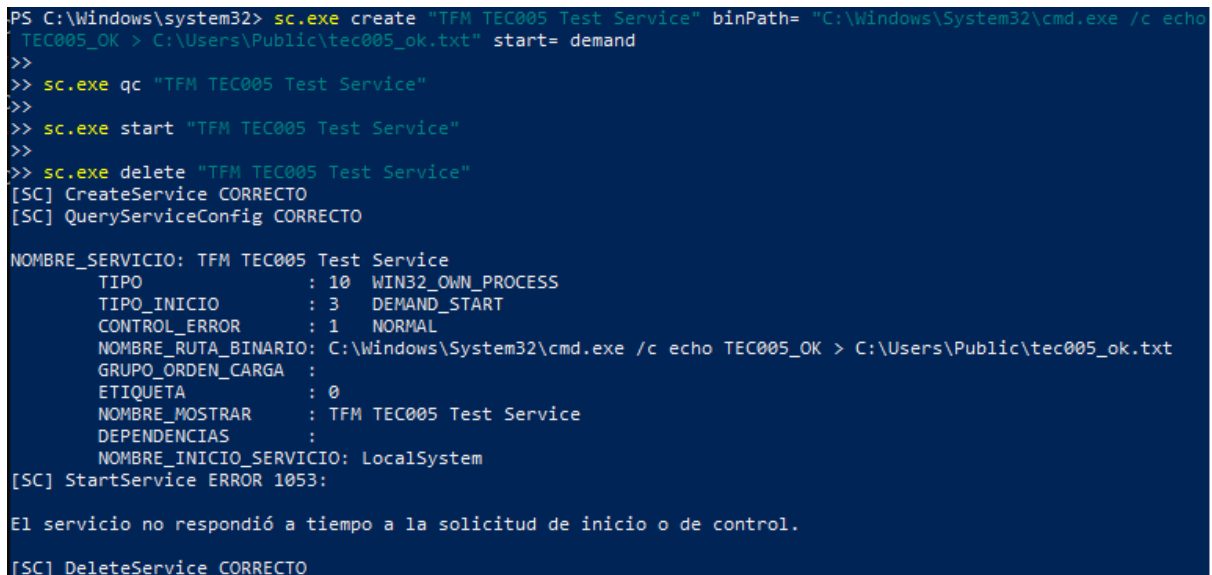
```
$ sc.exe create "TFM TEC005 Test Service" binPath=
"C:\Windows\System32\cmd.exe /c echo TEC005_OK >
C:\Users\Public\tec005_ok.txt" start= demand

$ sc.exe qc "TFM TEC005 Test Service"

$ sc.exe start "TFM TEC005 Test Service"

$ sc.exe delete "TFM TEC005 Test Service"
```

Código 10: Procedimiento de ejecución de TEC-005



```
PS C:\Windows\system32> sc.exe create "TFM TEC005 Test Service" binPath= "C:\Windows\System32\cmd.exe /c echo
TEC005_OK > C:\Users\Public\tec005_ok.txt" start= demand
>>
>> sc.exe qc "TFM TEC005 Test Service"
>>
>> sc.exe start "TFM TEC005 Test Service"
>>
>> sc.exe delete "TFM TEC005 Test Service"
[SC] CreateService CORRECTO
[SC] QueryServiceConfig CORRECTO

NOMBRE_SERVICIO: TFM TEC005 Test Service
TIPO           : 10  WIN32_OWN_PROCESS
TIPO_INICIO    : 3   DEMAND_START
CONTROL_ERROR  : 1   NORMAL
NOMBRE_RUTA_BINARIO: C:\Windows\System32\cmd.exe /c echo TEC005_OK > C:\Users\Public\tec005_ok.txt
GRUPO_ORDEN_CARGA :
ETIQUETA       : 0
NOMBRE_MOSTRAR  : TFM TEC005 Test Service
DEPENDENCIAS   :
NOMBRE_INICIO_SERVICIO: LocalSystem
[SC] StartService ERROR 1053:

El servicio no respondió a tiempo a la solicitud de inicio o de control.

[SC] DeleteService CORRECTO
```

Figura 21: Evidencia de ejecución del servicio TEC-005

- **Comportamiento esperado en el host:**
 - creación de un servicio Windows temporal;
 - arranque del servicio;

- ejecución de cmd.exe como binario asociado;
- creación del fichero C:\Users\Public\tec005_ok.txt;
- eliminación posterior del servicio;
- visibilidad en VR;
- notificación/salida externa opcional mediante Discord

Esta prueba reproduce abuso de servicios Windows como mecanismo de ejecución, manteniendo un flujo controlado y reversible.

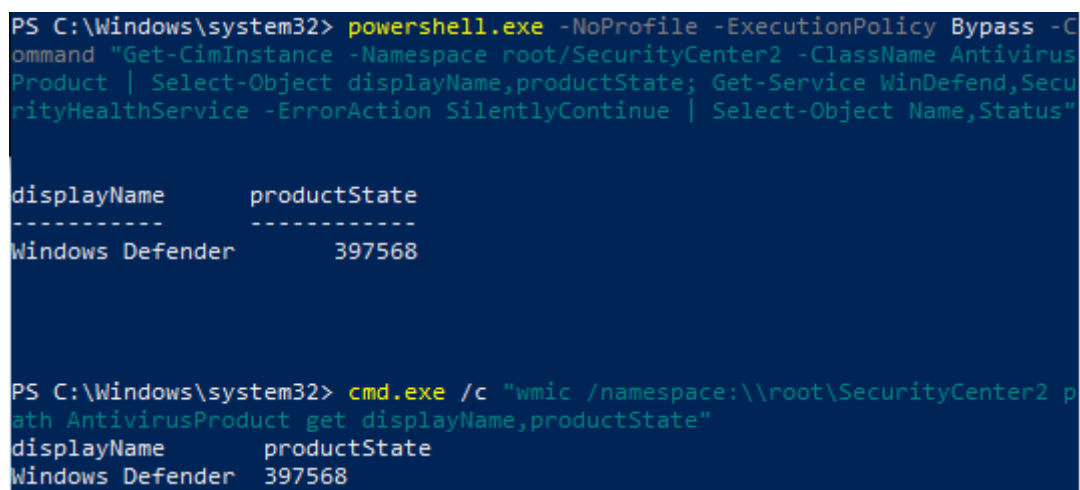
5.2.6. TEC-006 - Descubrimiento de software de seguridad

La sexta prueba se correspondió con T1518.001 - Security Software Discovery. Se ejecutaron consultas benignas para enumerar productos de seguridad registrados en root\SecurityCenter2, usando PowerShell y WMIC. En las notas se documentan ambas variantes, con Get-CimInstance y con wmic.

- **Herramienta utilizada:** powershell.exe, cmd.exe, wmic.exe
- **Procedimiento ejecutado mediante PowerShell:**

```
$ powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Get-
CimInstance -Namespace root/SecurityCenter2 -ClassName AntivirusProduct |
Select-Object displayName,productState; Get-Service WinDefend,SecurityHealthService -ErrorAction SilentlyContinue | Select-
Object Name,Status"
```

Código 11: Procedimiento de ejecución de TEC-006 mediante PowerShell



```
PS C:\Windows\system32> powershell.exe -NoProfile -ExecutionPolicy Bypass -C
ommand "Get-CimInstance -Namespace root/SecurityCenter2 -ClassName Antivirus
Product | Select-Object displayName,productState; Get-Service WinDefend,Secu
rityHealthService -ErrorAction SilentlyContinue | Select-Object Name,Status"

displayName      productState
-----
Windows Defender 397568

PS C:\Windows\system32> cmd.exe /c "wmic /namespace:\\root\SecurityCenter2 p
ath AntivirusProduct get displayName,productState"

displayName      productState
-----
Windows Defender 397568
```

Figura 22: Evidencia de descubrimiento de software de seguridad en TEC-006

- **Procedimiento ejecutado mediante WMIC:**

```
$ cmd.exe /c wmic /namespace:\\root\SecurityCenter2 path AntivirusProduct  
get displayName,productState
```

Código 12: Procedimiento de ejecución de TEC-006 mediante WMIC

- **Comportamiento esperado en el host:**
 - ejecución de PowerShell o CMD;
 - consulta a root\SecurityCenter2;
 - enumeración de productos AntivirusProduct;
 - posible aparición de WMIC.exe como proceso hijo;
 - ausencia de modificación sobre los productos de seguridad;
 - visibilidad en VR;
 - notificación/salida externa opcional mediante Discord

Esta prueba representa una fase de discovery en la que un atacante podría identificar soluciones defensivas antes de ejecutar acciones posteriores.

5.2.7. TEC-007 - Cifrado de datos

La séptima prueba se correspondió con T1486 - Data Encrypted for Impact, técnica principal del TFM por su relación directa con ransomware. Se ejecutó un simulador benigno desarrollado en PowerShell, denominado Scriptransom.ps1, sobre una carpeta de laboratorio con datos dummy. El flujo documentado incluye lectura de lista_archivos.csv y password.txt, creación de backups, generación de ficheros temporales cifrados y salida final con extensión. aes.

- **Herramienta utilizada:** powershell.exe
- **Procedimiento ejecutado**¹³

¹³ Para la técnica TEC-007 se desarrollaron varios scripts y un entorno concreto de trabajo. Todo esto se encuentra documentado como repositorio complementario en GitHub: Julio García-Amorena Reina, Controlled File Encryption, 2026. Disponible en: <https://github.com/Juliog-a/Controlled-File-Encryption>.

```
$ powershell.exe -ExecutionPolicy Bypass -File "<HOST_LAB>\
Desktop\TFM\Pruebas\Ransomware\Scriptransom.ps1"
```

Código 13: Procedimiento de ejecución de TEC-007

```

=== TEC-007 - T1486 Data Encrypted for Impact ===
Script enumeration: (<RUTA_ARTIFACTS>)
Script cifrado: C:<RUTA_ARTIFACTS>
Dataset: C:<RUTA_EVIDENCIAS>

ANTES:

TXT saved: C:<RUTA_EVIDENCIAS>
CSV saved: C:<RUTA_EVIDENCIAS>
Files enumerated: 50
Controlled encryption completed.
Encrypted files: 25
Validation: .aes files generated before restore.

DESPUES:
Count Name
-----
    50 .txt
    25 .aes
    25 .txt
    25 ....

MUESTRA .AES:
d...
d...
d...
d...
d...
d...
d...
d...
d...
d...
d...

PS <RUTA_EVIDENCIAS> > .\Scriptransom.ps1
=== INICIANDO PRUEBA TFM TEC-007 ===
WorkDir: C:<RUTA_EVIDENCIAS>
ListFile: C:<RUTA_EVIDENCIAS>
AES encrypted files: 25
AES evidence: <RUTA_EVIDENCIAS>      _32.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _33.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _34.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _35.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _36.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _37.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _38.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _39.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _40.txt.aes
AES evidence: <RUTA_EVIDENCIAS>      _41.txt.aes

```

Figura 23: Evidencia de cifrado controlado en TEC-007

- **Comportamiento esperado en el host:**
 - ejecución de PowerShell con ExecutionPolicy Bypass;
 - lanzamiento del script `Scriptransom.ps1`;
 - lectura de ficheros auxiliares del laboratorio;
 - creación de carpeta backups;
 - generación de archivos cifrados con extensión. aes;
 - operación limitada exclusivamente a datos dummy;
 - visibilidad en VR;

- notificación/salida externa opcional mediante Discord

La prueba se ejecutó sobre un directorio aislado de laboratorio dummy y con snapshot previo recomendado.

5.2.8. TEC-008 – Destrucción de datos

La octava prueba se correspondió con T1485 - Data Destruction. Esta técnica representa acciones orientadas a destruir, borrar o inutilizar datos del sistema. En el laboratorio se simuló mediante un script benigno de PowerShell, denominado sabotaje.ps1, limitado exclusivamente al directorio de pruebas <RUTA_EVIDENCIAS>.

A diferencia de TEC-007, esta prueba no reproduce cifrado ni generación de ficheros .aes, sino borrado directo de archivos dummy mediante comandos de eliminación. Esta distinción es metodológicamente importante: TEC-007 se asocia a cifrado para impacto, mientras que TEC-008 se asocia a destrucción directa de datos.

- **Herramienta utilizada:** powershell.exe

```
$ powershell.exe -ExecutionPolicy Bypass -File "<HOST_LAB>\Desktop
\TFM\01_ACTIVE_TESTS\Pruebas\TEC-008_Data_Destruction\sabotaje.ps1"
```

Código 14: Procedimiento de ejecución de TEC-008



Figura 24: Script de borrado de fichero

- **Comportamiento esperado en el host:**
 - ejecución de powershell.exe;
 - uso de ExecutionPolicy Bypass;
 - lanzamiento del script sabotaje.ps1;
 - enumeración de ficheros dentro del directorio de laboratorio;
 - eliminación de archivos dummy mediante Remove-Item;

- generación de eventos asociados a borrado de ficheros;
- ausencia de afectación sobre datos reales del sistema;
- posibilidad de restauración mediante recreación del dataset dummy;
- visibilidad en logs de PowerShell y Sysmon;
- posible generación de alerta si la lógica de detección identifica borrado masivo.

La telemetría esperada para esta técnica se basa principalmente en PowerShell Script Block Logging, especialmente Event ID 4104, y en eventos de Sysmon asociados a borrado de ficheros. En la configuración metodológica del TFM se decidió emplear Sysmon Event ID 26 como evidencia forense de eliminación, evitando utilizar cada evento de borrado individual como alerta directa para no generar ruido operativo excesivo ni carga exagerada al sistema (Microsoft, 2026; Russinovich & Garnier, 2026).

5.2.9. TEC-009 - Staging, archivado ZIP y exfiltración HTTP controlada

La novena prueba combina T1074.001 - Local Data Staging, T1560.001 - Archive Collected Data y T1048.003 - Exfiltration Over Unencrypted Non-C2 Protocol. Representa una exfiltración HTTP controlada en laboratorio: preparación de datos dummy, compresión en ZIP y transferencia HTTP hacia un receptor controlado (MITRE, s. f.-a; MITRE, s. f.-g; MITRE, s. f.-i).

En el laboratorio, TEC-009 se diseñó como staging, archivado ZIP y exfiltración HTTP controlada hacia un receptor del investigador. La ejecución prepara datos dummy, genera un archivo comprimido, calcula un SHA-256 local y realiza un envío HTTP controlado equivalente a Invoke-WebRequest, Invoke-RestMethod o upload HTTP. Las rutas y nombres de scripts se documentan por reproducibilidad, pero no forman parte de la lógica de detección (García-Amorena Reina, 2026e).

```

$ cd <RUTA_ARTIFACTS>

$ python .\receiver_tfm_v4.py `

    --host 0.0.0.0 `

    --port 8088 `

    --out-dir .\received `

    --log-file .\received\receiver_log.jsonl `

    --max-seconds 600 `

    --max-uploads 5 `

    --max-bytes 104857600

```

Código 15: Ejecución del receptor HTTP controlado

```

<RUTA_ARTIFACTS>
>>
>> python .\receiver_tfm_v4.py `
>> --host 0.0.0.0 `
>> --port 8088 `
>> --out-dir .\received `
>> --log-file .\received\receiver_log.jsonl `
>> --max-seconds 600 `
>> --max-uploads 5 `
>> --max-bytes 104857600
TFM receiver v4 listening on http://0.0.0.0:8088/upload
Health URL: http://0.0.0.0:8088/health
C<RUTA_EVIDENCIAS>
<RUTA_EVIDENCIAS>
Limits: max_seconds=600, max_uploads=5, max_bytes=104857600
[2026-06-24T07:46:53.237845+00:00] <HOST_LAB> - "POST /upload HTTP/1.1" 200 -
<HOST_LAB> - "POST /upload HTTP/1.1" 200 -

```

Figura 25: Evidencia del receptor HTTP controlado

- **Herramientas utilizadas: PowerShell y receptor HTTP controlado.**

correlaciona con ZIP, envío HTTP, hash, bytes transferidos y contexto de ejecución (García-Amorena Reina, 2026e).

5.2.10. Runner de ejecución global de técnicas

Además de la ejecución individual de cada técnica, se desarrolló un runner global para automatizar la campaña completa de pruebas TEC-001 a TEC-009. El objetivo de este script fue garantizar una ejecución homogénea, reproducible y trazable de todas las técnicas seleccionadas, reduciendo errores derivados de la ejecución manual y facilitando la comparación posterior entre telemetría, detección, alerta y rendimiento.

El runner utilizado fue TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1, ya que se hicieron pruebas con múltiples versiones anteriores. Este script ejecuta de forma secuencial las técnicas definidas en el laboratorio, introduce pausas entre pruebas, registra tiempos de inicio y fin, valida evidencias generadas y almacena los resultados en ficheros de salida. Además, integra comprobaciones sobre eventos relevantes de Sysmon y PowerShell 4104, lo que permite relacionar cada técnica con la telemetría generada en el endpoint.

El script se ejecutó desde la carpeta de runners del TFM:

```
$ cd "<HOST_LAB>\Desktop\TFM\03_RUNNERS"

$ Set-ExecutionPolicy -Scope Process Bypass -Force

$env:TFM_BASEPATH = "<HOST_LAB>\Desktop\TFM\01_ACTIVE_TESTS
\Pruebas"

$ .\TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1

-ReceiverUrl http://<HOST_LAB>:8088/upload

-EnableExfilUpload $true

-ExfilTimeoutSec 60

-KeepExfilArtifacts $true

-KeepArtifacts $true
```

Código 17: Ejecución del runner global de técnicas TEC-001 a TEC-009



El parámetro TFM_BASEPATH fija la ruta base del entorno activo de pruebas. A partir de ella, el runner localiza el dataset de laboratorio, las carpetas específicas de las técnicas, los scripts auxiliares y las rutas de salida. Para TEC-009 se utiliza ReceiverUrl, que indica el receptor HTTP controlado empleado para documentar la exfiltración HTTP controlada en laboratorio.

El runner no modifica artifacts, reglas ni configuración del sistema de alertado. Su función es exclusivamente ejecutar las técnicas de laboratorio y generar evidencias operativas. Las acciones se mantienen dentro del dataset dummy y de las rutas controladas del TFM.

Las funciones principales del runner son:

- ejecutar TEC-001 a TEC-009 de forma secuencial;
- restaurar el dataset de laboratorio cuando una técnica lo modifica;
- evitar que TEC-007, TEC-008 o TEC-009 contaminen pruebas posteriores;
- registrar hora de inicio, hora de fin y duración de cada técnica;
- recoger códigos de salida de los comandos ejecutados;
- comprobar evidencias esperadas por técnica;
- contar eventos relevantes en Sysmon y PowerShell 4104;
- generar logs y resúmenes de campaña;
- facilitar la repetición de pruebas en campañas posteriores.

En el caso de TEC-007, el runner ejecuta la enumeración del dataset y el script de cifrado controlado, validando posteriormente la aparición de archivos .aes. En TEC-008, ejecuta un borrado controlado sobre ficheros dummy y compara el número de archivos antes y después de la operación. En TEC-009, ejecuta la prueba de staging, ZIP y upload HTTP, registra el receptor configurado, el estado del upload, el hash SHA-256 del ZIP generado y el resumen de transferencia.

La ejecución genera evidencias en la carpeta de logs del laboratorio, incluyendo un log detallado de campaña y ficheros resumen en formato TXT, JSON y CSV. Estos ficheros permiten reconstruir la ejecución completa sin depender únicamente de capturas de pantalla.

Las salidas principales son:

- log completo de ejecución;
- resumen textual de la campaña;
- resumen JSON estructurado;
- resumen CSV para análisis tabular;
- evidencias por técnica;
- recuentos de eventos Sysmon y PowerShell 4104.

El uso del runner fue especialmente importante para asegurar que las campañas de Velociraptor, Wazuh base y Wazuh custom se ejecutaran sobre un patrón de actividad comparable. De este modo, las

diferencias observadas posteriormente pueden atribuirse principalmente a la capacidad de visibilidad, detección y alertado de cada herramienta, y no a variaciones manuales en la ejecución de las pruebas.

5.2.11. Runner de falsos positivos

Además del runner de técnicas MITRE ATT&CK, se desarrolló un runner específico de falsos positivos denominado TFM_Run_FP_Tests_v1.ps1. Su objetivo fue ejecutar actividad benigna y controlada que pudiera parecer sospechosa desde el punto de vista de la telemetría, pero que no debía clasificarse automáticamente como ataque.

Esta prueba es relevante porque una herramienta HIDS no solo debe detectar comportamientos potencialmente maliciosos, sino también reducir el ruido operativo. Una detección útil debe diferenciar entre actividad maliciosa, actividad sospechosa y actividad administrativa legítima. Por tanto, el runner de falsos positivos permite evaluar la precisión de los artifacts y reglas desarrolladas, así como el riesgo de generar alertas innecesarias ante operaciones normales del sistema.

El runner se ejecutó desde la carpeta de runners del TFM:

```
$ cd "<HOST_LAB>\Desktop\TFM\03_RUNNERS"
$ Set-ExecutionPolicy -Scope Process Bypass -Force
$ .\TFM_Run_FP_Tests_v1.ps1
```

Código 18: Ejecución del runner de falsos positivos

```
===== TFM FP RUN SUMMARY =====
RunId: TFM_FP_20260624_152518
GlobalStatus: WARN
OK/WARN/FAIL/SKIPPED: 9/0/0/1
Total VR hits: 0
```

```
<RUTA_EVIDENCIAS>                                rts_fp_clean.json1"
>>
>> if (Test-Path $fpJson) {
>>     $rows = Get-Content $fpJson |
>>         Where-Object { $_.Trim() -ne "" } |
>>         ConvertFrom-Json
>>
>>     "TOTAL_ALERTAS_FP = $($rows.Count)"
>>
>>     $rows |
>>     Group-Object Profile |
>>     Sort-Object Count -Descending |
>>     Select-Object Name,Count |
>>     Format-Table -AutoSize
>>
>>     $rows |
>>     Group-Object TEC |
>>     Sort-Object Count -Descending |
>>     Select-Object Name,Count |
>>     Format-Table -AutoSize
>>
>>     $rows |
>>     Select-Object Timestamp,Profile,TEC,MITRE,DetectionName,Artifact |
>>     Format-Table -AutoSize
>> }
TOTAL_ALERTAS_FP = 0
```

El script genera un identificador único de ejecución, crea un workspace temporal y almacena los resultados en la carpeta de falsos positivos del laboratorio. Además, puede consultar el fichero JSONL de alertas generado por el flujo de Velociraptor para comprobar si las operaciones benignas han producido alertas P1, P2, P3 o P4.

La ruta de salida utilizada por defecto es <HOST_LAB>\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs.

El runner genera los siguientes ficheros principales:

- log detallado de ejecución;
- resumen TXT;
- resumen JSON;
- resumen CSV;
- fichero TXT con posibles hits detectados por Velociraptor;
- fichero CSV con posibles hits detectados por Velociraptor.

Las pruebas benignas incluidas fueron:

ID	Descripción	Finalidad
FP-001	PowerShell administrativo legítimo	Verificar si comandos administrativos básicos generan alertas indebidas.
FP-002	CMD legítimo	Comprobar ruido asociado a ejecución normal de cmd.exe.
FP-003	Scheduled Task benigno temporal	Evaluar si una tarea programada legítima se clasifica como persistencia maliciosa.
FP-004	Registro benigno no persistente	Comprobar ruido en operaciones de registro sin finalidad maliciosa.
FP-005	Consulta de servicios	Validar si consultas normales sobre servicios generan detecciones.
FP-006	Discovery legítimo de seguridad	Contrastar actividad administrativa similar a Security Software Discovery.
FP-007	ZIP benigno local	Evaluar si una compresión local legítima se confunde con staging.
FP-008	Borrado benigno controlado	Medir ruido ante eliminación controlada de ficheros temporales.

ID	Descripción	Finalidad
FP-009	HTTP benigno local	Evaluar si una conexión HTTP controlada genera alerta de exfiltración.
FP-010	Operaciones normales masivas	Medir sensibilidad ante actividad de ficheros de volumen elevado.

Tabla 48: Casos benignos incluidos en el runner de falsos positivos

Cada caso registra estado, duración, artefactos generados y detalles de ejecución. Posteriormente, el runner revisa la ventana temporal correspondiente en el fichero `soc_alerts.jsonl` para identificar si alguna operación benigna ha generado una alerta. Cuando se encuentra una coincidencia, el script clasifica el impacto potencial del falso positivo según el perfil de severidad observado.

La interpretación de este runner es distinta a la del runner principal. No se utiliza para medir cobertura MITRE, sino para evaluar ruido, precisión y carga operativa. Una alerta generada durante una prueba de falso positivo no implica una detección correcta, sino una posible sobreclasificación que debe revisarse manualmente.

El resultado de estas pruebas permite valorar:

- si las reglas son demasiado genéricas;
- si PowerShell o CMD generan ruido excesivo;
- si las reglas P4 aportan visibilidad útil o demasiadas alertas;
- si las técnicas de staging, ZIP o HTTP requieren mayor contexto;
- si las reglas custom necesitan exclusiones, umbrales o correlación adicional.

Por tanto, el runner de falsos positivos complementa la validación principal. Mientras el runner global evalúa la capacidad de detectar técnicas MITRE ATT&CK controladas, el runner de falsos positivos evalúa la capacidad de no alertar indebidamente ante actividad benigna. Esta distinción resulta esencial para valorar la utilidad operativa de Velociraptor como aproximación HIDS.

5.3. Recopilación de telemetría

La recopilación de telemetría se diseñó para mantener la trazabilidad entre la técnica ejecutada, la fuente de datos observada y la evidencia conservada. En esta fase no se valoran todavía los resultados de detección, sino las fuentes desde las que se espera obtener información útil para el análisis posterior.

La telemetría se agrupó en tres niveles:

- telemetría generada por el propio sistema Windows;
- telemetría observada y procesada por Velociraptor;
- salida externa generada por el flujo de alertado hacia Discord.

Esta separación permite distinguir entre evento bruto, detección, alerta y evidencia. Un evento registrado en Sysmon o PowerShell no implica por sí mismo una detección MITRE. Del mismo modo, una notificación enviada a Discord solo valida la salida externa del flujo, no la calidad de la lógica de detección.

5.3.1. Telemetría captada en el sistema

La primera fuente de información procede del propio endpoint Windows. Para ello se utilizaron principalmente Sysmon, PowerShell Script Block Logging y eventos nativos de Windows.

Las fuentes principales fueron:

- Sysmon Event ID 1: creación de procesos, línea de comandos, usuario, imagen ejecutada y proceso padre.
- Sysmon Event ID 3: conexiones de red asociadas a procesos.
- Sysmon Event ID 11: creación de ficheros.
- Sysmon Event ID 12/13/14: creación, modificación y borrado de claves o valores de registro.
- Sysmon Event ID 26: evidencia de borrado de ficheros, usada especialmente para TEC-008.
- PowerShell Event ID 4104: bloques de código PowerShell ejecutados.
- Windows System Event ID 7045: instalación o creación de servicios Windows.
- Logs del receptor HTTP: evidencia auxiliar para TEC-009 cuando el upload HTTP está habilitado.

La siguiente tabla resume la telemetría esperada por técnica:

Técnica	Telemetría esperada
TEC-001	Sysmon ID 1 por ejecución de powershell.exe; PowerShell 4104 con parámetros como NoProfile, ExecutionPolicy Bypass, Base64 o IEX.
TEC-002	Sysmon ID 1 por ejecución de cmd.exe; línea de comandos; posible proceso hijo de PowerShell; fichero de evidencia temporal.
TEC-003	Sysmon ID 1 por schtasks.exe y cmd.exe; eventos asociados a creación, ejecución y eliminación de tarea programada.
TEC-004	Sysmon ID 1 por reg.exe; Sysmon ID 12/13/14 por modificación temporal de clave Run del registro.

Técnica	Telemetría esperada
TEC-005	Sysmon ID 1 por sc.exe y cmd.exe; Windows System Event ID 7045 por creación del servicio temporal.
TEC-006	Sysmon ID 1 por powershell.exe, cmd.exe o wmic.exe; PowerShell 4104; consultas a root\SecurityCenter2.
TEC-007	Sysmon ID 1 por powershell.exe; PowerShell 4104; Sysmon ID 11 por creación de ficheros .aes y backups.
TEC-008	Sysmon ID 1 por powershell.exe; PowerShell 4104; Sysmon ID 26 por borrado de ficheros dummy.
TEC-009	Sysmon ID 1 por powershell.exe; PowerShell 4104; Sysmon ID 11 por creación del ZIP; Sysmon ID 3 si existe conexión HTTP; logs del receiver si hay upload.

Tabla 49: Telemetría esperada por técnica experimental

En las técnicas TEC-007, TEC-008 y TEC-009 se evita utilizar cada evento como alerta directa, ya que un borrado/cifrado/envío masivo puede generar un volumen elevado de eventos individuales.

TEC	Estado	Evidencia VR
TEC-001	OK	TEC001_PowerShell_4104, 1 fila, Confidence=HIGH, FromBase64String, IEX
TEC-002	OK	TEC002_CMD_Sysmon, 27 filas, cmd.exe /c, algo ruidoso
TEC-003	OK	TEC003_ScheduledTask_Sysmon, 8 filas, schtasks.exe create/delete
TEC-004	OK	TEC004_RunKey_Sysmon, 6 filas, reg.exe, Run key
TEC-005	OK	TEC005_Service_System7045, 1 fila, System 7045
TEC-006	OK	TEC006_SecuritySoftwareDiscovery_4104, 2 filas; Sysmon fallback 5 filas
TEC-007	OK	TEC007_DataEncryptedForImpact_4104, 4 filas, crypto/AES
TEC-008	OK	TEC008_DataDestruction_4104, 1 fila, Get-ChildItem, Remove-Item, -Force
TEC-009	OK parcial	P1 4104 fuerte real + P3 Sysmon process context; P3 4104 fuerte tiene ruido

Figura 27: Evidencias obtenidas en Velociraptor

5.3.2. Telemetría captada en Velociraptor

Velociraptor actúa como capa de recogida, filtrado y estructuración de eventos. La telemetría generada por Windows, Sysmon y PowerShell puede ser consultada mediante artifacts de fábrica o procesada mediante artifacts personalizados.

En el laboratorio se emplearon dos tipos de observación:

- observación puntual, mediante collections o consultas sobre logs;

- monitorización continua, mediante artifacts de tipo CLIENT_EVENT.

La monitorización continua es la más relevante para la aproximación HIDS, ya que permite emitir eventos durante la ejecución de la técnica. Aun así, la existencia de eventos en Velociraptor no equivale automáticamente a detección. Para que exista detección, el artifact debe aplicar una lógica capaz de clasificar la actividad como relevante y asociarla a una técnica o perfil concreto.

La telemetría observada por Velociraptor se utilizó para:

- registrar procesos y líneas de comando;
- consultar eventos PowerShell 4104;
- consultar eventos Sysmon;
- observar modificaciones del registro;
- detectar creación de servicios;
- identificar creación o eliminación de ficheros;
- observar conexiones de red asociadas al endpoint;
- normalizar eventos mediante artifacts personalizados;
- generar salidas estructuradas para análisis posterior.

Las evidencias obtenidas desde Velociraptor se conservaron como capturas de GUI, resultados de Client Events, exportaciones CSV/JSONL y registros de ejecución asociados a cada técnica.

5.3.3. Salida externa de alertado (JSONL/Discord)

Discord no se considera una fuente primaria de telemetría ni un mecanismo de detección del host, sino un canal externo de notificación. Su función dentro del TFM es comprobar que una detección generada por un artifact CLIENT_EVENT puede procesarse mediante SERVER_EVENT y llegar a una salida externa.

En las capturas, el campo denominado «IOC» se interpreta como contexto o IOA capturado por el laboratorio cuando contiene rutas, nombres de scripts o etiquetas TEC. Esos valores no son IOC reales ni condiciones de detección y no se utilizan para acreditar cobertura.

El flujo utilizado fue:

CLIENT_EVENT → SERVER_EVENT → JSONL / Webhook → Discord

En este modelo, el artifact CLIENT_EVENT observa el comportamiento en el endpoint. Posteriormente, un artifact SERVER_EVENT procesa los eventos recibidos por el servidor, genera una salida estructurada y la envía a un webhook externo.

Para considerar válida la salida externa se comprobaron los siguientes elementos:

- existencia del evento en Velociraptor;
- procesamiento por el artifact SERVER_EVENT;
- generación de una entrada estructurada en formato JSONL;
- envío mediante webhook;
- respuesta satisfactoria del servicio externo;
- recepción visible de la notificación en Discord.

Las evidencias asociadas a esta salida externa incluyen:

- capturas de Discord;
- fichero soc_alerts.jsonl;
- timestamp de generación;
- perfil de severidad asignado;
- técnica asociada;
- nombre del artifact que originó el evento;
- campos principales del evento procesado.

La notificación en Discord no se interpreta como detección por sí misma. Su valor metodológico consiste en demostrar que la detección puede salir del entorno de Velociraptor y llegar a un canal externo, manteniendo trazabilidad con el evento original.

Prueba	Acción benigna	Estado	Hits custom
FP-001	PowerShell administrativo legítimo	OK	0
FP-002	CMD legítimo	OK	0
FP-003	Scheduled Task benigna temporal	OK	0
FP-004	Registro benigno no persistente	OK	0
FP-005	Consulta de servicios	OK	0
FP-006	Discovery legítimo de seguridad	OK	0
FP-007	ZIP benigno local	OK	0
FP-008	Borrado benigno controlado	OK	0
FP-009	HTTP benigno local	OK	0
FP-010	Operaciones normales masivas	OK	0

Tabla 50: Resultados definitivos del runner de falsos positivos

5.3.3.1. TEC-001

[MEDIUM / HIGH] CU-001 TEC-001 - TEC-001 PowerShell suspicious execution

PowerShell script block contains execution bypass, decoder or IEX-style behavior.
| PowerShell 4104 | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-21T13:31:34Z	LT29.ait.us.es / C.364907172793f8c7	CU-001 / TEC-001

MITRE	Profile	Source
T1059.001	P3	Microsoft-Windows- PowerShell/Operational / EventID 4104

Artifact
Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC001_PowerShell_4104

IOA
powershell -ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

IOC
powershell -ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

Action
Review parent process and whether this was expected administrative automation.
Velociraptor • 21/06/2026 15:31

[Medium / Medium] CU-001 TEC-001 - PowerShell suspicious execution - basic profile v2

powershell.exe/pwsh.exe with suspicious execution flags or command patterns |
Sysmon ID 1 + PowerShell image + suspicious flags; excludes router JSONL self-
events | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:30:24Z	LT29.ait.us.es / C.364907172793f8c7	CU-001 / TEC-001

MITRE	Profile	Source
T1059.001	P4	Microsoft-Windows- Sysmon/Operational / EventID 1

Artifact
Custom.TFM.HIDS.P4.Low.Basic_v2/CU001_Source_TEC001_PowerShell_Basic

IOA
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -
ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

IOC
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -
ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

Action
Review command line, parent process and whether this is expected
administration.
Velociraptor • 21/06/2026 15:30

[HIGH / HIGH] CU-001 TEC-001 - TEC-001 forensic PowerShell 4104

HTTP tool + file-upload + ZIP/upload in live 4104; POST enriches but is not a blocker | EvidenceRole=Detection | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-21T13:32:28Z	LT29.ait.us.es / C.364907172793f8c7	CU-001 / TEC-001
MITRE	Profile	Source
T1059.001	P2_EVENT	watch_evtx PowerShell Operational / EventID 4104

Artifact

Custom.TFM.HIDS.P2.High.Forensic.Event_v1/Forensic_EVENT_PowerShell4104_Strong_By_Technique

IOA

powershell -ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

IOC

powershell -ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

Action

Review forensic fields, ScriptBlockId, EventDataJSON and SystemJSON before escalation.

Velociraptor • 21/06/2026 15:32

5.3.3.2. TEC-002

[MEDIUM / MEDIUM] CU-002 TEC-002 - TEC-002 forensic Sysmon ID 1

Sysmon ID 1 command-line behavior | EvidenceRole=Detection | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:09:05Z	LT29.ait.us.es / C.364907172793f8c7	CU-002 / TEC-002
MITRE	Profile	Source
T1059.003	P2_EVENT	watch_evtx Sysmon Operational / EventID 1

Artifact

Custom.TFM.HIDS.P2.High.Forensic.Event_v1/Forensic_EVENT_Sysmon_ID1_Process_By_Technique

IOA

```
C:\Windows\system32\cmd.exe /d /s /c ""C:\Program Files\Adobe\Acrobat  
DC\Acrobat\Browser\WCChromeExtn\WCChromeNativeMessagingHost.exe"  
chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/ --parent-window=0" <  
\.\pipe\chrome.nativeMessaging.in.ad6f06682d70748a >  
\.\pipe\chrome.nativeMessaging.out.ad6f06682d70748a
```

IOC

```
C:\Windows\system32\cmd.exe /d /s /c ""C:\Program Files\Adobe\Acrobat  
DC\Acrobat\Browser\WCChromeExtn\WCChromeNativeMessagingHost.exe"  
chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/ --parent-window=0" <  
\.\pipe\chrome.nativeMessaging.in.ad6f06682d70748a >  
\.\pipe\chrome.nativeMessaging.out.ad6f06682d70748a
```

Action

Review process tree, parent command line and related 4104 evidence.

Velociraptor • 21/06/2026 15:09

5.3.3.3. TEC-003

[HIGH / MEDIUM] CU-003 TEC-003 - TEC-003 Scheduled Task operation

schtasks.exe created, ran, changed or deleted a scheduled task. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-19T11:47:23Z	LT29.ait.us.es / C.364907172793f8c7	CU-003 / TEC-003
MITRE	Profile	Source
T1053.005	P3	Microsoft-Windows-Sysmon/Operational / EventID 1

Artifact
Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC003_ScheduledTask_Sysmon

IOA
schtasks.exe /delete /tn "TFM-TEC-003" /f

IOC
schtasks.exe /delete /tn "TFM-TEC-003" /f

Action
Inspect task action, author, trigger and cleanup behavior.

Velociraptor • 19/06/2026 13:47

5.3.3.4. TEC-004

[HIGH / MEDIUM] CU-004 TEC-004 - TEC-004 high-confidence Run Key persistence

P1 critical process creation signal with composed high-fidelity criteria. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-24T12:37:48Z	LT29.ait.us.es / C.364907172793f8c7	CU-004 / TEC-004
MITRE	Profile	Source
T1547.001	P1_CRITICAL	Microsoft-Windows- Sysmon/Operational / EventID 1

Artifact

Custom.TFM.HIDS.P1.Critical.Priority.Event_v1/P1_EVENT_Sysmon_ID1_Process_Basic

IOA

```
reg.exe add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v  
"TFM_T1547_001_test" /t REG_SZ /d "C:\Windows\System32\calc.exe" /f
```

IOC

```
reg.exe add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v  
"TFM_T1547_001_test" /t REG_SZ /d "C:\Windows\System32\calc.exe" /f
```

Action

Review command line, parent process and corresponding forensic P2/P3 evidence.

Velociraptor • hoy a las 14:37

5.3.3.5. TEC-005

[HIGH / HIGH] CU-005 TEC-005 - TEC-005 suspicious service command

P1 critical process creation signal with composed high-fidelity criteria. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-24T12:37:51Z	LT29.ait.us.es / C.364907172793f8c7	CU-005 / TEC-005
MITRE	Profile	Source
T1569.002	P1_CRITICAL	Microsoft-Windows- Sysmon/Operational / EventID 1

Artifact

Custom.TFM.HIDS.P1.Critical.Priority.Event_v1/P1_EVENT_Sysmon_ID1_Process_Basic

IOA

```
sc.exe create "TFM TEC005 Test Service" binPath=
"C:\Windows\System32\cmd.exe /c echo TEC005_OK ^">
<RUTA_EVIDENCIAS>
```

IOC

```
sc.exe create "TFM TEC005 Test Service" binPath=
"C:\Windows\System32\cmd.exe /c echo TEC005_OK ^">
<RUTA_EVIDENCIAS>
```

Action

Review command line, parent process and corresponding forensic P2/P3 evidence.

Velociraptor • hoy a las 14:37

5.3.3.6. TEC-006

[MEDIUM / MEDIUM] CU-006 TEC-006 - TEC-006 Security Software

Discovery process context

Process creation command line queries security product or Defender inventory. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:15:59Z	LT29.ait.us.es / C.364907172793f8c7	CU-006 / TEC-006
MITRE	Profile	Source
T1518.001 context	P3	Microsoft-Windows- Sysmon/Operational / EventID 1

Artifact

Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC006_SecuritySoftwareDiscovery_Sysmon

IOA

"C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26050.15-0\MpCmdRun.exe" -IdleTask -TaskName WdCacheMaintenance

IOC

"C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26050.15-0\MpCmdRun.exe" -IdleTask -TaskName WdCacheMaintenance

Action

Correlate with PowerShell 4104 if available.

Velociraptor • 21/06/2026 15:16

[MEDIUM / MEDIUM] CU-006 TEC-006 - TEC-006 forensic Sysmon ID 1

Sysmon ID 1 command-line behavior | EvidenceRole=Detection | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:15:17Z	LT29.ait.us.es / C.364907172793f8c7	CU-006 / TEC-006
MITRE	Profile	Source
T1518.001	P2_EVENT	watch_evtx Sysmon Operational / EventID 1

Artifact

Custom.TFM.HIDS.P2.High.Forensic.Event_v1/Forensic_EVENT_Sysmon_ID1_Process_By_Technique

IOA

"C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26050.15-O\MpCmdRun.exe" -IdleTask -TaskName WdCleanup

IOC

"C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26050.15-O\MpCmdRun.exe" -IdleTask -TaskName WdCleanup

Action

Review process tree, parent command line and related 4104 evidence.

Velociraptor • 21/06/2026 15:15

5.3.3.7. TEC-007

[HIGH / HIGH] CU-007 TEC-007 - TEC-007 Data Encrypted for Impact

PowerShell script block contains AES/CryptoStream/CreateEncryptor or encrypted-extension behavior. | PowerShell 4104 | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-19T11:47:25Z	LT29.ait.us.es / C.364907172793f8c7	CU-007 / TEC-007
MITRE	Profile	Source
T1486	P3	Microsoft-Windows- PowerShell/Operational / EventID 4104

Artifact

Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC007_DataEncryptedForImpact_4104

IOA

```
{  
$.FullName -and  
($.FullName -like "$WorkDir") -and  
(Test-Path -LiteralPath $.FullName) -and  
($.FullName -notlike ".aes")  
}
```

IOC

```
{  
$.FullName -and  
($.FullName -like "$WorkDir") -and  
(Test-Path -LiteralPath $.FullName) -and  
($.FullName -notlike ".aes")  
}
```

Action

Confirm scope is limited to dummy/lab files and review generated encrypted artifacts.

Velociraptor • 19/06/2026 13:47

[CRITICAL / HIGH] CU-007 TEC-007 - TEC-007 critical encryption behavior

P1 critical 4104 signal requiring a composed high-fidelity condition. | PowerShell 4104 | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-19T11:47:42Z	LT29.ait.us.es / C.364907172793f8c7	CU-007 / TEC-007
MITRE	Profile	Source
T1486	P1_CRITICAL	Microsoft-Windows- PowerShell/Operational / EventID 4104

Artifact

Custom.TFM.HIDS.P1.Critical.Priority.Event_v1/P1_EVENT_PowerShell4104_Basic

IOA

```
param(
[string]$WorkDir = "",
[string]$ListFile = "",
[string]$PasswordFile = "",
[int]$MaxFiles = 25
)

Set-StrictMode -Version Latest
$ErrorActionPreference = "Stop"

$BasePath = $env:TFM_BASEPATH
if ([string]::IsNullOrEmpty($BasePath)) {
$BasePath = <RAÍZ_TFM>\Pruebas
}
$env:TFM_BASEPATH = $BasePath

if ([string]::IsNullOrEmpty($WorkDir)) {
<RUTA_EVIDENCIAS>
}
if ([string]::IsNullOrEmpty($ListFile)) {
$ListFile = Join-Path $PSScriptRoot "lista_archivos.csv"
}
if ([string]::IsNullOrEmpty($PasswordFile)) {
$PasswordFile = Join-Path $PSScriptRoot "password.txt"
}
```

5.3.3.8. TEC-008

[MEDIUM / MEDIUM] CU-008 TEC-008 - TEC-008 forensic Sysmon ID 1

Sysmon ID 1 command-line behavior | EvidenceRole=Context | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-19T11:49:03Z	LT29.ait.us.es / C.364907172793f8c7	CU-008 / TEC-008
MITRE	Profile	Source
T1485 context	P2_EVENT	watch_evtx Sysmon Operational / EventID 1

Artifact

Custom.TFM.HIDS.P2.High.Forensic.Event_v1/Forensic_EVENT_Sysmon_ID1_Process_By_Technique

IOA

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "$target =  
<RUTA_EVIDENCIAS>  
Write-Host 'TEC008OK'; Get-ChildItem $target -Recurse -File | ForEach-Object {  
Remove-Item $_.FullName -Force }; Start-Sleep 3"
```

IOC

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "$target =  
<RUTA_EVIDENCIAS>  
Write-Host 'TEC008OK'; Get-ChildItem $target -Recurse -File | ForEach-Object {  
Remove-Item $_.FullName -Force }; Start-Sleep 3"
```

Action

Review process tree, parent command line and related 4104 evidence.

Velociraptor • 19/06/2026 13:49

[HIGH / HIGH] CU-008 TEC-008 - TEC-008 Data Destruction

PowerShell script block combines enumeration and forced or recursive deletion behavior. | PowerShell 4104 | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-19T11:48:01Z	LT29.ait.us.es / C.364907172793f8c7	CU-008 / TEC-008
MITRE	Profile	Source
T1485	P3	Microsoft-Windows- PowerShell/Operational / EventID 4104

Artifact

Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC008_DataDestruction_4104

IOA

\$target =

<RUTA_EVIDENCIAS>

Write-Host 'TEC008OK'; Get-ChildItem \$target -Recurse -File | ForEach-Object {
Remove-Item \$.FullName -Force }; Start-Sleep 3

IOC

\$target =

<RUTA_EVIDENCIAS>

Write-Host 'TEC008OK'; Get-ChildItem \$target -Recurse -File | ForEach-Object {
Remove-Item \$.FullName -Force }; Start-Sleep 3

Action

Review deletion scope and validate whether affected files are controlled test data.

Velociraptor • 19/06/2026 13:48

[CRITICAL / HIGH] CU-008 TEC-008 - TEC-008 critical destructive deletion behavior

P1 critical 4104 signal requiring a composed high-fidelity condition. | PowerShell 4104 | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-19T11:47:42Z	LT29.ait.us.es / C.364907172793f8c7	CU-008 / TEC-008
MITRE	Profile	Source
T1485	P1_CRITICAL	Microsoft-Windows- PowerShell/Operational / EventID 4104

Artifact

Custom.TFM.HIDS.P1.Critical.Priority.Event_v1/P1_EVENT_PowerShell4104_Basic

IOA

\$target =

<RUTA_EVIDENCIAS>

Write-Host 'TEC008OK'; Get-ChildItem \$target -Recurse -File | ForEach-Object {
Remove-Item \$.FullName -Force }; Start-Sleep 3

IOC

\$target =

<RUTA_EVIDENCIAS>

Write-Host 'TEC008OK'; Get-ChildItem \$target -Recurse -File | ForEach-Object {
Remove-Item \$.FullName -Force }; Start-Sleep 3

Action

Treat as priority alert. Correlate with P2/P3/P4 evidence and campaign summary.

Velociraptor • 19/06/2026 13:48

5.3.3.9. TEC-009

[HIGH / MEDIUM] CU-009 TEC-009 - TEC-009 HTTP upload process context

PowerShell process command line contains file execution/upload parameters or HTTP upload indicators. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:31:36Z	LT29.ait.us.es / C.364907172793f8c7	CU-009 / TEC-009
MITRE	Profile	Source
T1074.001 / T1560.001 / T1048.003 context	P3	Microsoft-Windows- Sysmon/Operational / EventID 1

Artifact

Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC009_HTTP_Upload_Sysmon_Process_Context

IOA

"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -
ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

IOC

"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -
ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

Action

Use as context; the primary alert is TEC009_HTTP_ZIP_Upload_4104.

Velociraptor • 21/06/2026 15:31

[HIGH / HIGH] CU-009 TEC-009 - TEC-009 controlled upload process invocation

P1 critical process creation signal with composed high-fidelity criteria. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-24T12:37:45Z	LT29.ait.us.es / C.364907172793f8c7	CU-009 / TEC-009
MITRE	Profile	Source
T1074.001 / T1560.001 / T1048.003	P1_CRITICAL	Microsoft-Windows- Sysmon/Operational / EventID 1

Artifact

Custom.TFM.HIDS.P1.Critical.Priority.Event_v1/P1_EVENT_Sysmon_ID1_Process_Basic

IOA

powershell.exe -NoProfile -ExecutionPolicy Bypass -File

<RUTA_ARTIFACTS>\exfiltracion_v3.ps1
<HOST_LAB>

IOC

powershell.exe -NoProfile -ExecutionPolicy Bypass -File

<RUTA_ARTIFACTS>\exfiltracion_v3.ps1
<HOST_LAB>

Action

Review command line, parent process and corresponding forensic P2/P3 evidence.

Velociraptor • hoy a las 14:37

[HIGH / MEDIUM] CU-009 TEC-009 - TEC-009 forensic Sysmon ID 1		
Sysmon ID 1 command-line behavior EvidenceRole=Context EventID=1		
Timestamp	Host / Client	CU / TEC
2026-06-21T13:31:00Z	LT29.ait.us.es / C.364907172793f8c7	CU-009 / TEC-009
MITRE	Profile	Source
T1074.001 / T1560.001 / T1048.003 context	P2_EVENT	watch_evtx Sysmon Operational / EventID 1
Artifact		
Custom.TFM.HIDS.P2.High.Forensic.Event_v1/Forensic_EVENT_Sysmon_ID1_Process_By_Technique		
IOA		
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" - ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri <HOST_LABEL>		
IOC		
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" - ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri <HOST_LABEL>		
Action		
Review process tree, parent command line and related 4104 evidence.		
Velociraptor • 21/06/2026 15:31		

5.4. Configuración y uso de artifacts

La configuración de artifacts se organizó en dos fases. Primero se revisaron artifacts de fábrica y públicos de Velociraptor para establecer una línea base de visibilidad. Después se desarrollaron artifacts personalizados para mejorar la clasificación, priorización y salida externa de eventos.

El criterio aplicado fue el siguiente:

- los artifacts de fábrica se emplean como línea base de observación;
- los artifacts personalizados se emplean para detección y clasificación;
- el router SERVER_EVENT se emplea para salida externa;
- los resultados concretos de detección se analizan en el Capítulo VI.

Esta distinción evita confundir telemetría bruta con detección. Un artifact puede observar procesos, ficheros o eventos sin que ello implique una detección MITRE válida.

5.4.1. Artifacts de fábrica utilizados

Los artifacts de fábrica y públicos se utilizaron para comprobar qué información podía obtenerse sin desarrollar lógica específica para el TFM. Su función principal fue validar visibilidad sobre procesos, logs, PowerShell, Sysmon, servicios, registro y conexiones de red.

Artifact	Tipo	Uso en el TFM
Windows.Events.ProcessCreation	CLIENT_EVENT	Observación de creación de procesos y líneas de comando.
Windows.EventLogs.PowerShell	Consulta sobre logs	Revisión de eventos PowerShell, especialmente Event ID 4104.
Windows.Events.ServiceCreation	CLIENT_EVENT	Observación de creación de servicios Windows.
Windows.Sysmon.*	Consulta / eventos Sysmon	Consulta de eventos Sysmon ID 1, 3, 11, 12, 13, 14 y 26.
Generic.Events.TrackNetworkConnections	CLIENT_EVENT	Observación de conexiones de red; sin atribución fiable de proceso en la campaña definitiva.
Windows.System.Pslist	CLIENT	Inventario puntual de procesos activos.
Windows.System.Services	CLIENT	Inventario puntual de servicios instalados y estado de servicios.
Windows.Registry.*	CLIENT	Consulta de claves y valores del registro.
Windows.Hayabusa.Monitoring	CLIENT_EVENT	Evaluación de reglas públicas sobre eventos Windows.
Windows.ETW.Monitoring	CLIENT_EVENT	Monitorización basada en ETW cuando la fuente resulta aplicable.

Tabla 51: Artifacts de fábrica y públicos considerados durante la fase experimental

Estos artifacts permitieron comprobar que Velociraptor dispone de visibilidad suficiente sobre múltiples fuentes del endpoint. Sin embargo, en varios casos la información obtenida requería interpretación adicional. Por ejemplo, la ejecución de powershell.exe, cmd.exe, reg.exe, schtasks.exe o sc.exe puede ser legítima, por lo que no debe tratarse automáticamente como detección.

Por tanto, los artifacts de fábrica se consideran una base útil de visibilidad, pero no siempre ofrecen detección operativa completa frente a las técnicas evaluadas.

5.4.2. Artifacts personalizados

A partir de las brechas detectadas en los artifacts públicos, se desarrolló una arquitectura personalizada basada en perfiles de severidad P1, P2, P3 y P4. Su objetivo fue transformar eventos host-based en señales más estructuradas, priorizadas y trazables.

La lógica personalizada se organizó por niveles:

Perfil	Artifact	Tipo	Filas CLIENT_EVENT	Función
P1	Custom.TFM.HIDS.P1.Critical.Priority.Event_v1	CLIENT_EVENT	19	Señales críticas selectivas
P2	Custom.TFM.HIDS.P2.High.Forensic.Event_v1	CLIENT_EVENT	118	Detección, contexto y evidencia forense; no constituye una unidad homogénea
P3	Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1	CLIENT_EVENT	109	Detección de comportamiento
P4	Custom.TFM.HIDS.P4.Low.Basic_v2	CLIENT_EVENT	133	Visibilidad básica
Router	Custom.TFM.HIDS.Router.JSONL.Discord.SOC_v3	SERVER_EVENT	No aplica	Enruta, persiste o notifica; no detecta

Tabla 52: Artifacts custom por perfil y función operativa

Los artifacts personalizados se diseñaron para:

- asociar eventos a técnicas TEC-001 a TEC-009;
- normalizar campos relevantes;
- asignar perfil de severidad;
- reducir dependencia de indicadores de laboratorio;
- evitar reglas basadas únicamente en nombres de scripts o rutas TFM;
- facilitar la salida estructurada hacia JSONL y Discord;
- diferenciar eventos de baja señal frente a comportamientos de impacto.

La arquitectura personalizada no sustituye la telemetría original. Su función es filtrar, enriquecer y clasificar eventos procedentes de Windows, PowerShell, Sysmon y Velociraptor.

5.4.3. Limitaciones encontradas durante la configuración

Durante la configuración de artifacts se identificaron varias limitaciones relevantes.

En primer lugar, muchas actividades administrativas legítimas generan la misma telemetría base que una técnica ofensiva. La ejecución de PowerShell, CMD, reg.exe, sc.exe o schtasks.exe no es

maliciosa por sí misma. Por ello, la detección requiere contexto adicional, como parámetros usados, ventana temporal, técnica ejecutada, fuente del evento y relación con otros eventos.

En segundo lugar, algunas técnicas generan demasiado volumen de eventos si se alertan de forma directa. TEC-008 es el caso más claro: un borrado masivo puede producir múltiples eventos de eliminación. Por este motivo, los eventos de borrado deben tratarse con agregación o como evidencia forense, no necesariamente como alerta individual por fichero.

En tercer lugar, TEC-009 exige separar dos evidencias. Los artifacts custom CLIENT_EVENT aportan detección host-based sobre staging, creación del ZIP, PowerShell y actividad de proceso. Por separado, el summary del runner independiente documenta ReceiverReachable=True, HTTPStatus=200, UploadSucceeded=True, SHA-256 local y bytes transferidos. El HTTP 200 es ground truth de ejecución, no una alerta CLIENT_EVENT ni una detección del artifact TrackNetwork; este último observó la conexión sin atribución fiable de proceso.

En cuarto lugar, la salida a Discord valida el flujo externo de alertado, pero no demuestra por sí sola la calidad de la detección. La calidad de la detección debe evaluarse en función de la lógica del artifact, la técnica asociada, la evidencia disponible y el ruido generado.

Por último, la configuración personalizada requiere ajuste iterativo. Velociraptor aporta flexibilidad y transparencia, pero esa misma flexibilidad obliga a diseñar, probar y refinar artifacts para reducir falsos positivos y evitar sobreclasificación.

Capítulo VI: Resultados de detección

Este capítulo expone los resultados finales obtenidos con Velociraptor en la campaña real del 24/06 y en la campaña de falsos positivos del 26/06. La unidad primaria de detección se define como una fila CLIENT_EVENT emitida por un artifact custom dentro de la ventana canónica de ejecución. Las salidas JSONL y Discord se conservan como salidas externas trazables derivadas del flujo de monitorización; su presencia permite comprobar persistencia o notificación posterior, pero la unidad primaria de detección en Velociraptor sigue siendo la fila CLIENT_EVENT emitida por el artifact correspondiente.

6.1. Resultados por técnica MITRE

Las nueve técnicas planificadas finalizaron en estado OK y generaron visibilidad de sistema y detección Velociraptor mediante CLIENT_EVENT. La cobertura funcional es, por tanto, 9/9 sobre el subconjunto experimental TEC-001 a TEC-009. Esta cifra no debe extrapolarse a toda la matriz MITRE ATT&CK Enterprise ni a cualquier artifact, solo se consideran válidos para la configuración concreta utilizada.

```
PowerShell - Runner global TEC

<RAÍZ_TFM>\03_RUNNERS> .\TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1

<HOST_LAB>
-KeepExfilArtifacts $true -KeepArtifacts $true

<RUTA_EVIDENCIAS>
Start: 2026-06-24T14:50:22+02:00
End: 2026-06-24T14:55:53+02:00
DurationSeconds: 331.66

<HOST_LAB>
GlobalStatus: OK
Techniques OK/WARN/FAIL: 9/0/0

TEC-001 | PowerShell / T1059.001 | OK | Sysmon=4 | PS4104=3
TEC-002 | Windows Command Shell / T1059.003 | OK | Sysmon=4 | PS4104=1
TEC-003 | Scheduled Task / T1053.005 | OK | Sysmon=6 | PS4104=0
TEC-004 | Registry Run Key / T1547.001 | OK | Sysmon=5 | PS4104=0
TEC-005 | Service Execution / T1569.002 | OK | Sysmon=6 | PS4104=0
TEC-006 | Security Software Discovery / T1518.001 | OK | Sysmon=2 | PS4104=2
TEC-007 | Data Encrypted for Impact / T1486 | OK | Sysmon=9 | PS4104=4
TEC-008 | Data Destruction / T1485 | OK | Sysmon=4 | PS4104=3
TEC-009 | Staging / ZIP / Controlled HTTP Upload | OK | Sysmon=7 | PS4104=1
```

Figura 28: Resumen del runner global de técnicas

TEC	MITRE	Técnica evaluada	Sysmon relevantes	Estado
TEC-001	T1059.001	PowerShell	4	OK
TEC-002	T1059.003	Windows Command Shell	4	OK
TEC-003	T1053.005	Scheduled Task	6	OK

TEC	MITRE	Técnica evaluada	Sysmon relevantes	Estado
TEC-004	T1547.001	Registry Run Key	5	OK
TEC-005	T1569.002	Service Execution	6	OK
TEC-006	T1518.001	Security Software Discovery	2	OK
TEC-007	T1486	Data Encrypted for Impact	9	OK
TEC-008	T1485	Data Destruction	4	OK
TEC-009	T1074.001 / T1560.001 / T1048.003	Staging, ZIP y upload HTTP controlado	7	OK

Tabla 53: Resultados finales de Velociraptor por técnica MITRE evaluada

La Figura 29 representa el volumen por perfil. Este volumen no equivale a cobertura: la cobertura se calcula por técnicas únicas y es 9/9

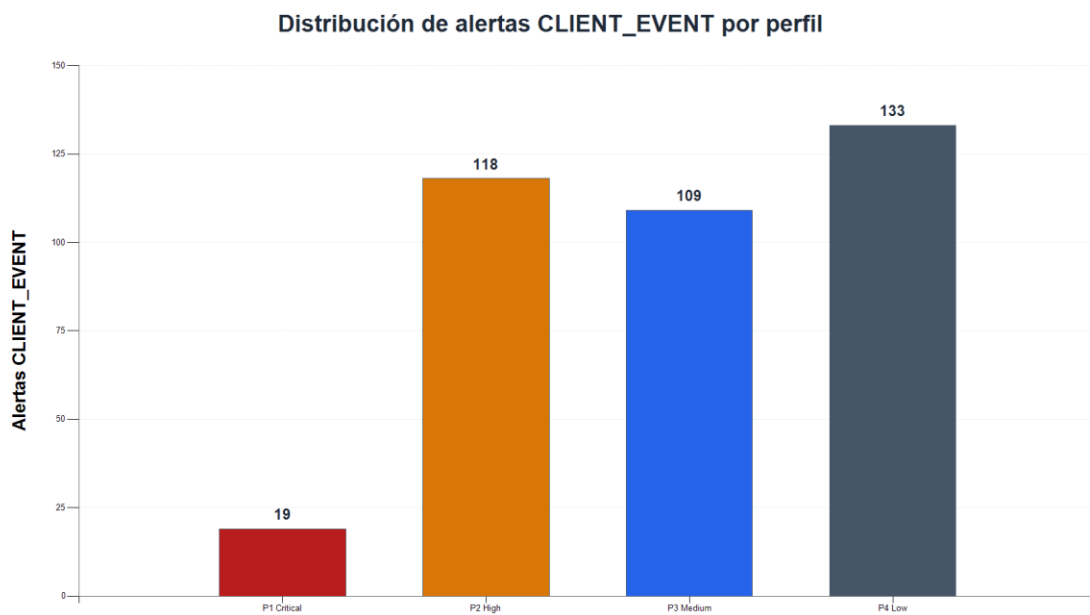


Figura 29: Distribución de las 379 filas CLIENT_EVENT emitidas/exportadas, incluidas emisiones repetidas por perfil custom

6.2. Cobertura de detección

La campaña final incluyó 379 filas CLIENT_EVENT reales dentro de la ventana de tiempo de ejecución del script e ingesta de estos datos. La distribución por perfiles de artifacts fue P1=19, P2=118, P3=109 y P4=133. En este contexto, P1/P2/P3/P4 no son documentos de prioridad ni herramientas distintas, sino perfiles operativos de severidad y visibilidad: P4 aporta visibilidad básica, P3 comportamiento, P2 evidencia de alta relevancia y P1 señales críticas (García-Amorena Reina, 2026e).

```

●●● Velociraptor artifacts

Arquitectura de artifacts Velociraptor
-----
CLIENT_EVENT P1_CRITICAL: comportamientos criticos; TEC-004, TEC-005, TEC-007, TEC-008, TEC-009
CLIENT_EVENT P2_HIGH: evidencia forense; TEC-001 a TEC-009
CLIENT_EVENT P3_MEDIUM: comportamiento relevante; TEC-001 a TEC-009
CLIENT_EVENT P4_LOW: visibilidad basica; TEC-001 a TEC-005
SERVER_EVENT Router SOC_v3: normalizacion y salida JSONL/Discord; no detecta por si mismo

Flujo: CLIENT_EVENT -> SERVER_EVENT -> JSONL / Webhook -> Discord

```

Figura 30: Arquitectura de artifacts custom y router

Perfi l	Artifact	Tipo	Filas emitidas/exportada s	Lectura
P1	Custom.TFM.HIDS.P1.Critical.Priority.Event_v1	CLIENT_EVENT	19	Señales críticas selectivas
P2	Custom.TFM.HIDS.P2.High.Forensic.Event_v1	CLIENT_EVENT	118	Detección, contexto y evidencia forense; interpretar según la source
P3	Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1	CLIENT_EVENT	109	Comportamiento relevante
P4	Custom.TFM.HIDS.P4.Low.Basic_v2	CLIENT_EVENT	133	Visibilidad básica
Total	P1 + P2 + P3 + P4	CLIENT_EVENT	379	379 filas emitidas/exportadas, incluidas emisiones repetidas; no equivalen a alertas únicas ni a cobertura

Tabla 54: Distribución de filas CLIENT_EVENT emitidas/exportadas por perfil custom

Técnica	Filas CLIENT_EVENT incluidas	Lectura
TEC-001	42	PowerShell observable por proceso y 4104.
TEC-002	214	CMD observable por proceso y línea de comandos.
TEC-003	30	Scheduled task observable por schtasks.exe.
TEC-004	37	Run Key observable por actividad de registro/proceso.
TEC-005	14	Servicio observable, con 7045 como evidencia principal.
TEC-006	18	Discovery observable por PowerShell/WMI/WMIC.

Técnica	Filas CLIENT_EVENT incluidas	Lectura
TEC-007	5	Cifrado controlado observable por 4104 y .aes.
TEC-008	4	Borrado controlado observable; Sysmon 26 refuerza evidencia forense.
TEC-009	15	Staging y archivado observables; el HTTP 200 procede del runner independiente.

Tabla 55: Distribución de filas CLIENT_EVENT por técnica en la campaña real

6.3. Casos sin detección y detección parcial

En la campaña final no se identificó ninguna técnica del subconjunto TEC-001 a TEC-009 sin detección mediante artifacts CLIENT_EVENT. Todas las técnicas generaron algún tipo de visibilidad en Velociraptor, aunque no todas ofrecieron el mismo nivel de evidencia ni la misma facilidad de interpretación.

Las limitaciones detectadas no correspondieron tanto a una ausencia completa de detección, sino a problemas de visibilidad, correlación y calidad de la evidencia. En particular, los eventos generados por perfiles de baja criticidad, como P4, no deben interpretarse como alertas críticas, sino como señales básicas de observación. La clasificación operativa depende de los perfiles superiores y de la correlación con la telemetría del sistema.

El caso más relevante fue TEC-009, correspondiente al staging, archivado ZIP y exfiltración HTTP controlada. Los artifacts custom permitieron detectar señales host de preparación, archivado y ejecución, pero no acreditaron por sí solos la atribución fiable de la conexión a un proceso. Por ello, el resultado se interpreta junto al ground truth del runner, sin convertir la evidencia HTTP externa en una alerta Velociraptor.

Durante las iteraciones se reforzó la recogida de PowerShell 4104 y Sysmon para mejorar la trazabilidad de staging, creación del ZIP y procesos. La transferencia se validó mediante un receptor HTTP controlado y el summary del runner independiente. Esa validación externa se mantiene separada de la detección CLIENT_EVENT y de TrackNetwork, que en la campaña definitiva presentó PID=0 y ProcInfo vacío.

En la campaña final, TEC-009 queda formulada como exfiltración HTTP controlada en laboratorio. ReceiverReachable=True, HTTPStatus=200, UploadSucceeded=True, el hash SHA-256 local y los bytes transferidos proceden del summary del runner independiente. No se atribuyen a una alerta CLIENT_EVENT ni a Generic.Events.TrackNetworkConnections. La detección custom y el

ground truth de ejecución se conservan como capas distintas; la ausencia del receiver_log.jsonl o del ZIP recibido, si aplica, permanece como limitación documental secundaria.

```
PowerShell - TEC-009 evidence

TEC-009 EVIDENCE CHECK
-----
ReceiverPrecheck: Reachable=True | Status=OK
Command: powershell.exe -NoProfile -ExecutionPolicy Bypass -File exfiltracion_v3.ps1
<HOST_LABEL>
ExitCode: 0
HTTPStatus: 200
UploadSucceeded: True
ZIP: tec009_controlled_archive.zip
ZIP copy: tec009_controlled_archive_copy.zip
LocalSHA256: 31e202c49811f354b51162baecb4ed174e4b0f0805c50c9f91c9f7ec5f642a59
ReceiverResponse: {"status":"ok","bytes_received":6245,"sha256":"31e202c49811f354b51162baecb4ed174e4b0f0805c50c9f91c9f7ec5f642a59"}
PowerShell4104 upload evidence: Count=1; QueryStatus=OK

Interpretacion: staging + archivado ZIP + upload HTTP controlado. Limitacion: no describir como exfiltracion completa si no se conserva receiver_log.jsonl o ZIP recibido.
```

Figura 31: Captura textual de la evidencia de upload HTTP controlado en TEC-009

6.4. Evidencias observadas y falsos positivos

La campaña de falsos positivos del 26/06 ejecutó diez acciones benignas y obtuvo 10 OK, 0 WARN, 0 FAIL y 0 SKIPPED. El resumen final indica VelociraptorHits.Total=0 y el fichero vr_hits.csv no contiene hits definitivos.

```
PowerShell - Runner FP

<RAIZ_TFM>\03_RUNNERS> .\TFM_Run_FP_Tests_v1.ps1

TFM FP Runner Summary - Campana definitiva
RunId: TFM_FP_20260626_101948
GlobalStatus: OK
FP planificados: 10
FP ejecutados OK: 10
FP skipped: 0
FP fail: 0
Velociraptor TotalHits: 0

FP-001 PowerShell administrativo legitimo      OK
FP-002 CMD legitimo                             OK
FP-003 Scheduled Task benigno temporal          OK
FP-004 Registro benigno no persistente          OK
FP-005 Consulta de servicios                   OK
FP-006 Discovery legitimo de seguridad          OK
FP-007 ZIP benigno local                        OK
FP-008 Borrado benigno controlado               OK
FP-009 HTTP benigno local                      OK
FP-010 Operaciones normales masivas            OK

Interpretacion: no se observaron falsos positivos en los perfiles P1/P2/P3/P4 durante la campana FP definitiva.
```

Figura 32: Campaña definitiva de falsos positivos

6.5. Calidad de datos y limitaciones

Control	Estado	Decisión metodológica
TEC-001 a TEC-009 presentes	OK	Se usan como resultados finales del subconjunto evaluado.
FP-001 a FP-010 presentes	OK	La muestra FP final queda completa.
VR real separado de FP	WARN	Se filtra por ventana/carpeta y se excluyen filas no definitivas.
FP hits	OK	Total Hits=0; no se inflan falsos positivos.
Fuentes con SHA-256	OK	102 fuentes hasheadas en el paquete de evidencias.

Figura 33: Controles de calidad aplicados a los resultados de detección y FP

6.6. Resultados definitivos de artifacts públicos

La evaluación pública se mantiene separada de la arquitectura custom. La Tabla 56 resume seis campañas CLIENT_EVENT y la Figura 34 distingue su resultado operativo. Un resultado POSITIVO acredita visibilidad útil en las condiciones ensayadas; no implica necesariamente detección específica de una técnica MITRE.

Artifact público	Visibilidad	Detección específica	Resultado	Limitación
Windows.Hayabusa.Monitoring	Sí	3/9: TEC-001, TEC-003 y TEC-005	POSITIVO	CH high: 4 coincidencias en FP; CHM no ejecutada
Windows.Events.ServiceCreation	Sí: System 7045	No por sí solo	POSITIVO	EI 7045 acredita creación de servicio, no malicia
Windows.Events.ProcessCreation	Sí	No por sí solo	POSITIVO	Proceso observado no equivale a técnica detectada
Windows.Sysinternals.SysmonLogForward	Sí: Sysmon	No por sí solo	POSITIVO	Telemetría forense; ID 26 no es alerta individual
Windows.ETW.Monitoring	No acreditada	No evaluable	NO CONCLUYENTE	Sin campaña reproducible completa
Generic.Events.TrackNetworkConnections	Sí: conexión	No por sí solo	POSITIVO	PID=0 y ProcInfo vacío; HTTP 200 del runner independiente

Tabla 56: Resultados definitivos de las campañas con artifacts públicos

Hayabusa CH alcanzó 3/9 técnicas específicas. Sus coincidencias high no se suman a la cobertura custom y la campaña de falsos positivos registró cuatro matches CH concentrados en un caso benigno FP-003. Por ello, el resultado se presenta como capacidad específica acotada y no como cobertura general.

Windows.ETW.Monitoring permanece NO CONCLUYENTE: la ausencia de runner, summary TEC y manifiesto de configuración impide clasificarlo como positivo o negativo. Generic.Events.TrackNetworkConnections observó la conexión al receptor, pero PID=0 y ProcInfo vacío impidieron una atribución fiable de proceso.

Para TEC-009, el HTTP 200, UploadSucceeded=True y los bytes transferidos pertenecen al runner independiente. No se computan como detección CLIENT_EVENT, ni como resultado de TrackNetwork, del Router o de Discord.

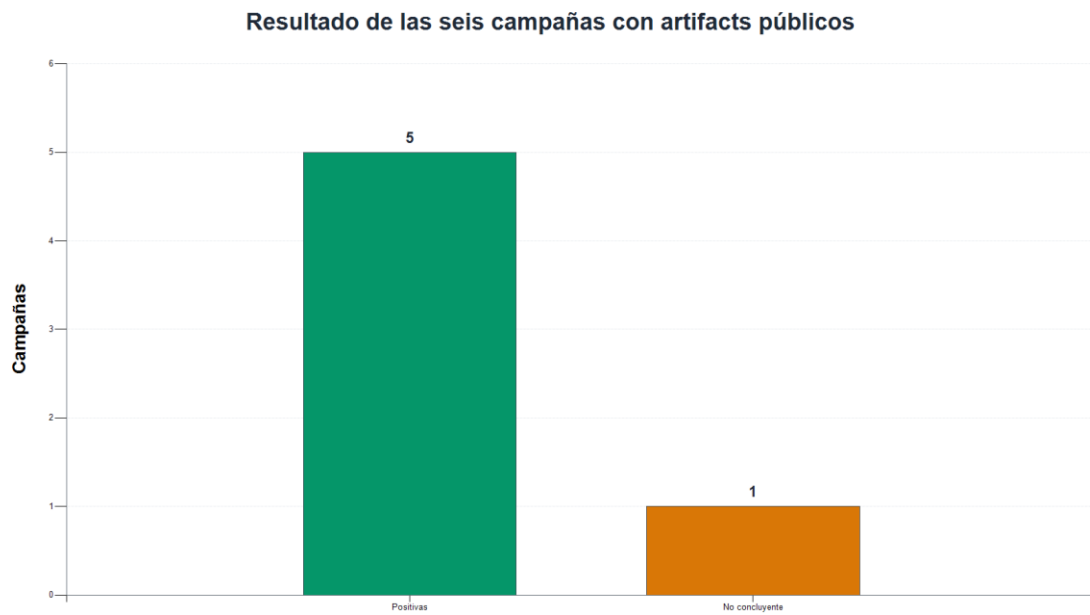


Figura 34: Resultado de las seis campañas con artifacts públicos

Capítulo VII: Evaluación de Velociraptor y benchmark de rendimiento

Este capítulo separa dos planos de evaluación: los resultados de detección y falsos positivos ya consolidados, y el benchmark de rendimiento del cliente Velociraptor. El benchmark se plantea como medición independiente del coste operativo; no mide cobertura, falsos positivos ni calidad de detección. Su objetivo es estimar el consumo observado en el laboratorio bajo tres escenarios reproducibles.

7.1. Metodología de benchmark

La fuente canónica del benchmark es TFM_BENCHMARK_RENDIMIENTO_VELOCIRAPTOR_DEFINITIVO_20260712.xlsx. Sus controles registran 9/9 runs válidos, tres escenarios y tres repeticiones por escenario. El benchmark principal mide exclusivamente el impacto del cliente Velociraptor sobre el endpoint. SERVER_GUI se registra separadamente y se excluye del cálculo principal porque pertenece al servidor y no al cliente evaluado. Los flags reproducibles son IncludeServerGuiInTotal=False y ServerGuiExcludedFromClientMetrics=True. En VR_TEC_RUNNER se observaron 0,95 % de CPU media, 54 % de pico, 57,42 MB de RAM media y 57,97 MB de pico. RunnerStillRunningAtEnd=True se conserva como WARN no bloqueante en las tres repeticiones VR_TEC_RUNNER. Estas métricas no atribuyen CPU o RAM a P1, P2, P3 o P4 individualmente y no miden calidad de detección (García-Amorena Reina, 2026d)

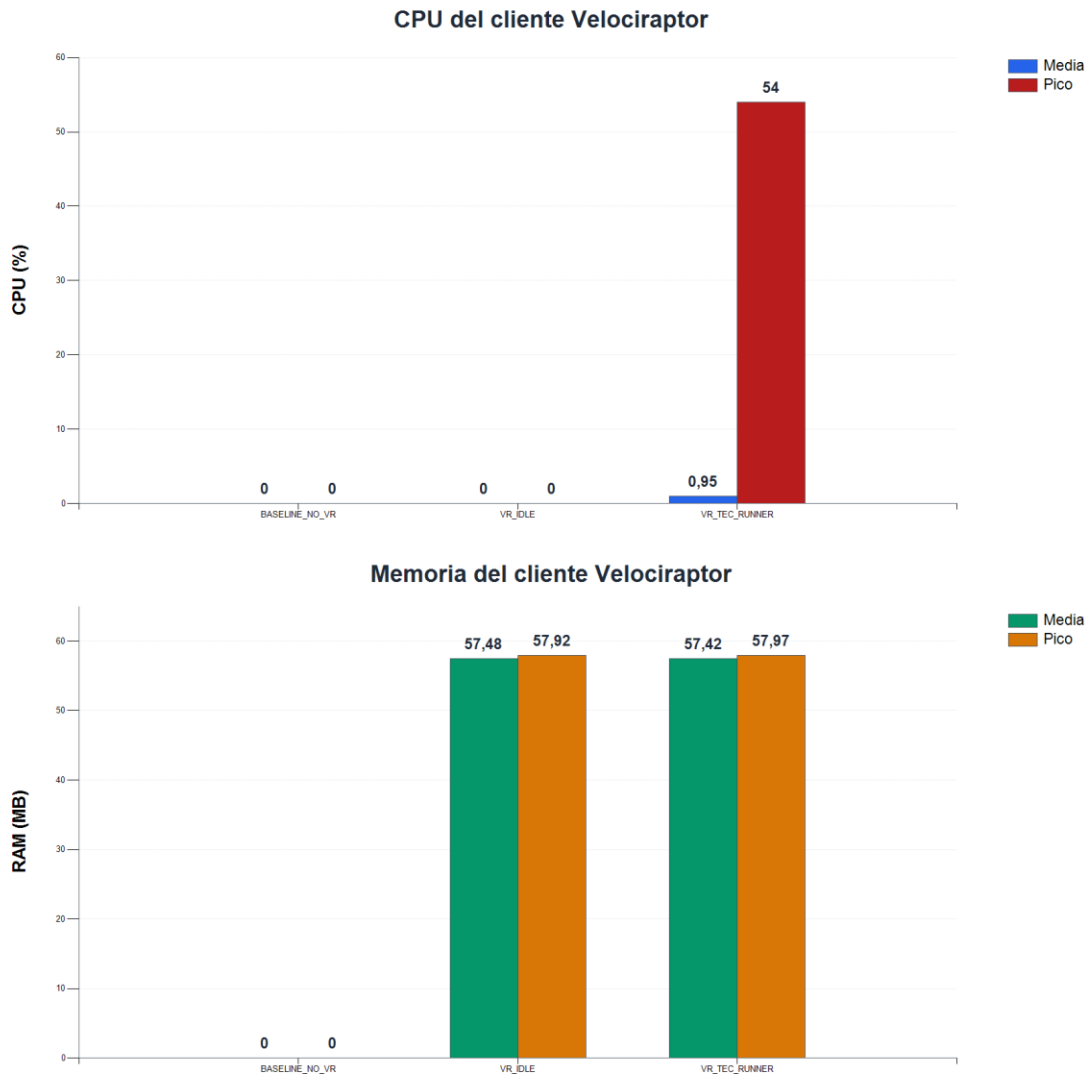


Figura 35: Coste operativo observado del cliente Velociraptor por escenario

7.2. Impacto en CPU

El cliente Velociraptor introduce una carga medible en el endpoint, especialmente visible en picos puntuales de CPU. En el laboratorio evaluado, la CPU media fue 0 % en reposo y 0,95 % durante VR_TEC_RUNNER, con un pico observado del 54 %. Esta lectura no debe extrapolarse como garantía universal para otros endpoints (García-Amorena Reina, 2026d).

7.3. Impacto en memoria

La memoria residente del cliente se mantuvo estable: 57,48 MB en VR_IDLE y 57,42 MB en VR_TEC_RUNNER, con pico de 57,97 MB. En el entorno medido, el consumo medio de memoria fue bajo y no aumentó de forma relevante durante la ejecución de la carga TEC (García-Amorena Reina, 2026d).

7.4. Impacto en disco

El apartado de disco queda documentado en el Excel final y sus hojas de calidad, pero no se usa como conclusión principal de coste. La memoria evita inferir impacto de I/O si no existe una métrica agregada equivalente a CPU/RAM en los datos resumidos.

7.5. Relación detección / coste

Los resultados de benchmark se interpretan solo como coste operativo. Aunque el TFM obtuvo 9/9 detecciones CLIENT_EVENT en el subconjunto evaluado, esa cobertura pertenece al capítulo de resultados y no se deriva del benchmark ni se evalúa con métricas de CPU/RAM (García-Amorena Reina, 2026d; García-Amorena Reina, 2026e).

Control de calidad	Estado	Decisión
SchemaVersion 1.1	OK	Todos los runs válidos usan el esquema esperado.
9/9 runs válidos	OK	No faltan repeticiones.
SERVER_GUI excluido	OK	No suma en métricas de cliente.
notepad.exe runner = 0	OK	Evita contaminación del runner.
RunnerStillRunningAtEnd=True	WARN	No bloqueante en VR_TEC_RUNNER según criterio D042.
Benchmark separado de detección	OK	No se usa para inferir FP ni cobertura.

Figura 36: Controles de calidad del benchmark final

Capítulo VIII: Validación comparativa con Wazuh como HIDS

Este capítulo documenta el despliegue de Wazuh como plataforma HIDS/SIEM de contraste dentro del entorno experimental del TFM. Su finalidad no es sustituir la evaluación principal de Velociraptor, sino aportar una línea externa para comparar visibilidad host-based, generación de alertas, trazabilidad de evidencias, severidad, ruido operativo y facilidad de configuración (Wazuh, s. f.-b; Wazuh, s. f.-c).

La comparación se plantea de forma separada para evitar sesgos metodológicos y no trata Wazuh y Velociraptor como unidades equivalentes directas. Wazuh trabaja con logs, reglas, archives y alerts; Velociraptor trabaja con artifacts, VQL, CLIENT_EVENT y evidencia host-based. La coexistencia simultánea Velociraptor + Wazuh se reserva únicamente para analizar ruido operativo (Velociraptor, s. f.-a; Wazuh, s. f.-a; Wazuh, s. f.-b).

8.1. Objetivo de la comparación

El objetivo del despliegue fue disponer de una plataforma independiente con la que contrastar modelos de visibilidad y alerta. Wazuh se utilizó como herramienta HIDS/SIEM de referencia operativa, no como sustituto de Velociraptor, por su capacidad para recoger eventos de Windows, procesar telemetría de Sysmon, aplicar reglas de correlación y mostrar alertas en un dashboard centralizado.

La comparación se orienta a los siguientes criterios: visibilidad sobre eventos crudos, capacidad de detección, evidencias disponibles, trazabilidad, severidad, latencia aproximada, volumen de ruido, impacto operativo y esfuerzo de despliegue/configuración.

Criterio	Interpretación en el capítulo
Visibilidad	Existencia de eventos crudos en archives.json o en la GUI de Wazuh.
Detección	Generación de alertas en alerts.json o en el dashboard a partir de reglas base o reglas custom.
Trazabilidad	Capacidad de vincular alerta, agente, fuente de telemetría, Event ID, regla y técnica MITRE.
Severidad	Nivel de alerta de Wazuh y mapeo aproximado con perfiles P1/P2/P3/P4 de Velociraptor.
Ruido	Eventos o alertas no atribuibles a la técnica evaluada, incluyendo actividad operacional legítima.

Criterio	Interpretación en el capítulo
Velocidad de implementación	Velocidad en la que se ha implementado y desplegado la herramienta.
Comparación con VR	Comparación funcional basada en modelo operativo, visibilidad, reglas/artifacts, alertas y trazabilidad; no equivalencia numérica directa.

Tabla 57: Criterios empleados para evaluar Wazuh como solución de comparación

8.2. Arquitectura del entorno Wazuh

La arquitectura se desplegó en un entorno Ubuntu virtualizado sobre Oracle VirtualBox. Se utilizó una máquina virtual Ubuntu Server dedicada para los componentes centrales de Wazuh y la misma máquina Windows de laboratorio utilizada en las pruebas HIDS como endpoint monitorizado. Esta decisión permitió mantener constante el sistema observado y reutilizar las mismas técnicas TEC empleadas en el resto del TFM.

Elemento	Valor / descripción
Servidor Wazuh	Ubuntu Server 24.04.3 LTS en VirtualBox.
Modo de instalación	All-in-one: Wazuh Manager, Wazuh Indexer y Wazuh Dashboard en la misma VM.
IP servidor	<HOST_LAB>.
Recursos VM Ubuntu	4 vCPU, 8 GB RAM y disco VDI dinámico de 60 GB.
Red	Adaptador puente para comunicación directa con el endpoint y el host.
Endpoint monitorizado	Windows LT29 / LT29.ait.us.es.
IP endpoint	<HOST_LAB>.
Wazuh Agent ID	001.
Estado agente	Active.

Tabla 58: Parámetros principales del entorno Wazuh desplegado

```
admin@admin:~$ sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: admin (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: LT29, IP: any, Active

List of agentless devices:
```

Figura 37: Validación del agente LT29 como agente activo desde agent_control

8.3. Instalación del servidor Wazuh

La instalación se realizó sobre Ubuntu Server 24.04.3 LTS. Tras instalar el sistema operativo, se habilitó OpenSSH para administrar la máquina desde el host Windows mediante PowerShell. Posteriormente se descargó el script oficial de instalación de Wazuh y se ejecutó el modo all-in-one.

```
$ sudo apt update && sudo apt upgrade -y

$ sudo apt install openssh-server -y

$ sudo systemctl enable --now ssh

$ sudo apt install curl nano net-tools -y

$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh

$ sudo bash ./wazuh-install.sh -a

$ sudo systemctl status wazuh-manager --no-pager

# Active

$ sudo systemctl status wazuh-indexer --no-pager

# Active

$ sudo systemctl status wazuh-dashboard --no-pager

# Active
```

Código 19: Instalación base de dependencias y despliegue all-in-one de Wazuh

El dashboard quedó disponible desde el host físico en la URL https://<HOST_LAB>. Se utilizó el usuario admin generado durante la instalación. La contraseña generada automáticamente por el instalador no se incorpora en la memoria ni en las capturas por criterio de seguridad.

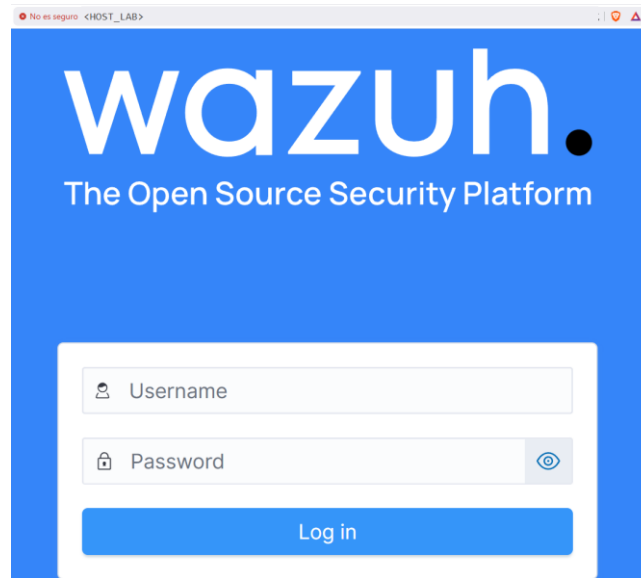


Figura 38: Panel de acceso al dashboard de Wazuh

8.4. Instalación y validación del agente Windows

Antes de instalar el agente se validó la conectividad desde Windows hacia el servidor Wazuh. Se comprobaron los puertos 443, 1514 y 1515, asociados respectivamente al acceso web, la comunicación del agente y el enrolamiento/registro.

```
$ Test-NetConnection <HOST_LAB> -Port 443  
$ Test-NetConnection <HOST_LAB> -Port 1514  
$ Test-NetConnection <HOST_LAB> -Port 1515
```

Código 20: Validación de conectividad desde Windows hacia Wazuh

El agente se instaló mediante el paquete MSI oficial de Wazuh correspondiente a la versión 4.14.5. En la instalación se indicó la IP del manager y posteriormente se inició el servicio WazuhSvc (Wazuh, s. f.-b).

```

$ $WAZUH_MANAGER="<HOST_LAB>"

$ Invoke-WebRequest `
    -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.5-1.msi"
    -OutFile "$env:TEMP\wazuh-agent.msi"

$ Start-Process msixec.exe -Wait -ArgumentList "/i `"$env:TEMP\wazuh-agent.msi`" /qn WAZUH_MANAGER=`"$WAZUH_MANAGER`" ""

$ Start-Service WazuhSvc

$ Get-Service WazuhSvc

```

Código 21: Instalación del agente Wazuh en Windows

```

PS <HOST_LAB> -Port 443
>> <HOST_LAB> -Port 1514
>> <HOST_LAB> -Port 1515

ComputerName      : <HOST_LAB>
RemoteAddress     : <HOST_LAB>
RemotePort        : 443
InterfaceAlias    : Ethernet 9
SourceAddress     : <HOST_LAB>
TcpTestSucceeded : True

ComputerName      : <HOST_LAB>
RemoteAddress     : <HOST_LAB>
RemotePort        : 1514
InterfaceAlias    : Ethernet 9
SourceAddress     : <HOST_LAB>
TcpTestSucceeded : True

ComputerName      : <HOST_LAB>
RemoteAddress     : <HOST_LAB>
RemotePort        : 1515
InterfaceAlias    : Ethernet 9
SourceAddress     : <HOST_LAB>
TcpTestSucceeded : True

PS C:\Windows\system32> $WAZUH_MANAGER=<HOST_LAB>
>>
>> Invoke-WebRequest `
>> -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.5-1.msi" `
>> -OutFile "$env:TEMP\wazuh-agent.msi"
>>
>> msixec.exe /i "$env:TEMP\wazuh-agent.msi" /q WAZUH_MANAGER="$WAZUH_MANAGER"
>>
>> Start-Service wazuhsvc

```

Figura 39: Validación de conectividad e instalación del agente Wazuh desde PowerShell

```

PS C:\>
<RUTA_EVIDENCIAS>
Status Name DisplayName
-----
Running WazuhSvc Wazuh

```

Figura 40: Comprobación del servicio WazuhSvc en ejecución en el endpoint Windows

8.5. Fuentes de telemetría configuradas

Para que Wazuh pudiera observar las mismas fuentes relevantes que Velociraptor, el agente Windows se configuró para remitir eventos de PowerShell Operational, Sysmon, Security y System. Estas fuentes resultan necesarias para observar ejecución de scripts, creación de procesos, creación de ficheros, instalación de servicios y eventos generales del sistema (Microsoft, 2026; Russinovich & Garnier, 2026; Wazuh, s. f.-d).

```
<localfile>
  <location>Microsoft-Windows-PowerShell/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
<localfile>
  <location>Security</location>
  <log_format>eventchannel</log_format>
</localfile>
<localfile>
  <location>System</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Código 22: Fuentes eventchannel configuradas en el agente Windows

Tras modificar la configuración del agente, se reinició el servicio para aplicar los cambios.

```
$ Restart-Service WazuhSvc
$ Get-Service WazuhSvc
```

Código 23: Reinicio y comprobación del agente Wazuh en Windows

Fuente	Evento / uso principal	Utilidad para el TFM
PowerShell Operational	Event ID 4104	Visibilidad de Script Block Logging y comandos PowerShell.
Sysmon	Event ID 1	Creación de procesos, imagen, usuario y línea de comandos.
Sysmon	Event ID 11	Creación de ficheros y evidencias de staging/cifrado/borrado.
System	Event ID 7045	Instalación de servicios Windows.
Security	Eventos de seguridad Windows	Apoyo contextual y auditoría del endpoint.

Tabla 59: Fuentes de telemetría utilizadas por Wazuh en la campaña

8.6. Reglas personalizadas TFM en Wazuh

Para comparar Wazuh frente a Velociraptor en condiciones metodológicamente comparables, se creó una capa de reglas personalizadas TFM. El ruleset efectivo de la campaña fue `tfm_wazuh_custom_rules_v2.xml`, compatible con los identificadores 110301, 110402, 110203 y 110202 y con el gap 110201=0 para TEC-009. Las reglas custom se interpretan como aproximación funcional y no como equivalentes directos de los artifacts de Velociraptor (Wazuh, s. f.-a; García-Amorena Reina, 2026c; García-Amorena Reina, 2026e).

```
$ sudo mv /var/ossec/etc/rules/tfm_wazuh_custom_rules_v2.xml
$ <HOST_LAB>/tfm_wazuh_custom_rules_v1.DISABLED.xml
$ sudo systemctl restart wazuh-manager
$ sudo systemctl status wazuh-manager --no-pager
```

Código 24: Deshabilitación de la primera versión de reglas tras problemas de estabilidad

Rule ID	Perfil equivalente	Descripción funcional	Level
110201	P1	Posible exfiltración HTTP/ZIP mediante PowerShell 4104.	15

Rule ID	Perfil equivalente	Descripción funcional	Level
110202	P1	Borrado destructivo mediante PowerShell.	14
110203	P1	Comportamiento de cifrado/ransomware mediante PowerShell.	15
110301	P2	PowerShell sospechoso detectado en ScriptBlock 4104.	10
110401	P3	Posible persistencia Run Key por Sysmon Event ID 1.	8
110402	P3	Instalación de servicio Windows Event ID 7045.	8
110501	P4	Ejecución de PowerShell observada por Sysmon Event ID 1.	5

Tabla 60: Reglas personalizadas Wazuh utilizadas para aproximar los perfiles P1-P4 de Velociraptor

```
$ sudo /var/ossec/bin/wazuh-analysisd -t
$ sudo systemctl restart wazuh-manager
$ sudo systemctl status wazuh-manager --no-pager
```

Código 25: Validación sintáctica y reinicio tras cargar reglas custom TFM

```
sudo mv <RUTA_ARTIFACTS> /var/ossec/etc/rules/tfm_wazuh_custom_rules_v2.xml

sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager --no-pager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-22 11:25:14 UTC; 81ms ago
   Process: 8108 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)

   Tasks: 200 (limit: 9261)
   Memory: 487.8M (peak: 488.1M)
   CPU: 27.446s
   CGroup: /system.slice/wazuh-manager.service
           └─8174 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─8175 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─8176 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─8179 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─8182 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─8223 /var/ossec/bin/wazuh-authd
                       └─8238 /var/ossec/bin/wazuh-db
                         └─8273 /var/ossec/bin/wazuh-execd
                           └─8287 /var/ossec/bin/wazuh-analysisd
                             └─8297 /var/ossec/bin/wazuh-syscheckd
                               └─8315 /var/ossec/bin/wazuh-remoted
                                 └─8397 /var/ossec/bin/wazuh-logcollector
                                   └─8414 /var/ossec/bin/wazuh-monitord
                                     └─8430 /var/ossec/bin/wazuh-modulesd
                                       └─9117 dpkg-query -s vsftpd

Jun 22 11:25:09 admin env[8108]: Started wazuh-syscheckd...
Jun 22 11:25:10 admin env[8108]: Started wazuh-remoted...
Jun 22 11:25:10 admin env[8108]: Started wazuh-logcollector...
Jun 22 11:25:11 admin env[8108]: Started wazuh-monitord...
Jun 22 11:25:11 admin env[8427]: 2026/06/22 11:25:11 wazuh-modulesd:router: INFO: Loaded router module.
Jun 22 11:25:11 admin env[8427]: 2026/06/22 11:25:11 wazuh-modulesd:content_manager: INFO: Loaded module.
Jun 22 11:25:11 admin env[8427]: 2026/06/22 11:25:11 wazuh-modulesd:inventory-harvester: INFO: Loaded module.
Jun 22 11:25:12 admin env[8108]: Started wazuh-modulesd...
Jun 22 11:25:14 admin env[8108]: Completed.
Jun 22 11:25:14 admin systemd[1]: Started wazuh-manager.service - Wazuh manager.
Hint: Some lines were ellipsized, use -l to show in full.
```

Figura 41: Reactivación de reglas personalizadas TFM y servicio wazuh-manager activo

8.7. Metodología de campañas separadas

La validación se estructuró en campañas separadas para diferenciar visibilidad, detección y ruido de coexistencia. Esta separación es crítica porque Wazuh puede registrar actividad legítima generada por Velociraptor, especialmente si el router SERVER_EVENT de Velociraptor ejecuta PowerShell, escribe JSONL o envía alertas mediante webhook.

Campaña	Condición	Objetivo	Uso en la comparación
A - Velociraptor custom	Artifacts P1/P2/P3/P4 + router	Medir detección/alertabilidad de Velociraptor.	Línea base del TFM.

Campaña	Condición	Objetivo	Uso en la comparación
B - Wazuh base	Reglas TFM desactivadas	Medir visibilidad y detección out-of-the-box.	Línea base Wazuh.
C - Wazuh custom	Reglas TFM activadas	Medir mejora aportada por reglas equivalentes.	Comparación funcional frente a VR custom.

Tabla 61: Campañas definidas para la validación comparativa

8.8. Campaña B: Wazuh base

En la campaña base se desactivaron las reglas custom TFM y se ejecutaron los mismos runners de pruebas desde el endpoint Windows. El objetivo fue medir qué telemetría y alertas genera Wazuh sin lógica personalizada diseñada específicamente para el TFM.

```
$ sudo mv /var/ossec/etc/rules/tfm_wazuh_custom_rules_v2.xml
<HOST_LAB>/tfm_wazuh_custom_rules_v2.DISABLED.xml
$ sudo systemctl restart wazuh-manager
$ sudo systemctl status wazuh-manager --no-pager
```

Código 26: Desactivación de reglas custom para la campaña Wazuh base

```
$ BASE=~/TFM_WAZUH_EVIDENCE/base_OK_$(date -u +"%Y%m%d_%H%M%S")
$ mkdir -p "$BASE"
$ date -u +"%Y-%m-%dT%H:%M:%SZ" | tee "$BASE/start_utc.txt"
$ sudo bash -c 'wc -l < /var/ossec/logs/archives/archives.json' | tee
"$BASE/archives_start_line.txt"
$ sudo bash -c 'wc -l < /var/ossec/logs/alerts/alerts.json' | tee
"$BASE/alerts_start_line.txt"
```

Código 27: Marcado de inicio y líneas iniciales de logs en campaña base

```
$ cd "<HOST_LAB>\Desktop\TFM\03_RUNNERS"
$ Set-ExecutionPolicy -Scope Process Bypass -Force
$ .\TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1
$ .\TFM_Run_FP_Tests_v1.ps1
```

Código 28: Ejecución de runners TEC y falsos positivos desde Windows

```

$ date -u +"%Y-%m-%dT%H:%M:%SZ" | tee "$BASE/end_utc.txt"

$ sudo bash -c 'wc -l < /var/ossec/logs/archives/archives.json' | tee $
"$BASE/archives_end_line.txt"

$ sudo bash -c 'wc -l < /var/ossec/logs/alerts/alerts.json' | tee $
"$BASE/alerts_end_line.txt"


$ ARCH0=$(cat "$BASE/archives_start_line.txt")
$ ARCH1=$(cat "$BASE/archives_end_line.txt")
$ ALERT0=$(cat "$BASE/alerts_start_line.txt")
$ ALERT1=$(cat "$BASE/alerts_end_line.txt")


$ sudo awk -v s="$ARCH0" -v e="$ARCH1" 'NR>s && NR<=e'
/var/ossec/logs/archives/archives.json > "$BASE/wazuh_base_archives_delta
.jsonl"

$ sudo awk -v s="$ALERT0" -v e="$ALERT1" 'NR>s && NR<=e'
/var/ossec/logs/alerts/alerts.json > "$BASE/wazuh_base_alerts_delta
.jsonl"

$ sudo chown -R admin:admin "$BASE"

```

Código 29: Extracción de delta de archives.json y alerts.json en campaña base

Métrica	Resultado
Eventos crudos en archives.json	902
Alertas en alerts.json	58
PowerShell Event ID 4104	34
Sysmon Event ID 1	457
Sysmon Event ID 11	413
System Event ID 7045	0
Reglas custom 110xxx	0

Tabla 62: Resultados de visibilidad y alerta de la campaña Wazuh base

```
admin@admin:~$ cat "$BASE/wazuh_base_summary.txt"
=== WAZUH BASE VISIBILITY SUMMARY ===
Folder: <RUTA_EVIDENCIAS>

Total archives delta:
90 <RUTA_EVIDENCIAS>

Total alerts delta:
58 <RUTA_EVIDENCIAS>

PowerShell 4104:
34

Sysmon Event ID 1:
457

Sysmon Event ID 11:
413

System 7045:
0

TFM custom rule IDs should be 0 in base:
```

Figura 42: Resumen de la campaña Wazuh base, con ausencia de reglas TFM 110xxx

El resultado confirma que Wazuh base recibe eventos del endpoint y genera alertas propias sin depender de reglas TFM. En la GUI se observaron alertas base asociadas a Sysmon, por ejemplo eventos de creación de ficheros en ubicaciones temporales clasificados como posible Ingress Tool Transfer mediante reglas internas de Wazuh.

8.9. Campaña C: Wazuh custom

En la campaña custom se reactivaron las reglas TFM y se repitió el flujo de ejecución con los mismos runners. El objetivo fue comprobar si Wazuh podía transformar la telemetría del endpoint en alertas metodológicamente comparables a los perfiles de severidad definidos en Velociraptor, sin asumir equivalencia directa entre sus unidades.

```
$ sudo mv <HOST_LAB>/tfm_wazuh_custom_rules_v2.DISABLED.xml  
/var/ossec/etc/rules/tfm_wazuh_custom_rules_v2.xml  
  
$ sudo /var/ossec/bin/wazuh-analysisd -t  
  
$ sudo systemctl restart wazuh-manager  
  
$ sudo systemctl status wazuh-manager --no-pager
```

Código 30: Reactivación de reglas custom TFM para la campaña Wazuh custom

```
$ CUSTOM=~/TFM_WAZUH_EVIDENCE/custom_OK_$(date -u +"%Y%m%d_%H%M%S")  
  
$ mkdir -p "$CUSTOM"  
  
$ date -u +"%Y-%m-%dT%H:%M:%SZ" | tee "$CUSTOM/start_utc.txt"  
  
$ sudo bash -c 'wc -l < /var/ossec/logs/archives/archives.json' | tee  
"$CUSTOM/archives_start_line.txt"  
  
$ sudo bash -c 'wc -l < /var/ossec/logs/alerts/alerts.json' | tee  
"$CUSTOM/alerts_start_line.txt"
```

Código 31: Inicio de campaña Wazuh custom

```

$ date -u +"%Y-%m-%dT%H:%M:%SZ" | tee "$CUSTOM/end_utc.txt"

$ sudo bash -c 'wc -l < /var/ossec/logs/archives/archives.json' | tee
"$CUSTOM/archives_end_line.txt"

$ sudo bash -c 'wc -l < /var/ossec/logs/alerts/alerts.json' | tee
"$CUSTOM/alerts_end_line.txt"


$ ARCH0=$(cat "$CUSTOM/archives_start_line.txt")
$ ARCH1=$(cat "$CUSTOM/archives_end_line.txt")
$ ALERT0=$(cat "$CUSTOM/alerts_start_line.txt")
$ ALERT1=$(cat "$CUSTOM/alerts_end_line.txt")


$ sudo awk -v s="$ARCH0" -v e="$ARCH1" 'NR>s && NR<=e'
/var/ossec/logs/archives/archives.json > "$CUSTOM/wazuh_custom_archives
_delta.jsonl"

$ sudo awk -v s="$ALERT0" -v e="$ALERT1" 'NR>s && NR<=e'
/var/ossec/logs/alerts/alerts.json > "$CUSTOM/wazuh_custom_alerts
_delta.jsonl"


$ sudo chown -R admin:admin "$CUSTOM"

```

Código 32: Extracción de delta en campaña Wazuh custom

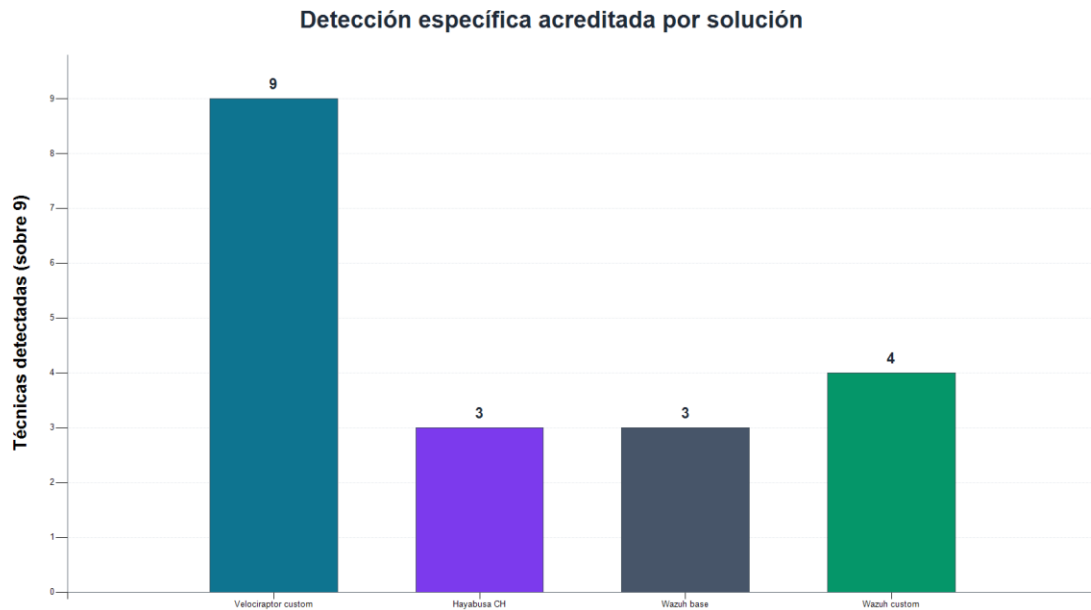
Métrica	Resultado
Eventos crudos en archives.json	18.285
Alertas en alerts.json	110
PowerShell Event ID 4104	99
Sysmon Event ID 1	127
Sysmon Event ID 11	24
System Event ID 7045	1
Alertas TFM 110xxx	56

Tabla 63: Resultados agregados de la campaña Wazuh custom

Rule ID	Número de alertas	Interpretación
110201	0	TEC-009 / exfiltración HTTP-ZIP no activada en esta campaña.
110202	5	Borrado destructivo mediante PowerShell.
110203	4	Comportamiento de cifrado/ransomware mediante PowerShell.
110301	2	PowerShell sospechoso en ScriptBlock 4104.
110402	1	Instalación de servicio Windows Event ID 7045.
110501	44	Ejecución de PowerShell observada por Sysmon Event ID 1.

Tabla 64: Distribución de reglas TFM activadas durante la campaña custom

La Figura 43 compara técnicas detectadas sobre un denominador común de nueve. Las alertas y los eventos se mantienen fuera de esta unidad



```

=== WAZUH CUSTOM VISIBILITY SUMMARY ===
Folder: <RUTA_EVIDENCIAS>

Total archives delta:
1{<RUTA_EVIDENCIAS>

Total alerts delta:
1{<RUTA_EVIDENCIAS>

PowerShell 4104:
99

Sysmon Event ID 1:
127

Sysmon Event ID 11:
24

System 7045:
1

TFM custom rule IDs:
  5 "id": "110202"
  4 "id": "110203"
  2 "id": "110301"
  1 "id": "110402"
  44 "id": "110501"

```

Nota: La regla 110201 no se activó durante la campaña custom. Este resultado se conserva como limitación de visibilidad y gap de detección para TEC-009, evidenciando que la cobertura de Wazuh depende de la lógica de reglas y de la fuente de eventos disponible.

8.10. Evidencias gráficas en el dashboard de Wazuh

El dashboard permitió validar visualmente la recepción de eventos del agente LT29, la visibilidad de fuentes concretas y la aparición de reglas personalizadas. Las capturas se obtuvieron aplicando filtros por agente, Event ID o rule.id, dentro de la ventana temporal de cada campaña.

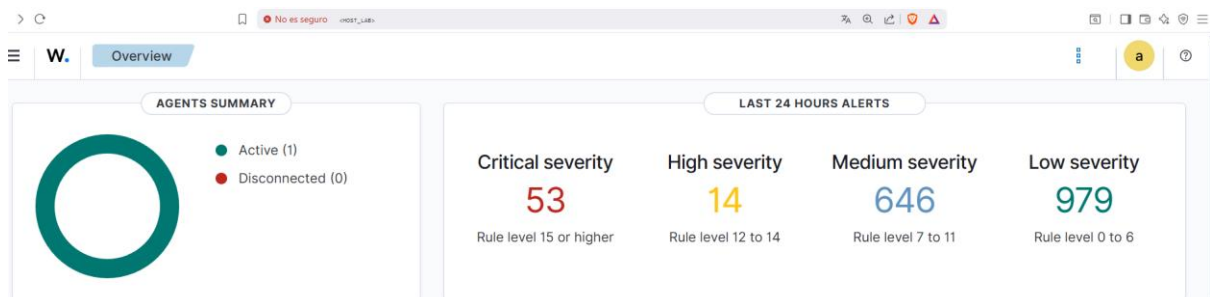


Figura 45: Resumen de severidades en el dashboard tras la campaña



Figura 46: Dashboard de Wazuh filtrado por agent.name LT29

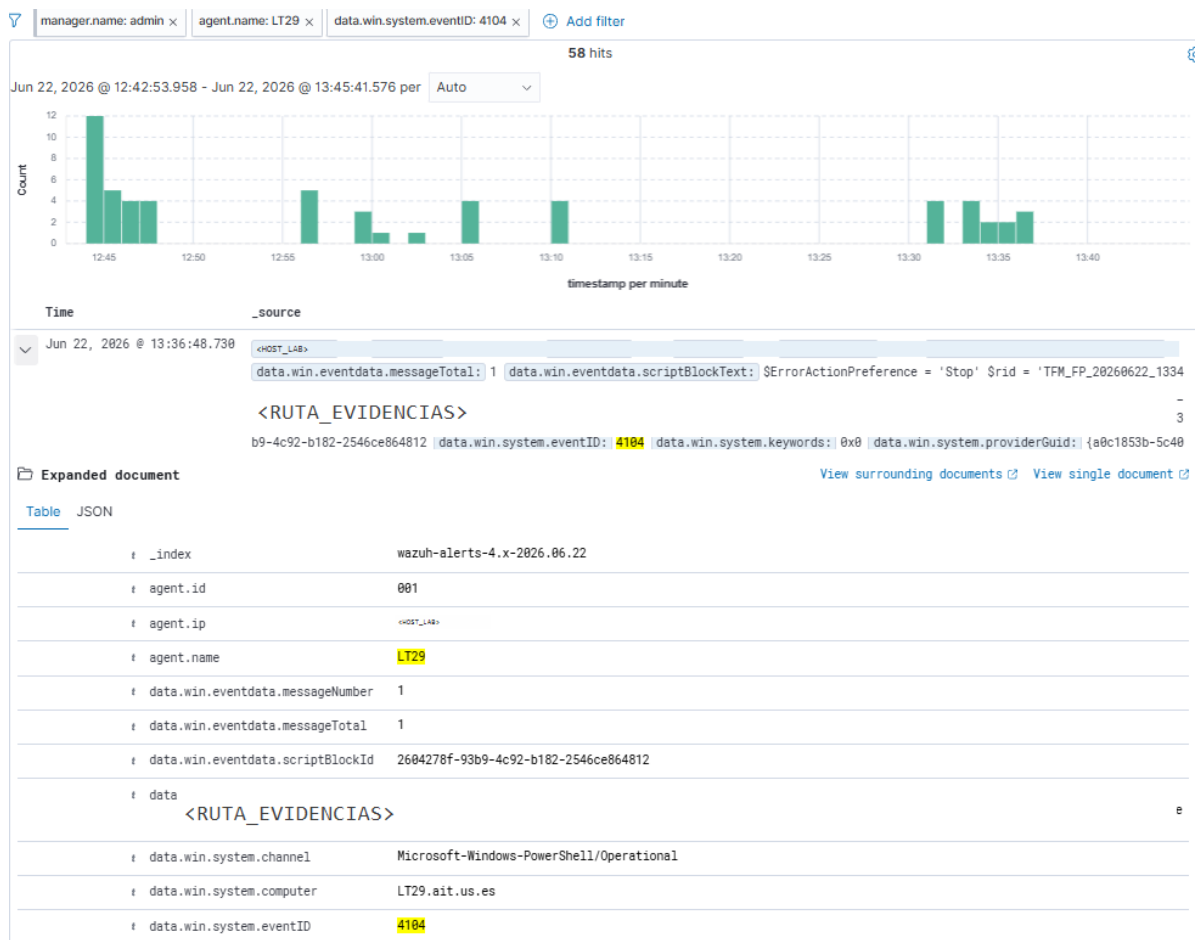


Figura 47: Visibilidad de PowerShell 4104 en Wazuh

# data.win.eventdata.sessionId	1
# data.win.eventdata.user	<HOST_LAB>
# data.win.eventdata.utcTime	2026-06-22 11:29:19.394
# data.win.system.channel	Microsoft-Windows-Sysmon/Operational
# data.win.system.computer	LT29.ait.us.es
# data.win.system.eventID	1

Figura 48: Visibilidad de Sysmon Event ID 1, asociado a creación de procesos

# data.win.eventdata.user	<HOST_LAB>
# data.win.eventdata.utcTime	2026-06-22 11:09:47.214
# data.win.system.channel	Microsoft-Windows-Sysmon/Operational
# data.win.system.computer	LT29.ait.us.es
# data.win.system.eventID	11

Figura 49: Visibilidad de Sysmon Event ID 11, asociado a creación de ficheros

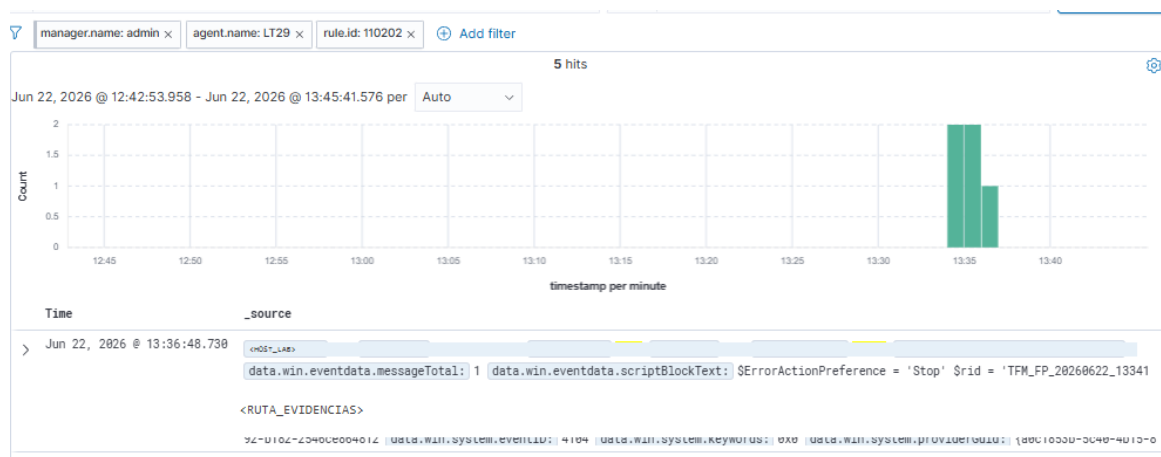


Figura 50: Ejemplo de detección custom mediante la regla 110202

manager.name: admin × agent.name: LT29 × rule.id: 110201 × Add filter

</>

No Results

Try selecting a different data source, expanding your time range or modifying the query & filters.

Figura 51: Ausencia de resultados para rule.id 110201 en la campaña custom

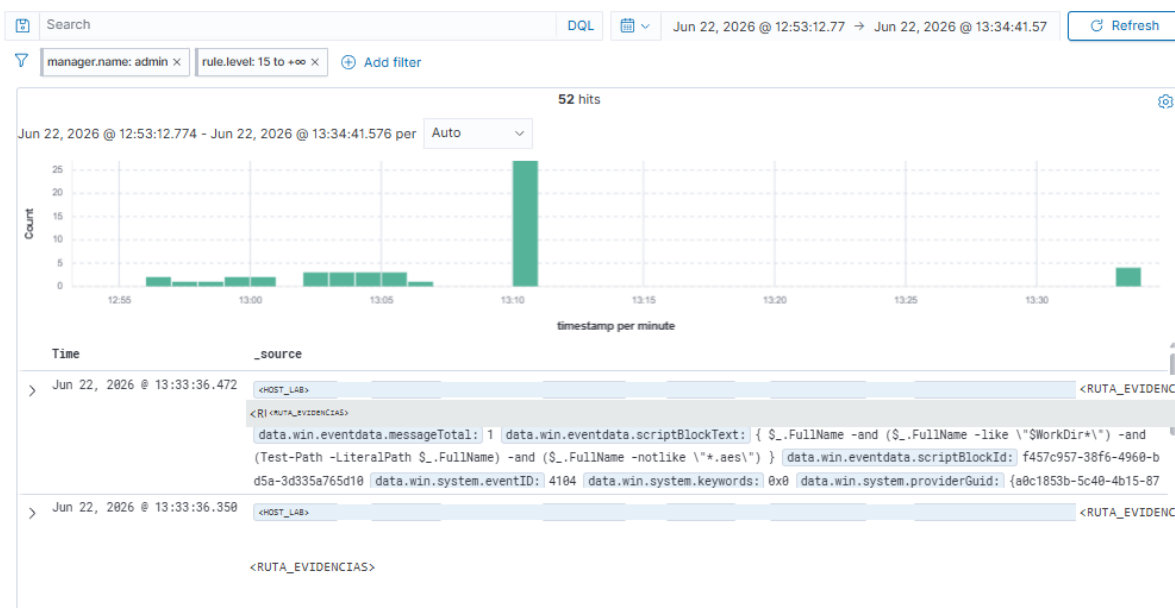


Figura 52: Vista de alertas de severidad alta/crítica generadas en Wazuh

8.11. Comparación Wazuh – Velociraptor

Sistema/campaña	Visibilidad	Detección específica	Técnicas	Limitación principal
Velociraptor custom	9/9	9/9	TEC-001 a TEC-009	CLIENT_EVENT detecta; Router/Discord no
Velociraptor públicos — Hayabusa CH	9/9	3/9	TEC-001, TEC-003, TEC-005	CHM no ejecutada; otros artifacts aportan visibilidad
Wazuh base	9/9	3/9	TEC-002, TEC-004, TEC-006	Ruleset nativo; alertas genéricas no suman técnica
Wazuh custom	9/9	4/9	TEC-001, TEC-005, TEC-007, TEC-008	110201=0; unión base+custom 7/9, no quinta solución

Tabla 65: Comparación final entre Wazuh base, Wazuh custom y Velociraptor custom

Perfil VR	Mapeo Wazuh aproximado	Criterio
P1_CRITICAL	Level 13-15	Eventos críticos o de impacto alto.

Perfil VR	Mapeo Wazuh aproximado	Criterio
P2_HIGH	Level 10-12	Alta evidencia forense o PowerShell sospechoso.
P3_MEDIUM	Level 7-9	Comportamientos relevantes de persistencia o servicio.
P4_LOW	Level 3-6	Visibilidad básica o ejecución observada.

Tabla 66: Mapeo de severidad usado entre perfiles Velociraptor y niveles Wazuh

Nota: La diferencia de volumen entre la campaña base y la campaña custom no debe interpretarse como una comparación estadística exacta del rendimiento. Son campañas separadas y pueden estar afectadas por variaciones temporales, actividad del sistema y telemetría auxiliar. La comparación principal debe centrarse en cobertura funcional, reglas activadas y calidad de evidencia.

8.12. Limitaciones observadas

Durante la validación se identificaron varias limitaciones relevantes. En primer lugar, Wazuh base genera alertas útiles sin reglas custom, pero algunas pueden tener severidad elevada ante actividad legítima o contextual del sistema. Esto obliga a revisar manualmente la semántica de la alerta antes de asociarla a una técnica MITRE concreta.

En segundo lugar, la regla 110501 generó un volumen notable de alertas P4 al detectar ejecución de PowerShell mediante Sysmon Event ID 1. Esta regla aporta visibilidad, pero puede generar ruido si no se contextualiza con la técnica ejecutada, el comando concreto y la ventana temporal de prueba.

En tercer lugar, la regla 110201, asociada a TEC-009, no se activó durante la campaña custom. Este resultado se conserva como gap de Wazuh y evidencia que la cobertura depende de la lógica de reglas y de la fuente de eventos disponible; no se corrige ni se oculta en esta iteración documental (García-Amorena Reina, 2026c; García-Amorena Reina, 2026e).

Por último, al coexistir con Velociraptor, Wazuh puede registrar actividad operacional del propio sistema de alertado de VR, como ConvertFrom-Json, FromBase64String, Add-Content, Invoke-RestMethod, escritura en soc_alerts.jsonl o envío a Discord. Por ello, dicha coexistencia debe tratarse como experimento de ruido operativo, no como campaña principal de detección.

8.13. Registro y almacenamiento de evidencias

La fuente cuantitativa canónica para visibilidad, detección, falsos positivos y comparación Wazuh es `TFM\_VISIBILIDAD\_DETECCION\_FP\_WAZUH\_DEFINITIVO\_20260712.xlsx`. Los documentos y normalizados históricos se conservan solo como trazabilidad y no se utilizan para fijar cifras finales. La comparación mantiene separadas las unidades de Wazuh —archives, alerts y reglas— de las filas CLIENT_EVENT de Velociraptor.

Capítulo IX: Discusión y trabajo futuro

Este capítulo presenta las principales líneas de trabajo futuro derivadas del desarrollo y validación experimental realizados. Aunque los resultados obtenidos permiten confirmar la viabilidad de Velociraptor como herramienta host-based para detección y respuesta forense en un entorno controlado, el alcance del proyecto queda limitado a un laboratorio Windows, a nueve técnicas MITRE ATT&CK y a una arquitectura basada en artifacts personalizados.

9.1. Discusión integrada de resultados

Los resultados solo son comparables cuando se conserva la unidad de análisis. Velociraptor custom acredita técnicas mediante filas CLIENT_EVENT clasificadas; los artifacts públicos aportan visibilidad o detección específica según su lógica; Wazuh acredita reglas y alertas; el benchmark mide recursos. Un evento, una alerta, una técnica, una evidencia forense y una salida externa no son magnitudes intercambiables.

Dentro del subconjunto TEC-001 a TEC-009, Velociraptor custom alcanzó 9/9, Hayabusa CH 3/9, Wazuh base 3/9 y Wazuh custom 4/9. La unión base+custom de Wazuh es complementaria y alcanza 7/9, pero no constituye una solución adicional. Estos resultados describen la configuración ensayada y no demuestran superioridad universal de Velociraptor ni equivalencia funcional con un EDR completo.

Las 379 filas CLIENT_EVENT emitidas/exportadas, incluidas emisiones repetidas describen volumen: P1=19, P2=118, P3=109 y P4=133. No miden por sí mismas cobertura ni calidad. De forma análoga, los volúmenes de alerts y archives de Wazuh se mantienen como contexto y no se utilizan para sumar técnicas.

El benchmark acredita 9/9 runs válidos y un coste agregado del cliente. No mide cada artifact P1–P4 de manera individual, ni detección, falsos positivos o escalabilidad. La evaluación pública conserva ETW como NO CONCLUYENTE y TrackNetwork como visible sin atribución fiable de proceso. En TEC-009, el HTTP 200 es ground truth del runner independiente, separado de la detección CLIENT_EVENT.

9.2. Ampliación de técnicas MITRE

El trabajo futuro más directo consiste en ampliar el subconjunto TEC-001 a TEC-009 hacia más tácticas de MITRE ATT&CK Enterprise, especialmente defensa evasión, credentials access, lateral movement y command and control. La ampliación debería mantener el mismo criterio: técnica observable en host, runner reproducible, datos dummy y separación entre ejecución, visibilidad, detección, alerta y evidencia.

Por proponer algunas, primero se deberían priorizar comportamientos frecuentes en campañas actuales. Por ejemplo:

MITRE	Técnica	Por qué interesa	Emulación segura
T1003	OS Credential Dumping	Muy relevante en Credential Access y movimiento lateral posterior	Simulación con eventos y accesos controlados, sin extraer credenciales reales
T1078	Valid Accounts	Muy usada en intrusiones reales y ransomware	Cuentas dummy, logon controlado, eventos 4624/4625
T1021	Remote Services	Clave para lateral movement	RDP/WinRM/SMB controlado entre VMs
T1562.001	Disable or Modify Tools	Muy relevante en evasión de defensas	Cambios simulados o sobre servicios dummy
T1055	Process Injection	Técnica muy usada para evasión	Solo PoC inocuo o detección teórica; cuidado con complejidad

Tabla 67: Técnicas MITRE ATT&CK propuestas para trabajo futuro

9.3. Desarrollo de nuevos artifacts

La arquitectura P1/P2/P3/P4 puede evolucionar con reglas más agregadas, correlación temporal y nuevas fuentes de telemetría. El objetivo sería reducir ruido en los perfiles P3/P4 sin perder evidencia forense, y reservar P1/P2 para señales de mayor criticidad.

Una mejora clara sería aplicar agregación temporal en técnicas como TEC-008. En lugar de generar una alerta por cada fichero eliminado, el artifact podría agrupar múltiples eventos Sysmon Event ID 26 dentro de una misma ventana temporal. De esta forma, el sistema detectaría el patrón de borrado masivo sin producir un volumen excesivo de alertas.

Otra línea de mejora sería crear artifacts específicos para cadenas multi-evento. Por ejemplo, una detección podría exigir la secuencia: ejecución de PowerShell, creación de archivo comprimido, conexión HTTP y escritura de evidencia en el summary. Este enfoque reduciría falsos positivos y aumentaría la calidad de las alertas.

9.4. Ciclo de vida y trazabilidad de los artifacts

Las evoluciones futuras de los artifacts deberán conservar un ciclo de vida trazable: creación como versión de trabajo, validación experimental frente al ground truth, revisión de ruido y promoción a versión validada solo cuando el resultado sea reproducible

9.5. Automatización de detecciones

La automatización debería evolucionar hacia un modelo más cercano a detection engineering. Actualmente, los artifacts CLIENT_EVENT generan eventos y el router SOC_v3 actúa como canal de salida hacia JSONL y Discord. Como trabajo futuro, este flujo podría ampliarse con validaciones automáticas de calidad, etiquetado de severidad y generación de evidencias normalizadas.

Una mejora útil sería generar automáticamente resúmenes por campaña, incluyendo número de técnicas ejecutadas, detecciones por perfil, alertas enviadas, eventos descartados y posibles falsos positivos. Esto permitiría comparar campañas de forma más rápida y reducir el trabajo manual de revisión. También podría incorporarse un sistema de validación continua de artifacts. Cada vez que se modifique un artifact P1/P2/P3/P4, el runner podría ejecutar una campaña mínima de regresión para comprobar que no se rompe la detección de técnicas ya validadas. Esta aproximación permitiría tratar las reglas como código, con control de versiones, hashes, changelog y pruebas reproducibles.

9.6. Integración SIEM/SOC y despliegue en entornos reales

Un despliegue real requeriría probar retención de eventos, control de ruido, seguridad de webhooks, gestión de credenciales, tuning por baseline de usuario y exportación hacia SIEM. En este trabajo se ha validado la viabilidad técnica en laboratorio, pero un entorno corporativo exige controles adicionales. La integración con SIEM permitiría centralizar los eventos generados por Velociraptor, correlacionarlos con otras fuentes y construir casos de uso SOC más completos. Por ejemplo, una alerta de PowerShell sospechoso podría enriquecerse con eventos de autenticación, conexiones de red, inventario de endpoint o actividad previa del usuario.

Otro punto importante sería la seguridad operativa del canal de salida. Los webhooks, tokens y credenciales no deberían quedar en texto plano dentro de artifacts o scripts. En un despliegue real deberían protegerse mediante mecanismos de gestión de secretos, permisos mínimos y rotación periódica.

Por último, sería necesario repetir el benchmark en hardware distinto, con más endpoints, mayor duración y cargas más realistas. Las métricas obtenidas en este TFM son válidas para el laboratorio evaluado, pero no deben extrapolarse directamente a producción sin pruebas adicionales.

Capítulo X: Conclusiones

Este capítulo recoge las conclusiones principales del TFM, relacionando los resultados obtenidos con los objetivos planteados al inicio del trabajo. La evaluación se centra en determinar hasta qué punto Velociraptor puede aproximarse a un comportamiento HIDS/DFIR en Windows mediante monitorización continua, artifacts personalizados y una metodología reproducible basada en técnicas MITRE ATT&CK Enterprise.

10.1. Conclusiones principales

El resultado del TFM muestra que Velociraptor puede utilizarse como una aproximación HIDS/DFIR en Windows cuando se combina monitorización continua mediante CLIENT_EVENT, artifacts custom orientados a comportamiento y una metodología estricta de validación. En la campaña final, el subconjunto TEC-001 a TEC-009 obtuvo detección mediante CLIENT_EVENT para las nueve técnicas evaluadas (García-Amorena Reina, 2026e).

La aportación técnica principal no reside únicamente en la ejecución de técnicas MITRE, sino en la construcción de una cadena completa de detection engineering. Esta cadena incluye la selección de técnicas, la ejecución controlada mediante runners, la generación de telemetría Windows, el desarrollo de artifacts VQL, la clasificación por perfiles de artifacts P1/P2/P3/P4, la separación entre detección y salida externa, el análisis de falsos positivos, el benchmark de rendimiento y la validación comparativa con Wazuh.

La evaluación definitiva de artifacts públicos comprende seis campañas CLIENT_EVENT: cinco resultados positivos de visibilidad y uno NO CONCLUYENTE. Windows.Hayabusa.Monitoring aportó detección específica CH en 3/9 técnicas —TEC-001, TEC-003 y TEC-005—, mientras que ServiceCreation, ProcessCreation y SysmonLogForward aportaron visibilidad o evidencia sin equivaler por sí solos a detección MITRE. Windows.ETW.Monitoring quedó NO CONCLUYENTE por falta de una campaña reproducible completa. Generic.Events.TrackNetworkConnections observó la conexión, pero con PID=0 y ProcInfo vacío; el HTTP 200 fue registrado por el runner independiente.

También se concluye que Velociraptor ofrece una arquitectura flexible para construir detecciones propias, pero exige un esfuerzo significativo de ajuste. La calidad de los resultados depende de la telemetría disponible en el endpoint, de la configuración de Sysmon y Windows Event Logs, de la precisión de los artifacts y de la validación experimental frente a ground truth. Por tanto, Velociraptor no debe entenderse como un HIDS cerrado y listo para producción, sino como una plataforma DFIR extensible sobre la que es posible construir capacidades de detección host-based.

La comparativa con Wazuh contextualiza los resultados sin afirmar superioridad universal. Sobre el mismo denominador de nueve técnicas, Wazuh base acreditó detección específica en 3/9 —TEC-002, TEC-004 y TEC-006— y las reglas Wazuh custom en 4/9 —TEC-001, TEC-005, TEC-007 y TEC-008—. La unión complementaria alcanza 7/9, pero no se trata como una quinta solución. El volumen de alertas se conserva como contexto y no como medida de cobertura, del mismo modo que las 379 filas CLIENT_EVENT de Velociraptor no equivalen a 379 técnicas detectadas.

10.2. Respuesta a objetivos

La respuesta sintética y sus límites se recogen en la Tabla 68.

Pregunta de investigación	Resultado	Evidencia principal	Limitación
RQ1 · Cobertura custom	9/9 en TEC-001 a TEC-009	Filas CLIENT_EVENT de P1–P4 y matriz por técnica	Solo aplica al subconjunto y laboratorio evaluados
RQ2 · Artifacts públicos	Cinco campañas positivas de visibilidad y una no concluyente; Hayabusa CH 3/9	Campañas públicas y matriz de resultados	ETW no concluyente; TrackNetwork sin atribución fiable
RQ3 · Rendimiento	9/9 runs válidos en tres escenarios	BASELINE_NO_VR, VR_IDLE y VR_TEC_RUNNER; CPU, RAM y duración	Coste del cliente en laboratorio; no extrapolable a producción
RQ4 · Arquitectura personalizada	P1–P4 cubren las brechas evaluadas y Router no detecta	Cuatro detectores CLIENT_EVENT y un Router SERVER_EVENT	P2 mezcla detección, contexto y evidencia; requiere interpretación por source
RQ5 · Falsos positivos	10/10 pruebas benignas; 0 hits definitivos	summary.json y vr_hits.csv reconciliados	82 filas no reconciliadas excluidas del cómputo definitivo
RQ6 · Separación de capas	Visibilidad, detección, alerta, evidencia y salida externa se tratan por separado	CLIENT_EVENT detecta; SERVER_EVENT/JSONL/Discord enrutan, persisten o notifican	La salida externa no acredita detección del host
RQ7 · Comparación con Wazuh	Wazuh base 3/9; custom 4/9; unión complementaria 7/9	Reglas nativas y tfm_wazuh_custom_rules_v2.xml	Unidades y modelos operativos no equivalentes

Tabla 68: Correspondencia entre preguntas de investigación, resultados y limitaciones

En conjunto, los objetivos se consideran cumplidos dentro del alcance experimental. La evaluación acredita detección custom Velociraptor en 9/9 técnicas, seis campañas de artifacts públicos con cinco resultados positivos y uno no concluyente, Wazuh base en 3/9, Wazuh custom en 4/9, 10/10 pruebas benignas con 0 hits y 9/9 runs válidos de benchmark. Cada resultado conserva su propia unidad:

técnica, alerta CLIENT_EVENT, evento, regla, evidencia forense, salida externa o métrica de rendimiento.

10.3. Limitaciones identificadas

Las conclusiones se limitan a un laboratorio controlado, con Windows 10, telemetría configurada, scripts dummy y un subconjunto reducido de técnicas. No se evalúa malware real, no se comprometen sistemas de terceros y no se afirma cobertura completa de MITRE ATT&CK Enterprise. Los resultados deben interpretarse como una validación experimental acotada, no como una certificación general de capacidad defensiva.

Una limitación relevante fue la dependencia de la visibilidad del endpoint. Algunas técnicas solo pudieron analizarse correctamente tras ajustar la telemetría disponible, especialmente mediante Sysmon y Windows Event Logs. Esto demuestra que la detección no depende únicamente del artifact de Velociraptor, sino también de la calidad de las fuentes host-based configuradas en el sistema (Microsoft, 2026; Russinovich & Garnier, 2026).

En Wazuh se conserva la limitación asociada a la regla 110201, que no llegó a activarse para TEC-009. Esto no invalida la comparativa, pero sí refleja que la cobertura de una técnica puede variar notablemente según la lógica de reglas disponible y la fuente de eventos utilizada.

El benchmark de rendimiento también presenta limitaciones. Mide el coste operativo agregado del cliente Velociraptor en el laboratorio, no la calidad de detección, el consumo individual de P1–P4, la escalabilidad en flotas ni el comportamiento bajo cargas prolongadas. Los valores observados —0,95 % de CPU media y 54 % de pico en VR_TEC_RUNNER; 57,42 MB de RAM media y 57,97 MB de pico— no se extrapolan a otros equipos o despliegues.

10.4. Valor del enfoque

El valor principal del enfoque reside en hacer auditable la cadena completa de razonamiento. Para cada técnica evaluada se puede identificar qué acción se ejecutó, qué fuente generó telemetría, qué artifact la clasificó, qué perfil se asignó, qué evidencia se conservó y qué salida externa se produjo. Esta trazabilidad permite defender los resultados frente a un tribunal técnico y evita conclusiones basadas únicamente en capturas aisladas o en afirmaciones no verificables.

La separación entre visibilidad, detección, alerta y evidencia aporta rigor metodológico. Un evento visible en Windows o Sysmon no se ha considerado automáticamente como detección; solo se ha tratado como detección cuando un artifact CLIENT_EVENT lo ha clasificado de forma reproducible y asociada a una técnica concreta. Esta decisión fortalece la validez del trabajo y reduce el riesgo de sobreinterpretar los resultados.

El uso de perfiles de artifacts P1/P2/P3/P4 también aporta valor operativo. Permite diferenciar entre señales críticas, evidencia forense relevante, comportamientos sospechosos y visibilidad básica. Esta clasificación evita que todos los eventos se traten con la misma severidad y aproxima el sistema a una lógica operativa de SOC sin confundir estos perfiles con prioridades documentales de planificación.

Finalmente, el TFM demuestra que Velociraptor puede ser una herramienta útil para construir capacidades HIDS/DFIR personalizadas, siempre que se acompañe de una metodología rigurosa, telemetría suficiente, validación experimental y documentación de limitaciones. La solución desarrollada no sustituye a un EDR comercial ni a un SIEM completo, pero sí muestra que Velociraptor puede aportar visibilidad, detección reproducible y evidencia forense trazable en escenarios host-based controlados.

Capítulo XI: Referencias bibliográficas y anexos

Las referencias se han normalizado en formato APA y se separan entre fuentes externas oficiales y fuentes internas del TFM. Las fuentes externas sustentan conceptos, taxonomías y documentación de herramientas. Las fuentes internas sustentan resultados experimentales, decisiones metodológicas, datasets, artifacts y evidencias generadas durante el laboratorio.

Las fuentes internas no sustituyen a la bibliografía externa: se citan para mantener trazabilidad académica entre afirmaciones de resultados y evidencias locales. Cuando una afirmación depende de la campaña experimental, se apoya en los Excel finales, logs normalizados, artifacts validados o documentos de contexto del TFM.

- García-Amorena Reina, J. (2026a). Artifacts Velociraptor validados P1/P2/P3/P4 y router SOC_v3 [Artifacts YAML internos del TFM]. 01_ARTIFACTS/validated.
- García-Amorena Reina, J. (2026b). Decisiones metodológicas, matriz de pruebas e índice de evidencias del TFM [Documentación interna].
<RAÍZ_TFM>\00_CONTEXT\DECISIONS.md;
<RAÍZ_TFM>\00_CONTEXT\TEST_MATRIX.md;
<RAÍZ_TFM>\00_CONTEXT\EVIDENCE_INDEX.md.
- García-Amorena Reina, J. (2026c). Reglas Wazuh personalizadas del TFM [Archivo XML interno]. 10_WAZUH/tfm_wazuh_custom_rules_v2.xml.
- García-Amorena Reina, J. (2026d). TFM_BENCHMARK_RENDIMIENTO_VELOCIRAPTOR_DEFINITIVO_20260712.xlsx [Conjunto de datos interno definitivo del TFM]. <RUTA_EVIDENCIAS>.
- García-Amorena Reina, J. (2026e). TFM_VISIBILIDAD_DETECCION_FP_WAZUH_DEFINITIVO_20260712.xlsx [Conjunto de datos interno definitivo del TFM]. <RUTA_EVIDENCIAS>.
- Microsoft. (2026). about_Locking_Windows. Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging_windows?view=powershell-7.6
- MITRE. (s. f.-a). Archive collected data: Archive via utility (T1560.001). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1560/001/>
- MITRE. (s. f.-b). Boot or logon autostart execution: Registry run keys / startup folder (T1547.001). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1547/001/>

- MITRE. (s. f.-c). Command and scripting interpreter: PowerShell (T1059.001). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1059/001/>
- MITRE. (s. f.-d). Command and scripting interpreter: Windows command shell (T1059.003). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1059/003/>
- MITRE. (s. f.-e). Data destruction (T1485). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1485/>
- MITRE. (s. f.-f). Data encrypted for impact (T1486). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1486/>
- MITRE. (s. f.-g). Data staged: Local data staging (T1074.001). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1074/001/>
- MITRE. (s. f.-h). Enterprise matrix. MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/matrices/enterprise/>
- MITRE. (s. f.-i). Exfiltration over alternative protocol: Exfiltration over unencrypted non-C2 protocol (T1048.003). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1048/003/>
- MITRE. (s. f.-j). Scheduled task/job: Scheduled task (T1053.005). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1053/005/>
- MITRE. (s. f.-k). Software discovery: Security software discovery (T1518.001). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1518/001/>
- MITRE. (s. f.-l). System services: Service execution (T1569.002). MITRE ATT&CK. Recuperado el 8 de julio de 2026, de <https://attack.mitre.org/techniques/T1569/002/>
- Russinovich, M., & Garnier, T. (2026, 26 de marzo). Sysmon v15.2. Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
- Velociraptor. (s. f.-a). Artifacts. Velociraptor documentation. Recuperado el 8 de julio de 2026, de <https://docs.velociraptor.app/docs/artifacts/>
- Velociraptor. (s. f.-b). Event queues. Velociraptor documentation. Recuperado el 8 de julio de 2026, de https://docs.velociraptor.app/docs/artifacts/event_queues/
- Velociraptor. (s. f.-c). Server monitoring. Velociraptor documentation. Recuperado el 8 de julio de 2026, de https://docs.velociraptor.app/docs/server_automation/server_monitoring/
- Velociraptor. (s. f.-d). VQL. Velociraptor documentation. Recuperado el 8 de julio de 2026, de <https://docs.velociraptor.app/docs/vql/>
- Wazuh. (s. f.-a). Custom rules. Rules. Wazuh documentation. Recuperado el 8 de julio de 2026, de <https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html>

- Wazuh. (s. f.-b). Deploying Wazuh agents on Windows endpoints. Wazuh documentation. Recuperado el 8 de julio de 2026, de <https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html>
- Wazuh. (s. f.-c). Installing the Wazuh indexer using the assisted installation method. Wazuh documentation. Recuperado el 8 de julio de 2026, de <https://documentation.wazuh.com/current/installation-guide/wazuh-indexer/installation-assistant.html>
- Wazuh. (s. f.-d). Log data collection and Windows event monitoring. Wazuh documentation. Recuperado el 8 de julio de 2026, de <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html>

Anexos

11.1. Finalidad de los anexos

Los anexos reúnen la trazabilidad técnica necesaria para defender la memoria sin duplicar evidencias brutas. Su función es conectar técnicas, laboratorio, artifacts, telemetría, datasets, benchmark y comparación con Wazuh. No sustituyen a los ficheros originales conservados en las carpetas de evidencias.

11.2. Anexo A. Técnicas MITRE seleccionadas y trazabilidad

- A.1. Criterio general. Las técnicas se seleccionaron por ser observables en Windows, ejecutables de forma controlada, reproducibles mediante runners y analizables mediante telemetría host-based. El conjunto no pretende cubrir toda la matriz ATT&CK Enterprise, sino un subconjunto defendible para evaluar capacidad HIDS/DFIR en laboratorio.
- A.2. TEC-001 - T1059.001 PowerShell. Táctica principal: Execution. Objetivo experimental: ejecución controlada de PowerShell y generación de telemetría 4104/Sysmon. Evidencia esperada: proceso powershell.exe, Script Block Logging y eventos de creación de proceso. Fuente MITRE: MITRE (s. f.-c).
- A.3. TEC-002 - T1059.003 Windows Command Shell. Táctica principal: Execution. Objetivo experimental: ejecución controlada mediante cmd.exe y comandos encadenados. Evidencia esperada: Sysmon Event ID 1, línea de comandos y relación proceso padre/hijo. Fuente MITRE: MITRE (s. f.-d).

- A.4. TEC-003 - T1053.005 Scheduled Task. Tácticas principales: Execution y Persistence. Objetivo experimental: creación y ejecución de tarea programada controlada. Evidencia esperada: eventos de tarea, creación de proceso asociado y trazas de runner. Fuente MITRE: MITRE (s. f.-j).
- A.5. TEC-004 - T1547.001 Registry Run Keys / Startup Folder. Tácticas principales: Persistence y Privilege Escalation. Objetivo experimental: modificación controlada de claves Run/RunOnce. Evidencia esperada: modificación de registro y proceso asociado. Fuente MITRE: MITRE (s. f.-b).
- A.6. TEC-005 - T1569.002 Service Execution. Tácticas principales: Execution y Persistence. Objetivo experimental: creación/ejecución controlada de servicio. Evidencia esperada: Sysmon Event ID 1 y System 7045 cuando aplica. Fuente MITRE: MITRE (s. f.-l).
- A.7. TEC-006 - T1518.001 Security Software Discovery. Táctica principal: Discovery. Objetivo experimental: consulta controlada de software o servicios de seguridad. Evidencia esperada: comandos de enumeración, PowerShell 4104 y creación de procesos. Fuente MITRE: MITRE (s. f.-k).
- A.8. TEC-007 - T1486 Data Encrypted for Impact. Táctica principal: Impact. Objetivo experimental: cifrado controlado de datos dummy, sin afectar datos reales. Evidencia esperada: operaciones sobre ficheros dummy, scripts de laboratorio y señales de comportamiento. Fuente MITRE: MITRE (s. f.-f).
- A.9. TEC-008 - T1485 Data Destruction. Táctica principal: Impact. Objetivo experimental: eliminación controlada de ficheros dummy. Evidencia esperada: PowerShell 4104 y Sysmon Event ID 26 como evidencia forense, no como alerta individual. Fuente MITRE: MITRE (s. f.-e).
- A.10. TEC-009 - T1074.001, T1560.001 y T1048.003. Tácticas principales: Collection y Exfiltration. Objetivo experimental: staging, archivado ZIP y exfiltración HTTP controlada. El runner independiente acredita HTTPStatus=200 y UploadSucceeded=True; la detección Velociraptor se apoya en señales CLIENT_EVENT host-based y no en ese HTTP 200. Fuentes MITRE: MITRE (s. f.-g), MITRE (s. f.-a) y MITRE (s. f.-i).
- A.11. Resultado sintético. Las nueve técnicas TEC-001 a TEC-009 aparecen como ejecutadas y detectadas mediante CLIENT_EVENT en el Excel final de visibilidad. Este resultado se limita al laboratorio y no representa cobertura completa de MITRE ATT&CK Enterprise (García-Amorena Reina, 2026e).

11.3. Anexo B. Configuración del laboratorio

- B.1. Alcance. El laboratorio se diseñó para evaluar técnicas host-based en Windows mediante ejecución controlada. No se empleó malware, no se afectaron sistemas de terceros y los datos usados fueron dummy.
- B.2. Endpoint observado. Sistema Windows de laboratorio con agente Velociraptor, Sysmon y logging de PowerShell habilitado. La memoria utiliza referencias de rutas internas únicamente como trazabilidad experimental; esas rutas no deben tratarse como IOC.
- B.3. Velociraptor. La arquitectura evaluada se basa en cliente, servidor, artifacts, CLIENT_EVENT para detección, SERVER_EVENT para enrutamiento/persistencia/notificación y exportaciones JSONL/CSV como salidas de explotación analítica. SERVER_EVENT no se formula como capa de detección.
- B.4. Sysmon. Sysmon aporta eventos de creación de procesos, conexiones de red, creación de ficheros y eliminación detectada. En este TFM, Sysmon Event ID 26 se usa como evidencia forense de borrado, no como alerta individual (Russovich & Garnier, 2026).
- B.5. PowerShell. PowerShell Script Block Logging aporta Event ID 4104 para observar comandos y bloques de script ejecutados. Esta fuente es relevante para PowerShell, discovery, cifrado controlado, eliminación y exfiltración HTTP controlada cuando el comportamiento se ejecuta mediante PowerShell (Microsoft, 2026).
- B.6. Wazuh. Wazuh se usa como validación comparativa HIDS/SIEM, no como sustituto de Velociraptor. Su modelo se apoya en logs, reglas, archives y alerts, mientras que Velociraptor se apoya en artifacts, VQL, CLIENT_EVENT y evidencia host-based.
- B.7. Restricción metodológica. Codex no valida experimentalmente resultados. La validez experimental depende de las campañas ejecutadas por el autor y de las evidencias conservadas en los Excel finales, logs normalizados y paquetes de evidencias.

11.4. Anexo C. Fuentes de telemetría y criterio de evidencia

- C.1. Separación de capas. La memoria distingue ejecución de técnica, visibilidad host-based, detección, alerta, salida externa, evidencia, falsos positivos, benchmark y comparación con Wazuh. Esta separación evita inferir detección a partir de mera presencia de eventos.
- C.2. Visibilidad host-based. Se considera visibilidad cuando una fuente local registra actividad observable: Windows Event Logs, Sysmon, PowerShell 4104, ficheros creados, claves de registro, servicios o conexiones. La visibilidad por sí sola no equivale a detección.

- C.3. Detección. Se considera detección defendible cuando un artifact CLIENT_EVENT clasifica un evento o patrón, lo vincula a una técnica concreta y conserva evidencia trazable. La detección defendible en este TFM ocurre en CLIENT_EVENT.
- C.4. Alerta. Se considera alerta cuando una detección se normaliza y se eleva a una señal operativa con perfil de severidad. P1/P2/P3/P4 son perfiles de artifacts y severidad/visibilidad, no prioridades de planificación.
- C.5. Salida externa. JSONL y Discord son salidas externas o mecanismos de notificación. No son fuente primaria de detección y no sustituyen a CLIENT_EVENT ni a la evidencia local.
- C.6. Evidencia forense. Se considera evidencia forense cualquier dato conservado que permita reconstruir la actividad: eventos, logs, hashes, capturas o resúmenes de ejecución. Sysmon ID 26 se trata como evidencia de eliminación, no como alerta individual.
- C.7. Falsos positivos. La campaña de FP se analiza separada de la campaña real. El Excel final registra 10/10 acciones benignas y 0 hits definitivos; 82 filas no reconciliadas fueron excluidas y no se computaron como falsos positivos definitivos, manteniendo separadas las filas mixtas mediante filtros de ventana/carpeta (García-Amorena Reina, 2026e).

11.5. Anexo D. Artifacts Velociraptor validados

- D.1. Control de versiones. Los artifacts validados se conservan en una ubicación separada y no se sobrescriben. Cualquier evolución futura se realiza sobre una nueva versión de trabajo, que debe validarse experimentalmente antes de su promoción.
- D.2. Custom.TFM.HIDS.P1.Critical.Priority.Event_v1.yaml. Perfil P1 Critical. Orientado a señales de alta criticidad, como comportamientos destructivos, cifrado, persistencia fuerte o exfiltración HTTP controlada cuando la evidencia lo soporta.
- D.3. Custom.TFM.HIDS.P2.High.Forensic.Event_v1.yaml. Perfil P2 High. Orientado a evidencia forense de alta relevancia, útil para consolidar detecciones y sostener trazabilidad técnica.
- D.4. Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1.yaml. Perfil P3 Medium. Orientado a comportamiento sospechoso que requiere contexto, correlación temporal o combinación con otras señales.
- D.5. Custom.TFM.HIDS.P4.Low.Basic_v2.yaml. Perfil P4 Low. Orientado a visibilidad básica. No debe formularse como alerta crítica ni como prueba suficiente de compromiso.
- D.6. Custom.TFM.HIDS.Router.JSONL.Discord.SOC_v3.yaml. Artifact SERVER_EVENT. Su función es enrutar, normalizar, persistir o notificar eventos generados por CLIENT_EVENT. No detecta por sí mismo.
- D.7. SHA256SUMS_P1_P2_RECALIBRATION.txt. Fichero de trazabilidad criptográfica asociado a artifacts recalibrados. Debe conservarse como control de integridad documental.

- D.8. Decisión de diseño. La arquitectura P1/P2/P3/P4 no representa herramientas independientes. Representa perfiles operativos de severidad y visibilidad sobre una lógica CLIENT_EVENT común, con salida posterior por router SERVER_EVENT cuando aplica (García-Amorena Reina, 2026a).

11.6. Anexo E. Runners, scripts y ground truth

- E.1. Función de los runners. Los runners automatizan la ejecución controlada de técnicas TEC-001 a TEC-009, registran tiempos y generan evidencias. Su función metodológica es construir ground truth, no detectar.
- E.2. Runner global. La memoria referencia 'TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1' como runner global de campaña. Ejecuta técnicas de forma secuencial, introduce pausas y facilita correlación temporal con telemetría.
- E.3. Ground truth. El ground truth se apoya en ejecución planificada, ventana temporal, técnica esperada, runner usado, evidencia producida y resumen de campaña. La detección se evalúa comparando esta verdad de laboratorio con las señales CLIENT_EVENT.
- E.4. Datos dummy. Los scripts operan sobre datos dummy y rutas controladas de laboratorio. No deben redactarse como malware real ni como compromiso de datos sensibles.
- E.5. Rutas y nombres. Las rutas de laboratorio se documentan para reproducibilidad, pero no forman parte de la lógica de detección ni se tratan como IOC.
- E.6. Control de seguridad. Las técnicas de impacto se limitaron a cifrado/eliminación controlados sobre ficheros dummy; las técnicas de exfiltración se limitaron a transferencia HTTP controlada hacia receptor del investigador.

11.7. Anexo F. Evidencias y datasets finales

- F.1. Excel final de visibilidad. '<RUTA_EVIDENCIAS>/TFM_VISIBILIDAD_DETECCION_FP_WAZUH_DEFINITIVO_20260712.xlsx' es la fuente canónica para resultados de visibilidad, detección, artifacts públicos, falsos positivos, Wazuh y trazabilidad (García-Amorena Reina, 2026e).
- F.2. Normalizados de visibilidad. Los CSV y JSON derivados de iteraciones anteriores se conservan únicamente como trazabilidad histórica. Ninguna cifra, tabla o figura final de esta memoria se fija a partir de esos normalizados; la fuente cuantitativa exclusiva es el Excel definitivo de visibilidad.
- F.3. Excel final de benchmark. '<RUTA_EVIDENCIAS>/TFM_BENCHMARK_RENDIMIENTO_VELOCIRAPTOR_DEF

INITIVO_20260712.xlsx` es la fuente canónica para escenarios, runs válidos y métricas agregadas de CPU y RAM (García-Amorena Reina, 2026d).

- F.4. Normalizados de benchmark. Los CSV derivados de iteraciones anteriores se conservan únicamente como trazabilidad histórica. Las cifras y figuras finales de rendimiento proceden exclusivamente del Excel definitivo de benchmark.
- F.5. Calidad de datos. El dataset final registra 102 fuentes con SHA-256, 9 TEC planificadas/ejecutadas/OK, 10 FP planificados/OK, 379 filas CLIENT_EVENT reales incluidas y 0 hits definitivos en FP (García-Amorena Reina, 2026e).
- F.6. Limitación documental TEC-009. El summary del runner independiente documenta HTTP 200, UploadSucceeded=True, SHA-256 y bytes transferidos. Es ground truth de ejecución y no una alerta CLIENT_EVENT ni una detección de TrackNetwork. Si no aparece receiver_log.jsonl o el ZIP recibido, se mantiene como limitación documental secundaria.
- F.7. Limitación documental Wazuh. La lectura final de Wazuh debe apoyarse en el Excel consolidado y normalizados. La ruta histórica `ENTREGA\_WAZUH\_VISIBILIDAD\_20260622\_114027` aparece registrada en dataset_summary, pero no se conserva completa en la ruta revisada actual.

11.8. Anexo G. Resultados completos de visibilidad, detección y falsos positivos

- G.1. Resultado global. La campaña real consolidada registra 9/9 técnicas TEC-001 a TEC-009 detectadas mediante CLIENT_EVENT y 379 filas CLIENT_EVENT emitidas/exportadas, incluidas emisiones repetidas, dentro de la ventana de ejecución (García-Amorena Reina, 2026e).
- G.2. Distribución por perfiles. P1=19, P2=118, P3=109 y P4=133. Estos valores son perfiles de artifacts/severidad, no prioridades documentales.
- G.3. Distribución por técnica. TEC-001=42 filas, TEC-002=214, TEC-003=30, TEC-004=37, TEC-005=14, TEC-006=18, TEC-007=5, TEC-008=4 y TEC-009=15. La diferencia de volumen no equivale por sí sola a mayor o menor calidad de detección; depende de fuentes, filtros y comportamiento generado.
- G.4. Fuentes observadas por técnica. El dataset normalizado registra eventos Sysmon y PowerShell 4104 por técnica. Ejemplos: TEC-001 incluye 4 eventos Sysmon y 3 de PowerShell 4104; TEC-005 incluye 6 Sysmon y 4 System 7045; TEC-009 incluye 7 Sysmon y 1 PowerShell 4104.
- G.5. Falsos positivos. La campaña FP registra 10/10 acciones benignas ejecutadas y 0 hits definitivos; 82 filas no reconciliadas fueron excluidas y no se computaron como falsos positivos

definitivos. Se documentan filas CLIENT_EVENT a revisar por mezcla de fuentes, pero el resultado final conserva TotalHits=0 tras aplicar filtros de ventana/carpeta.

- G.6. Interpretación. El resultado 9/9 solo aplica al subconjunto TEC-001 a TEC-009, al laboratorio Windows, a la telemetría configurada y a los artifacts validados. No debe extrapolarse a toda ATT&CK Enterprise ni a producción.
- G.7. Defensa técnica. La defensa de resultados debe apoyarse en la relación técnica -> ejecución controlada -> fuente de telemetría -> artifact CLIENT_EVENT -> perfil -> evidencia conservada -> limitación documentada.

11.9. Anexo H. Benchmark de rendimiento

- H.1. Fuente válida. El benchmark válido es 'TFM_BENCHMARK_RENDIMIENTO_VELOCIRAPTOR_DEFINITIVO_20260712.xlsx' (García-Amorena Reina, 2026d).
- H.2. Diseño. El benchmark contiene 3 escenarios y 3 repeticiones por escenario: BASELINE_NO_VR, VR_IDLE y VR_TEC_RUNNER. El resultado incluye 9/9 runs válidos.
- H.3. Exclusión de SERVER_GUI. El benchmark principal mide exclusivamente el impacto del cliente Velociraptor sobre el endpoint. SERVER_GUI se registra separadamente y se excluye del cálculo principal porque pertenece al servidor y no al cliente evaluado. Los flags empleados son IncludeServerGuiInTotal=False y ServerGuiExcludedFromClientMetrics=True.
- H.4. CPU. VR_TEC_RUNNER registra CPU media aproximada de 0,95 % y pico observado del 54 %. BASELINE_NO_VR y VR_IDLE aparecen con media 0 % en el resumen consolidado.
- H.5. Memoria. VR_IDLE registra 57,48 MB de RAM media y VR_TEC_RUNNER 57,42 MB, con pico de 57,97 MB. La interpretación correcta es consumo medible pero bajo y estable en el laboratorio, no ausencia de consumo.
- H.6. RunnerStillRunningAtEnd. Se interpreta como WARN no bloqueante cuando no invalida los 9/9 runs ni los controles de calidad del Excel.
- H.7. Alcance. El benchmark mide el coste operativo agregado del cliente Velociraptor. No mide calidad de detección, cobertura, falsos positivos ni consumo individual de los artifacts P1, P2, P3 o P4.
- H.8. Limitación. Los resultados no se extrapolan directamente a producción. Para entornos reales habría que repetir con más endpoints, cargas prolongadas, hardware distinto y retención/eventos representativos.

11.10. Anexo I. Validación comparativa con Wazuh

- I.1. Finalidad. Wazuh se mantiene como validación comparativa HIDS/SIEM. No sustituye a Velociraptor y no se trata como unidad equivalente directa.
- I.2. Modelo Velociraptor. Velociraptor trabaja con artifacts, VQL, CLIENT_EVENT y evidencia host-based. La detección defendible ocurre en CLIENT_EVENT; SERVER_EVENT enruta, persiste o notifica.
- I.3. Modelo Wazuh. Wazuh trabaja con logs, rules, archives y alerts. La comparación permite contrastar visibilidad, modelo operativo, reglas y alertabilidad centralizada (Wazuh, s. f.-a; Wazuh, s. f.-b).
- I.4. Fuentes Wazuh. La memoria documenta PowerShell Operational, Sysmon, Security y System como fuentes remitidas por el agente Windows. Esta configuración permite observar ejecución de scripts, procesos, creación de ficheros y servicios.
- I.5. Reglas custom. El ruleset efectivo de la campaña fue tfm_wazuh_custom_rules_v2.xml. El Excel consolidado conserva los resultados finales, los identificadores 110301, 110402, 110203 y 110202, y el gap 110201=0 para TEC-009.
- I.6. Resultados consolidados. Wazuh base acredita 3/9 técnicas —TEC-002, TEC-004 y TEC-006— y Wazuh custom 4/9 —TEC-001, TEC-005, TEC-007 y TEC-008—. En la campaña TEC se registran 54 alertas de reglas base y 56 alertas 110xxx, 110 en total; son volúmenes contextuales, no unidades de cobertura. La unión complementaria alcanza 7/9 y no constituye una quinta solución.
- I.7. Gap 110201. Wazuh conserva una limitación en la regla 110201 para TEC-009: 110201=0. Esto evidencia que la cobertura depende de la lógica de reglas y de la fuente de eventos disponible.
- I.8. Ruido operativo. La coexistencia Velociraptor + Wazuh puede generar ruido por actividad del propio flujo de alertado, como escritura JSONL, ConvertFrom-Json, FromBase64String, Invoke-RestMethod o envío a Discord. Esa coexistencia se trata como experimento de ruido, no como campaña principal de detección.

11.11. Anexo J. TEC-009 - Exfiltración HTTP controlada en laboratorio

- J.1. Formulación correcta. TEC-009 debe formularse como exfiltración HTTP controlada en laboratorio o como staging, archivado ZIP y exfiltración HTTP controlada hacia host receptor.

- J.2. Técnicas MITRE asociadas. TEC-009 se mapea a T1074.001 Local Data Staging, T1560.001 Archive via Utility y T1048.003 Exfiltration Over Unencrypted Non-C2 Protocol (MITRE, s. f.-g; MITRE, s. f.-a; MITRE, s. f.-i).
- J.3. Evidencia mínima. El ground truth del runner incluye preparación de datos dummy, creación del ZIP, ReceiverReachable=True, HTTPStatus=200, UploadSucceeded=True, SHA-256 local y bytes transferidos. La detección Velociraptor se acredita por las señales CLIENT_EVENT host-based; ambas capas se documentan por separado.
- J.4. Limitación documental secundaria. Si receiver_log.jsonl o el ZIP recibido no aparecen en el paquete revisado, la conservación del receptor final queda limitada. El HTTP 200 del summary sigue siendo evidencia del runner independiente, no una alerta CLIENT_EVENT.
- J.5. Redacción metodológica. La formulación correcta separa la ejecución HTTP controlada, acreditada por el runner, de la detección host-based custom. No se atribuye el HTTP 200 al Router, a Discord, a TrackNetwork ni a una alerta CLIENT_EVENT.
- J.6. Interpretación HIDS. TEC-009 expone el límite entre telemetría host y evidencia de red. TrackNetwork observó la conexión, pero PID=0 y ProcInfo vacío impidieron una atribución fiable de proceso. El ground truth del runner completa la ejecución experimental sin ampliar artificialmente la cobertura de detección Velociraptor.

11.12. Anexo K. Glosario técnico

- K.1. Artifact Velociraptor. Unidad YAML que empaqueta consultas VQL, parámetros y sources para recopilar o monitorizar información del endpoint.
- K.2. CLIENT_EVENT. Artifact/event artifact que se ejecuta en el endpoint y produce eventos de monitorización continua. En este TFM es la capa donde ocurre la detección defendible.
- K.3. SERVER_EVENT. Artifact del lado servidor. Enruta, normaliza, persiste o notifica eventos generados por CLIENT_EVENT. No detecta por sí mismo.
- K.4. JSONL. Formato de salida o persistencia línea a línea. No es fuente primaria de detección.
- K.5. Discord. Canal externo de notificación. No es fuente primaria de detección ni evidencia suficiente por sí sola.
- K.6. P1/P2/P3/P4. Perfiles de artifacts/severidad y visibilidad: P1 Critical, P2 High, P3 Medium, P4 Low. No deben confundirse con prioridades P1/P2/P3/P4 de planificación documental.
- K.7. Visibilidad. Capacidad de observar eventos o cambios en el endpoint. No equivale automáticamente a detección.
- K.8. Detección. Clasificación reproducible de un evento o patrón por lógica de artifact, asociada a una técnica y respaldada por evidencia.

- K.9. Alerta. Señal operativa derivada de una detección y normalizada con severidad, destino o canal de salida.
- K.10. Evidencia forense. Datos conservados que permiten reconstruir o defender una observación: eventos, logs, hashes, capturas, CSV, JSONL, summaries o ficheros generados.
- K.11. Benchmark. Medición de coste operativo en CPU/RAM/disco bajo escenarios definidos. No mide calidad de detección.
- K.12. Ground truth. Verdad de laboratorio basada en ejecución planificada, runner, tiempo, técnica esperada y evidencias generadas.
- K.13. Falso positivo. Señal generada por actividad benigna. En este TFM se evalúa en campaña separada FP-001 a FP-010.
- K.14. Wazuh base/custom. Wazuh base corresponde a comportamiento sin reglas TFM específicas; Wazuh custom incluye reglas TFM añadidas para aproximar perfiles de severidad, sin equivalencia directa con artifacts Velociraptor.

11.13. Anexo L. Checklist de defensa metodológica

- L.1. Alcance válido es TEC-001 a TEC-009 en laboratorio Windows.
- L.2. No mezclar benchmark con calidad de detección. El benchmark mide coste operativo; la cobertura se evalúa en el capítulo de resultados.
- L.3. Evitar redacciones que presenten consumo nulo. La redacción correcta es consumo medible pero bajo y estable en el laboratorio evaluado.
- L.4. No afirmar que Wazuh y Velociraptor son equivalentes directos. Comparan modelos operativos distintos.
- L.5. Formular SERVER_EVENT únicamente como capa de enrutamiento, normalización, persistencia o notificación de eventos generados por CLIENT_EVENT.
- L.6. No usar JSONL ni Discord como fuente primaria de detección. Son salidas externas o mecanismos de explotación/notificación.
- L.7. No tratar P4 como alerta crítica. P4 aporta visibilidad básica y debe interpretarse con contexto.
- L.8. No tratar rutas, nombres de scripts o etiquetas TEC como IOC. Son artefactos de laboratorio y trazabilidad interna.
- L.9. No ocultar limitaciones. Deben mantenerse las limitaciones de laboratorio, telemetría, Wazuh 110201, receiver final de TEC-009 si aplica y no extrapolación a producción.
- L.10. Mantener separadas ejecución, visibilidad, detección, alerta, salida externa, evidencia, falsos positivos, benchmark y comparación con Wazuh.

11.14. Anexo M. Declaración responsable sobre el uso de inteligencia artificial

Yo, Julio García-Amorena Reina, autor del Trabajo Fin de Máster titulado «Análisis de la capacidad de Velociraptor para la detección de ataques HIDS de la matriz MITRE ATT&CK Enterprise», declaro bajo mi responsabilidad que durante su elaboración se han utilizado herramientas de inteligencia artificial generativa únicamente como apoyo auxiliar.

En concreto, estas herramientas se han empleado para:

Revisar la redacción, ortografía, claridad y estructura de determinados apartados.

Proponer mejoras de estilo y organización del contenido.

- Ayudar en la revisión de scripts, comandos y procedimientos técnicos.
- Facilitar la interpretación, síntesis y presentación de resultados obtenidos durante las pruebas.
- Apoyar la elaboración de tablas, gráficos y documentación complementaria.
- El uso de estas herramientas no ha sustituido el trabajo técnico, experimental ni intelectual realizado por el autor. La definición de la metodología, el diseño y ejecución de las pruebas, la recopilación de evidencias, el análisis de los resultados y la formulación de las conclusiones han sido realizados y supervisados personalmente.

Asimismo, declaro que:

- Todas las respuestas generadas mediante inteligencia artificial han sido revisadas, verificadas y, cuando ha sido necesario, corregidas.
- No se han incorporado deliberadamente datos, referencias bibliográficas, resultados experimentales o evidencias sin comprobar su autenticidad.
- Las herramientas de inteligencia artificial no han sido consideradas fuentes académicas ni técnicas por sí mismas.
- Las decisiones finales sobre el contenido incluido en la memoria corresponden exclusivamente a mí, como autor.

Las principales herramientas utilizadas han sido ChatGPT y OpenAI Codex, empleadas como apoyo para la redacción, revisión documental, análisis técnico y asistencia en tareas de programación.